大数据算法

BIG DATA





大 数 据 算 法

CONTENTS

01 大数据挖掘算法概述
Introduction

02 分类算法
Sorting algorithms

03 聚类算法
Clustering algorithms

04 关联规则算法
Association rule algorithms

05 预测算法
Prediction algorithms

1.1 数据挖掘概念



- 20世纪80年代末，数据挖掘（Data Mining，DM）提出
- 1989年，KDD（Knowledge Discovery from Data）名词正式开始出现
- 1995年，“数据挖掘” 流传

- 从科学定义分析，数据挖掘是从大量的、有噪声的、不完全的、模糊和随机数据中，提取出隐含在其中的、人们事先不知道的、具有潜在利用价值的信息和知识的过程
- 从技术角度分析，数据挖掘利用一系列相关算法和技术，从大数据中提取出行业或公司所需要的、有实际价值知识的过程。知识表示形式可以是概念、规律、规则与模式等
- 准确地说，数据挖掘是整个知识发现流程中的一个具体步骤，也是知识发现过程中最重要的核心步骤

重庆大学 CHONGQING UNIVERSITY

1.1 数据挖掘概念

数据收集

- 根据需要对数据进行采集并存入数据库

数据清洗

- 对不完整、噪声、缺失数据进行处理，得到一致的数据

数据变换

- 将数据变换为易于挖掘的格式



数据挖掘的基本流程

挖掘过程

- 选择合适的挖掘算法对数据进行挖掘从而得到有价值信息

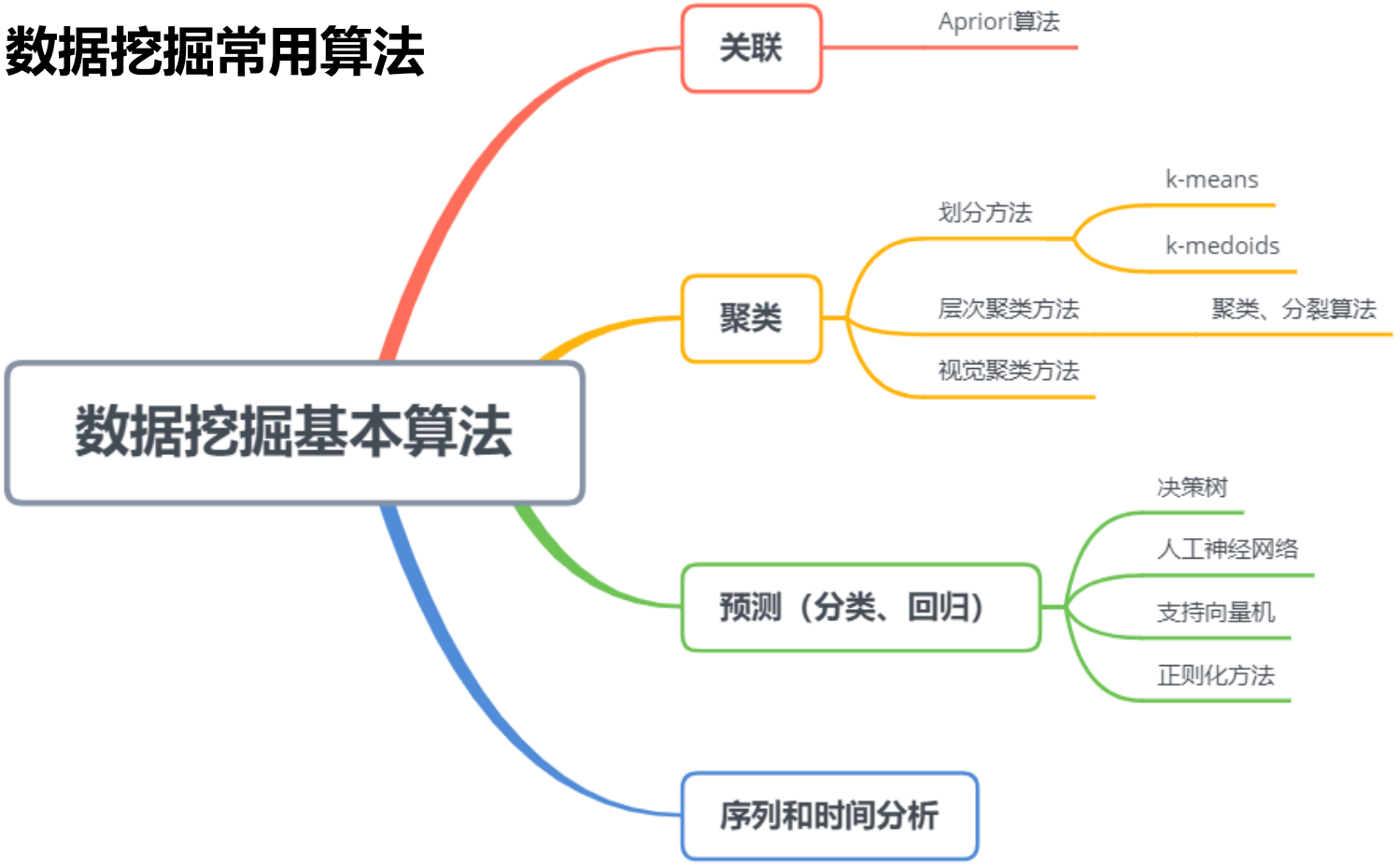
模式评估

- 由行业专家来验证数据挖掘结果的正确性

知识表示

- 将数据挖掘所得到的分析信息以可视化的方式呈现给用户

1.2 数据挖掘常用算法

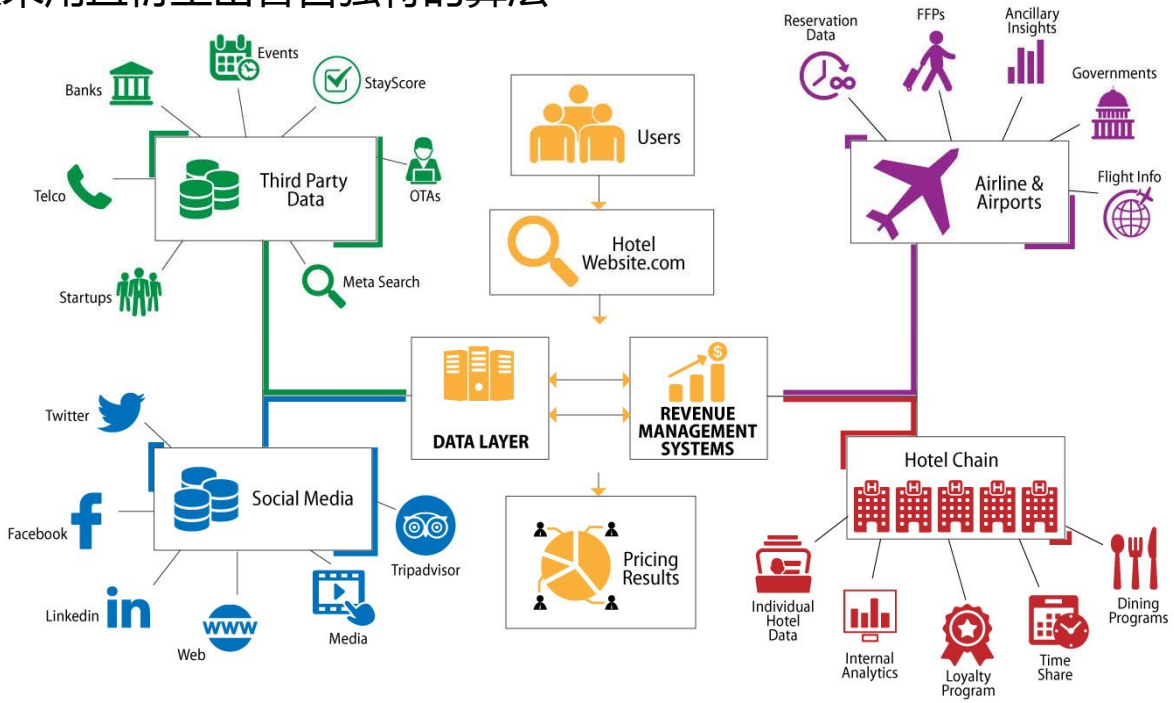

数据挖掘基本算法
关联
Apriori算法
聚类
划分方法
k-means
k-medoids
层次聚类方法
聚类、分裂算法
视觉聚类方法
预测（分类、回归）
决策树
人工神经网络
支持向量机
正则化方法
序列和时间分析

1.2 数据挖掘常用算法

- 分类
  - 数据挖掘方法中的一种重要方法就是分类，在给定数据基础上构建分类函数或分类模型，该函数或模型能够把数据归类为给定类别中的某一种类别，这就是分类的概念
- 聚类
  - 聚类也就是将抽象对象的集合分为相似对象组成的多个类的过程，聚类过程生成的簇称为一组数据对象的集合
- 关联规则
  - 关联规则属于数据挖掘算法中的一类重要方法，关联规则就是支持度与信任度分别满足用户给定阈值的规则
- 时间序列预测
  - 时间序列预测法是一种历史引申预测法，也即将时间数列所反映的事件发展过程进行引申外推，预测发展趋势的一种方法

1.3 数据挖掘算法应用场景

- 按照数据挖掘的应用场景分类，数据挖掘的应用主要涉及通信、股票、金融、银行、交通、商品零售、生物医学、精确营销、地震预测、工业产品设计等领域，在这些领域众多数据挖掘方法均被广泛采用且衍生出各自独特的算法



1.3 数据挖掘算法应用场景

■ 数据挖掘在电信行业的应用

- 数据挖掘广泛应用在电信行业，可以帮助企业制定**个性化**的**服务与资费标准**、防止欺诈、优惠政策，为公司决策者提供可靠的决策依据，为**市场营销、客户服务、全网业务、经营决策**等提供有效的数据支撑，进一步完善了国内电信公司对省、市电信运营的指导，在业务运营中发挥重要的作用，从而为精细化运营提供技术与数据的基础

■ 数据挖掘在商业银行中的应用

- 在美国银行业与金融服务领域数据挖掘技术的应用十分广泛，由于金融业务的分析与评估往往需要大数据的支撑，从中可以发现客户的**信用评级**与**潜在客户**等有价值的信息，可成功地预测客户的需求

1.3 数据挖掘算法应用场景

■ 数据挖掘在信息安全中的应用

- 利用机器学习与数据挖掘等前沿技术与处理方法对**入侵检测**的数据进行自动分析，提取出尽可能多的隐藏安全信息，从中抽象出与安全有关的数据特征，从而能够发现未知的入侵行为。数据挖掘技术可以建立一种具备自适应性、自动的、系统与良好扩展性的入侵检测系统，能够解决传统入侵检测系统适应性与扩展性较差的弱点，大幅度提高入侵检测系统效能

■ 数据挖掘在科学探索中的应用

- 近年来，数据挖掘技术开始逐步应用到科学探索研究中。例如
  - 使用概率论模型对**蛋白质序列**进行多序列联配建模
  - 特定数据挖掘技术研究**基因数据库搜索**技术
  - 在被认为是人类征服顽疾的最有前途的攻关课题“DNA序列分析”过程中，由于DNA序列的构成多种多样，数据挖掘技术的应用可以为发现疾病蕴藏的基因排列信息提供新方法

1.4 数据挖掘工具

根据适用的范围，数据挖掘工具分为两类：

- 专用挖掘工具
  - 专用数据挖掘工具针对某个特定领域的问题提供解决方案，在涉及算法的时候充分考虑数据、需求的特殊性
- 通用挖掘工具
  - 对任何应用领域，专业的统计研发人员都可以开发特定的数据挖掘工具









1.4 数据挖掘工具

- Weka
  - 公开的数据挖掘工作平台，集成大量能承担数据挖掘任务的机器学习算法，包括对数据进行预处理、分类、回归、聚类、关联规则，以及交互式界面上的可视化
- SPSS
  - SPSS采用类似Excel表格的方式输入与管理数据，数据接口较为通用，能方便地从其他数据库中读入数据。突出的特点是操作界面友好，且输出结果美观
- Clementine
  - Clementine提供出色、广泛的数据挖掘技术，确保用恰当的分析技术来处理相应的商业问题，以应对随时出现的问题
- RapidMine
  - RapidMiner并不支持分析流程图方式，当包含的运算符比较多时就不容易查看；具有丰富的数据挖掘分析和算法功能，常用于解决各种商业关键问题



大 数 据 算 法

CONTENTS

01 大数据挖掘算法概述
Introduction

02 分类算法
Sorting algorithms

03 聚类算法
Clustering algorithms

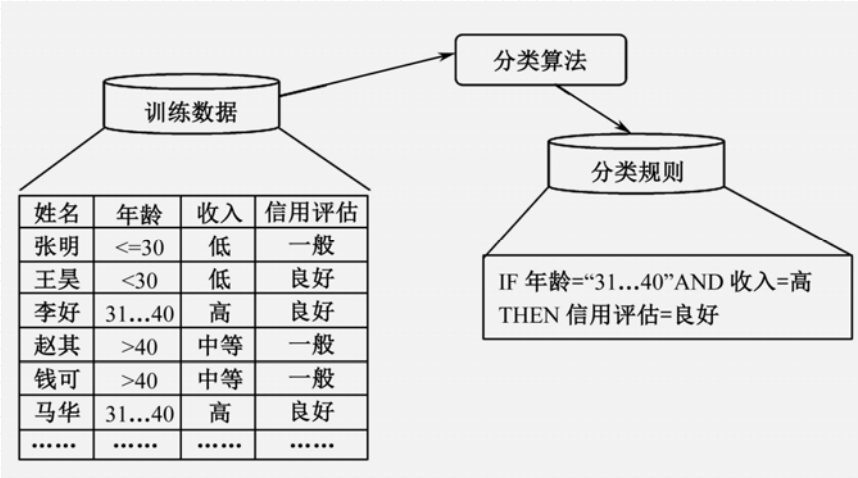
04 关联规则算法
Association rule algorithms

05 预测算法
Prediction algorithms

2.1 分类算法简介

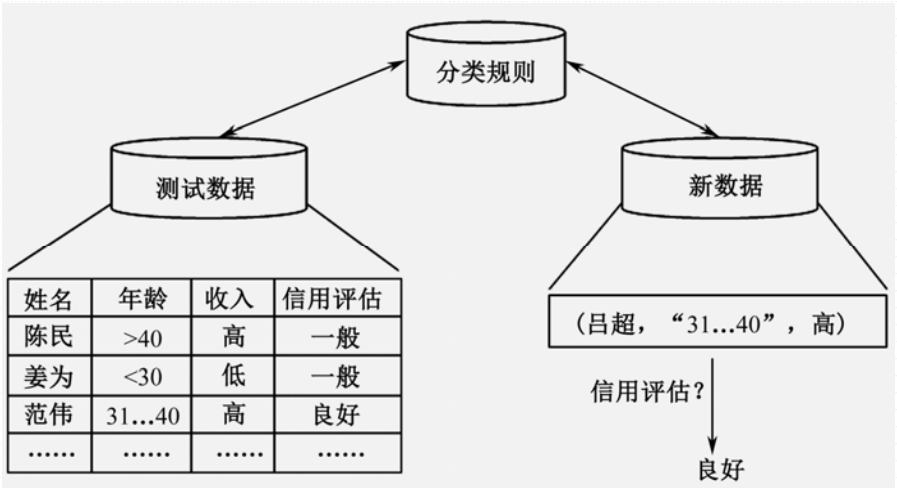
- 分类是一种重要的数据分析形式，根据重要数据类的特征向量值及其他约束条件，构造分类函数或分类模型（分类器），目的是根据数据集的特点把未知类别的样本映射到给定类别中

- 数据分类过程主要包括两个步骤，即学习和分类



姓名	年龄	收入	信用评估
张明	<=30	低	一般
王昊	<30	低	良好
李好	31...40	高	良好
赵其	>40	中等	一般
钱可	>40	中等	一般
马华	31...40	高	良好
.....	.....	.....	.....

1.建立模型学习



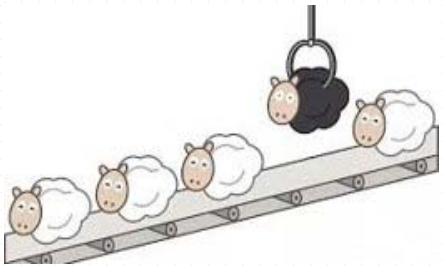
姓名	年龄	收入	信用评估
陈民	>40	高	一般
姜为	<30	低	一般
范伟	31...40	高	良好
.....	.....	.....	.....

2.分类

2.1 分类算法简介

- 分类分析在数据挖掘中是一项比较重要的任务，目前在商业上应用最多。
- 分类的目的是从历史数据记录中自动推导出对给定数据的推广描述，学会一个分类函数或分类模型（也常称作**分类器**），该模型能把数据库中的数据项映射到给定类别中某一个类中。
- 为建立模型而被分析的数据元组形成**训练数据集**，由一组数据库记录或元组构成，每个元组是一个由有关字段（又称属性或特征）值组成的**特征向量**，此外，每一个训练样本都有一个预先定义的类别标记，由一个被称为**类标签**的属性确定。
- 一个具体样本的形式可表示为$\{X_1,\cdots,X_n,C\}$，其中 X_n 表示字段值，C表示类别。

分类又称为有监督的学习



2.2 贝叶斯决策与分类器

算法思想

贝叶斯学派思想可概括为：**先验概率+数据=后验概率**

我们在实际问题中需要得到的**后验概率**，可以通过**先验概率和数据**一起综合得到。

贝叶斯学派**需要假设先验分布的模型**，如正态分布，beta分布等。

这个假设一般没有特定依据，因此一直被频率学派认为很荒谬。虽然难以从严密的数学逻辑里推出贝叶斯学派的逻辑，但是在很多实际应用中，贝叶斯理论很好用，比如：垃圾邮件分类，文本分类。

2.2 贝叶斯决策与分类器

算法流程

输入：训练数据 $T=\{(x_1,y_1),(x_2,y_2),\cdots,(x_N,y_N)\}$，其中 $x_i=(x_i^{(1)},x_i^{(2)},\cdots,x_i^{(n)})^{\mathrm{T}}$，$x_i^{(j)}$ 是第 i 个样本的第 j 个特征，$x_i^{(j)}\in\{a_{j1},a_{j2},\cdots,a_{jS_j}\}$，$a_{jl}$ 是第 j 个特征可能取的第 l 个值，$j=1,2,\cdots,n$，$l=1,2,\cdots,S_j$，$y_i\in\{c_1,c_2,\cdots,c_K\}$；实例 x；

输出：实例 x 的分类.

（1）计算先验概率及条件概率

$$P(Y=c_k)=\frac{\sum_{i=1}^{N}I(y_i=c_k)}{N},\quad k=1,2,\cdots,K$$

$$P(X^{(j)}=a_{jl}\mid Y=c_k)=\frac{\sum_{i=1}^{N}I(x_i^{(j)}=a_{jl},y_i=c_k)}{\sum_{i=1}^{N}I(y_i=c_k)}$$

$$j=1,2,\cdots,n;\quad l=1,2,\cdots,S_j;\quad k=1,2,\cdots,K$$

（2）对于给定的实例 $x=(x^{(1)},x^{(2)},\cdots,x^{(n)})^{\mathrm{T}}$，计算

$$P(Y=c_k)\prod_{j=1}^{n}P(X^{(j)}=x^{(j)}\mid Y=c_k),\quad k=1,2,\cdots,K$$

（3）确定实例 x 的类

$$y=\arg\max_{c_k}P(Y=c_k)\prod_{j=1}^{n}P(X^{(j)}=x^{(j)}\mid Y=c_k)$$

2.2.1 相关基础数学知识

■ 条件概率

事件A 在另外一个事件B 已经发生条件下的发生概率，称为在B 条件下A 的概率。表示 $P(A|B)$

$$P(A|B)=\frac{P(A\cap B)}{P(B)}$$

■ 联合概率

联合概率表示两个事件共同发生的概率。 A 与B 的联合概率表示为 $P(AB)$、 $P(A,B)$或者 $P(A\cap B)$

■ 贝叶斯定理

贝叶斯定理用来描述两个条件概率之间的关系，例如，$P(A|B)$ 与 $P(B|A)$ 根据乘法法则：

$$P(A\cap B)=P(A)P(B|A)=P(B)P(A|B) \qquad P(A|B)=\frac{P(A)P(B|A)}{P(B)}$$

2.2.1 相关基础数学知识

■ 全概率公式

全概率公式将对复杂事件A 的概率求解问题转化为在不同情况下发生的简单事件的概率的求和问题。

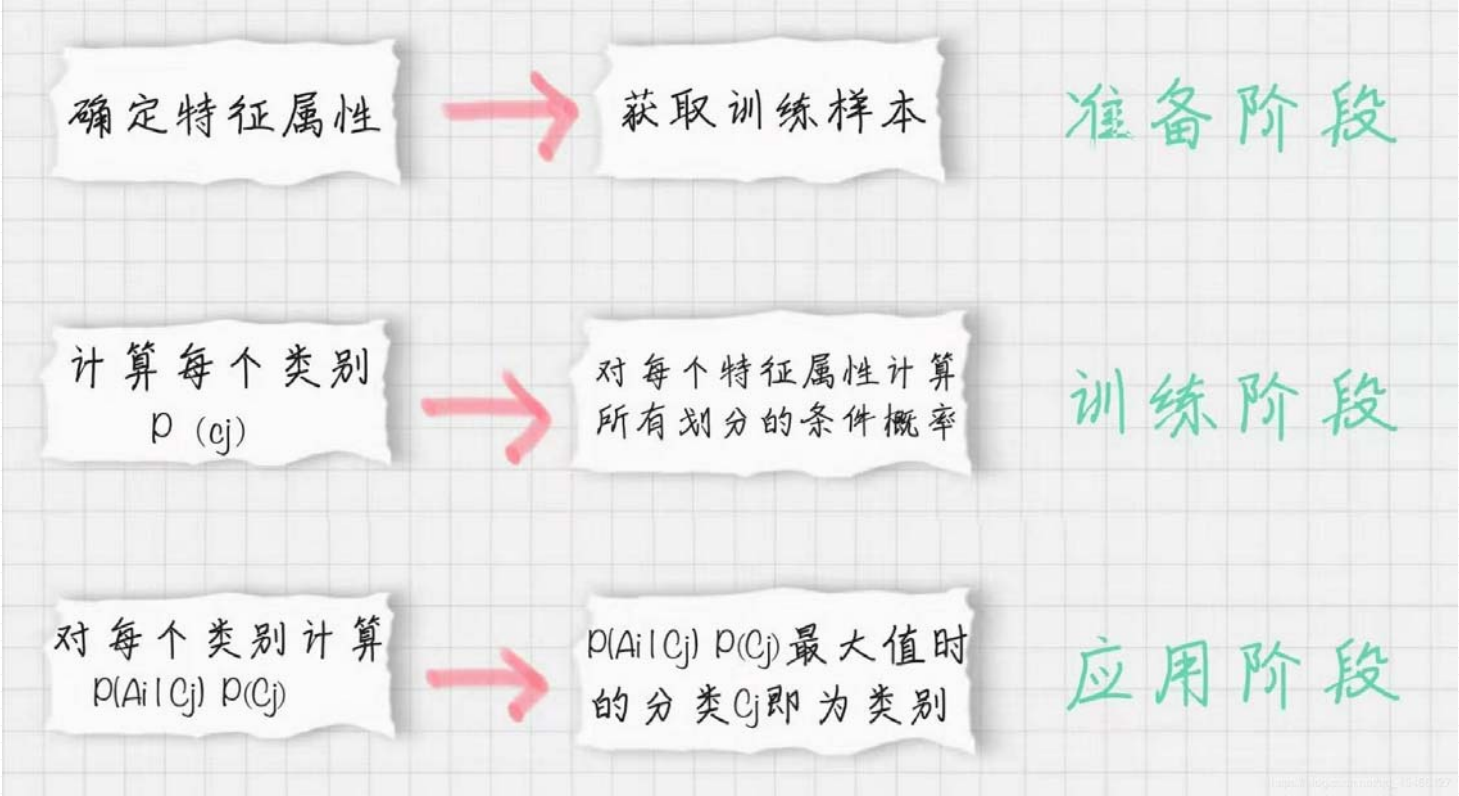
设 $B_1,\cdots,B_n$ 构成一个完备事件组，即它们两两互不相容，其和为全集，且

$$P(B_i)\geqslant 0(i=1,\cdots,n)$$

则事件A的概率为：

$$P(A)=P(A|B_1)P(B_1)+\cdots+P(A|B_n),P(B_n)=\sum_{i=1}^{n}P(A|B_i)P(B_i)$$

2.2.2 贝叶斯分类实例


确定特征属性
获取训练样本
准备阶段
计算每个类别
P（cj）
对每个特征属性计算
所有划分的条件概率
训练阶段
对每个类别计算
P(Ai|Cj) P(Cj)
P(Ai|Cj) P(Cj)最大值时
的分类Cj即为类别
应用阶段

2.2.2 贝叶斯分类流程---实例（1/2）

- 离散数据案例
  - 判断：身高“高”，体重“中”，鞋码“中”，对应性别

编号	身高	体重	鞋码	性别
1	高	重	大	男
2	高	重	大	男
3	中	中	大	男
4	中	中	中	男
5	矮	轻	小	女
6	矮	轻	小	女
7	矮	中	中	女
8	中	中	中	女

- A代表属性，C代表类别，贝叶斯公式可得

$$P(C_j \mid A_1A_2A_3) = \frac{P(A_1A_2A_3 \mid C_j)P(C_j)}{P(A_1A_2A_3)}$$

- 假设Ai相互独立，则

$$P(A1A2A3|Cj)=P(A1|Cj)P(A2|Cj)P(A3|Cj)$$

- 因此可以求出

$$P(A1A2A3|C1)>P(A1A2A3|C2)$$

为男性概率较大

2.2.2 贝叶斯分类流程---实例（1/2）

■ 连续数据案例

➢ 判断：身高“180”，体重“120”，鞋码“41”，对应性别

编号	身高（CM）	体重（斤）	鞋码（欧码）	性别
1	183	164	45	男
2	182	170	44	男
3	178	160	43	男
4	175	140	40	男
5	160	88	35	女
6	165	100	37	女
7	163	110	38	女
8	168	120	39	女

- A代表属性，C代表类别，贝叶斯公式可得

$$P(C_j \mid A_1A_2A_3) = \frac{P(A_1A_2A_3 \mid C_j)P(C_j)}{P(A_1A_2A_3)}$$

- 假设Ai相互独立，则

$$P(A1A2A3|C1)=P(A1|C1)P(A2|C1)P(A3|C1)$$
$$P(A1A2A3|C2)=P(A1|C2)P(A2|C2)P(A3|C2)$$

- 因此可以求出

P(A1A2A3|C1) = 4.9169e-6
P(A1A2A3|C1) = 2.7244e-9

为男性概率更大

打网球实例：估计 P(xi|C)

Outlook	Temperature	Humidity	Windy	Class
sunny	hot	high	false	N
sunny	hot	high	true	N
overcast	hot	high	false	P
rain	mild	high	false	P
rain	cool	normal	false	P
rain	cool	normal	true	N
overcast	cool	normal	true	P
sunny	mild	high	false	N
sunny	cool	normal	false	P
rain	mild	normal	false	P
sunny	mild	normal	true	P
overcast	mild	high	true	P
overcast	hot	normal	false	P
rain	mild	high	true	N

P(p) = 9/14
P(n) = 5/14

outlook	
P(sunny|p) = 2/9	P(sunny|n) = 3/5
P(overcast|p) = 4/9	P(overcast|n) = 0
P(rain|p) = 3/9	P(rain|n) = 2/5
temperature	
P(hot|p) = 2/9	P(hot|n) = 2/5
P(mild|p) = 4/9	P(mild|n) = 2/5
P(cool|p) = 3/9	P(cool|n) = 1/5
humidity	
P(high|p) = 3/9	P(high|n) = 4/5
P(normal|p) = 6/9	P(normal|n) = 2/5
windy	
P(true|p) = 3/9	P(true|n) = 3/5
P(false|p) = 6/9	P(false|n) = 2/5

打网球实例: 分类 X

X = <rain, hot, high, false>

P(X|p)·P(p) =
P(rain|p)·P(hot|p)·P(high|p)·P(false|p)·P(p) = 3/9·2/9·3/9·6/9·9/14 = 0.010582

P(X|n)·P(n) =
P(rain|n)·P(hot|n)·P(high|n)·P(false|n)·P(n) = 2/5·2/5·4/5·2/5·5/14 = 0.018286

样本 X 通过类 n (don't play)来分类

2.3 决策树

- 决策树（Decision Tree）模型
  - 通过对训练样本学习，建立分类规则
  - 依据分类规则，实现对新样本的分类
  - 属于有监督式学习方法，有两类变量
    - 目标标量（输入变量）
    - 属性变量（输出变量）
- 决策树模型特点
  - 决策树擅长处理非数值类型，与神经网络智能处理数值类型比较，免去很多数据预测处理工作
  - 每个决策都要求分组之间“差异最大”，各种决策树算法之间主要区别就是对“差异”衡量方式的区别
- 决策树模型和一般统计分类模型区别
  - -决策树的分类是基于逻辑的，一般统计分类是基于非逻辑的

2.3 决策树

例子：如何挑选西瓜[1]

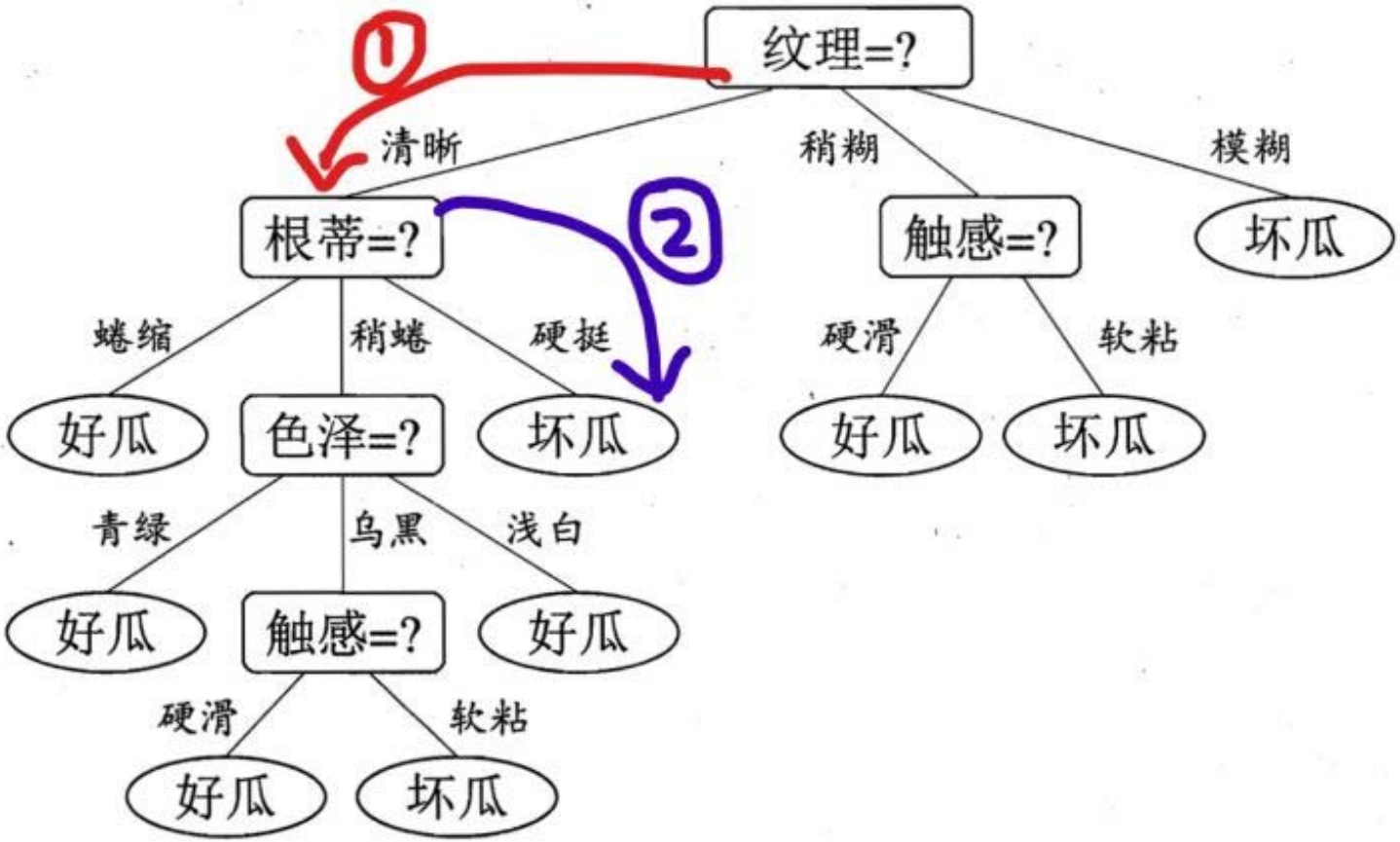
基本思路：从根节点开始，根据属性判断往下走，当达到叶子节点时，得出结论。

例如：

<纹理清晰，根蒂硬挺>

↓

坏瓜



[1]来自知乎：https://zhuanlan.zhihu.com/p/26760551

2.3 决策树典型算法—— ID3 、C4.5与C5.0算法

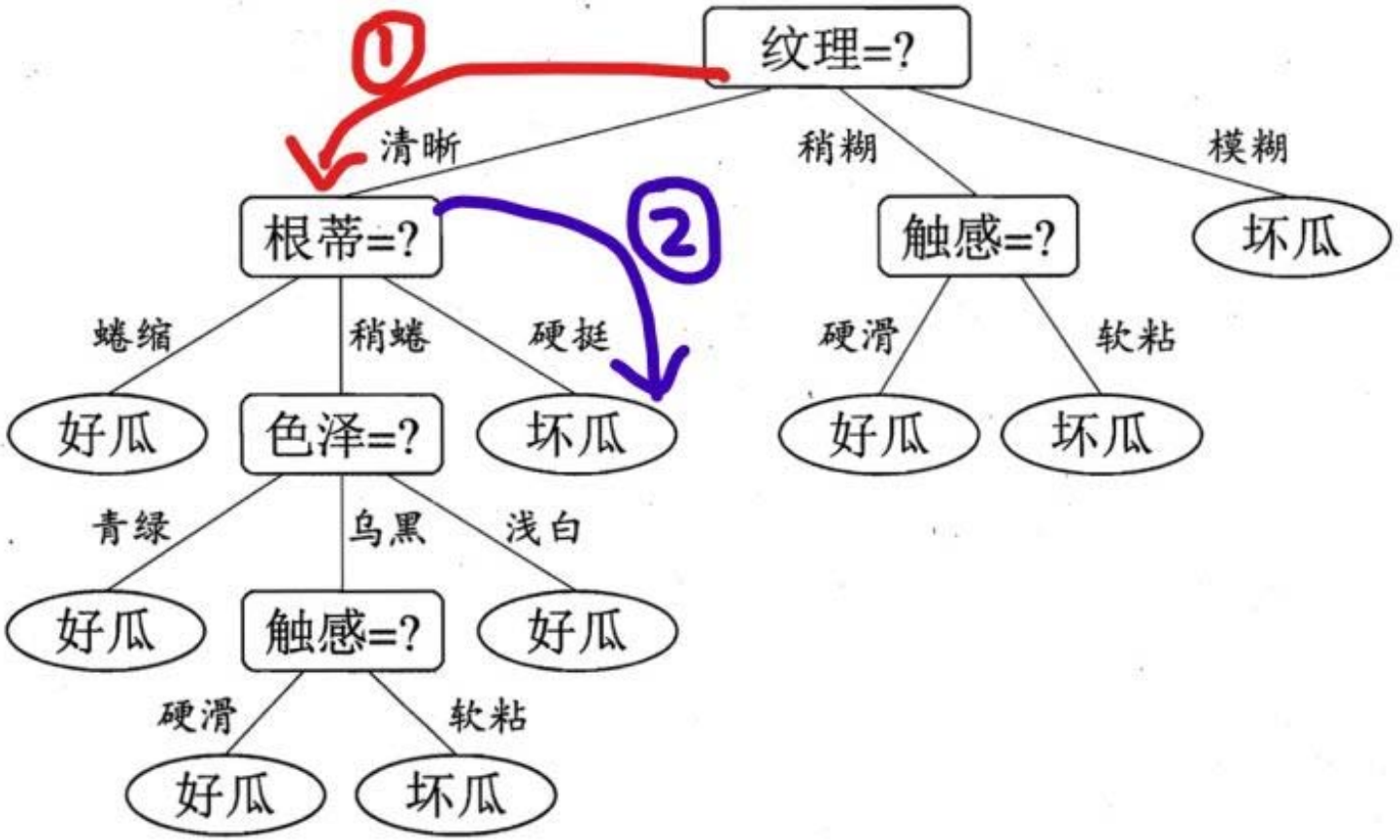
- ID3算法
  - 主要针对离散型属性数据，其后又不断的改进，形成C4.5。C4.5在ID3基础上增加了对连续属性的离散化
- C4.5
  - ID3算法的修订版，采用GainRatio（增益率通常指信息增益率，其表示节点的信息与节点分裂信息度量的比值，增益率通常作为属性选择的方法之一）来加以改进方法，选取有最大GainRatio的分割变量作为准则，避免ID3算法过度配适的问题
- C5.0
  - C4.5算法的修订版，适用于处理大数据集，采用Boosting（减小监督式学习中偏差的机器学习算法，面对的问题是Michael Kearns提出的：一组“弱学习者”的集合能否生成一个“强学习者”，主要是通过对样本集的操作获得样本子集，然后用弱分类算法在样本子集上训练生成一系列的基分类器）方式提高模型准确率，又称为BoostingTrees，在软件上计算速度比较快，占用的内存资源较少

2.3.1 ID3算法

决策树研究的就是**如何构建**出右图这样一颗分类树，从而实现分类。

决策树的关键：
在构建决策树的过程中，如何选择某个属性（纹理、根蒂、色泽、触感）进行分裂？

基本思想：
选择最优划分属性，希望划分后，分支结点的**“纯度”**越来越高。



2.3.1 ID3算法

- **信息熵**：熵这个概念最早起源于物理学，用于度量一个热力学系统的**无序程度**。在1948年，香农引入了信息熵（information entropy），用于刻画离散随机事件出现概率的不确定度，信息熵越高，则随机事件的不确定性越大。
- 对于有K个类别的分类问题来说，假定样本集合D中第 k 类样本所占的比例为p_k(k=1,2,...,K)，则样本集合D的信息熵定义为：

$$Ent(D) = -\sum_{k=1}^{K} p_k \cdot \log_2 p_k$$

- **例如**：考虑K=2的情况
  （1）当p1=0.1，p2=0.9时，信息熵=0.469
  （2）当p1=0.5，p2=0.5时，信息熵=1

思考：信息熵什么情况下最小？

显然，第（2）中情况下，不确定性最强，因此信息熵最大。
可以证明，K类情况下，当p1=p2=...=pk=1/K时，信息熵最大，为$\log_2 K$

2.3.1 ID3算法

- **信息增益**：在决策树的分类问题中，信息增益(information gain)是针对一个特定的分支标准（branching criteria）T，计算原有数据的信息熵与引入该分支标准后的**信息熵之差**，计算公式如下

$$Gain(D,a) = Ent(D) - \sum_{v=1}^{V} \frac{|D^v|}{|D|} Ent(D^v)$$

- **以右边天气数据为例**：每个样本包含如下四个特征：<Outlook, Temperature, Humidity, Windy)
  分类目标：是否出去玩(play vs not play)

- 表中一共14个样本，9个正样本，5个负样本，其信息熵如下：

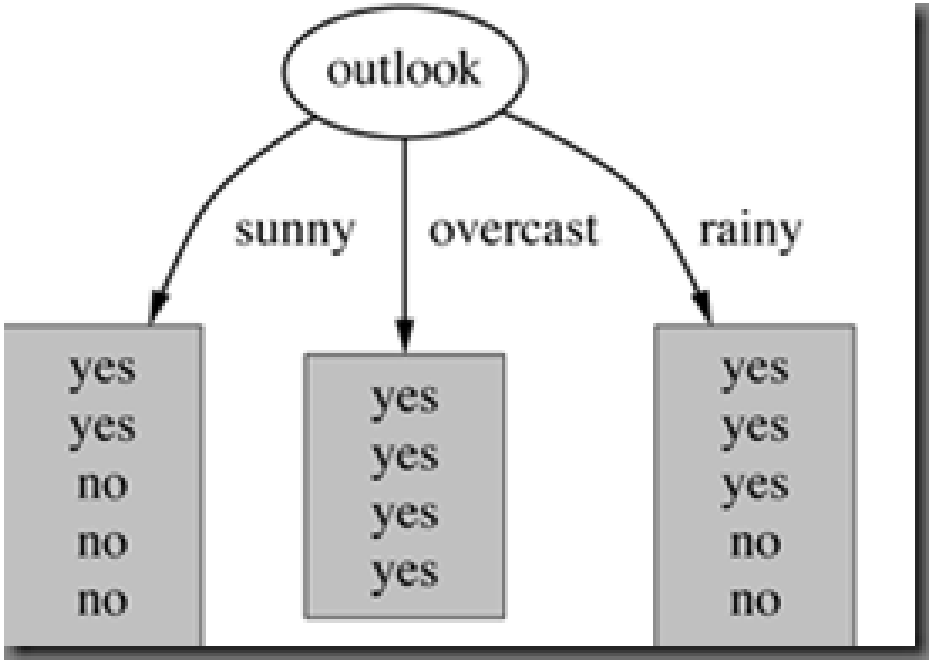
$$Ent(D) = -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} = 0.940286$$

表1 天气预报数据集例子

Outlook	Temperature	Humidity	Windy	Play?
sunny	hot	high	false	no
sunny	hot	high	true	no
overcast	hot	high	false	yes
rain	mild	high	false	yes
rain	cool	normal	false	yes
rain	cool	normal	true	no
overcast	cool	normal	true	yes
sunny	mild	high	false	no
sunny	cool	normal	false	yes
rain	mild	normal	false	yes
sunny	mild	normal	true	yes
overcast	mild	high	true	yes
overcast	hot	normal	false	yes
rain	mild	high	true	no

2.3.1 ID3算法

- 若以Outlook属性作为分支标准，根据sunny、overcast、rain这三个取值可将数据分为三类，如右图所示：
- 引入该分支标准后，数据被分成了3个分支，每个分支的信息熵计算如下：



$$Ent(D^{sunny}) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.970951$$

$$Ent(D^{overcast}) = -\frac{4}{4}\log_2\frac{4}{4} = 0$$

$$Ent(D^{sunny}) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} = 0.970951$$

- 引入该分支标准带来的信息增益为：

$$Gain(D,a) = 0.940286 - \frac{5}{14}\cdot 0.970951 - \frac{4}{14}\cdot 0 - \frac{5}{14}\cdot 0.970951 = 0.24695$$

2.3.1 ID3算法

■ **算法思想：**

首先计算出原始数据集的信息熵

↓

依次将数据中的每一个特征作为分支标准，并计算其相对于原始数据的信息增益

↓

选择最大信息增益的分支标准来划分数据，因为信息增益越大，区分样本的能力就越强，越具有代表性。

↓

重复上述过程从而生成一棵决策树

2.3.1 C5.0（1/3） -- 算法思想

- C5.0是经典决策树模型算法之一，可生成多分支决策树，目标变量为分类变量

- C5.0可以生成决策树（Decision Tree）或者规则集（rule sets），C5.0模型能根据能带来的最大信息增益（Information gain）的字段拆分样本

  - 第一次拆分确定样本的子集随后再次拆分，通常是根据另一个字段进行拆分，这一过程重复进行直到样本子集不能再被拆分为止

  - 最后，重新检验最低层次的拆分，对模型没有显著贡献的样本子集被剔除或者修剪

2.3.2 C5.0（2/3）--算法原理

- 信息熵：信息量的数学期望，是信源发出信息前的平均不确定性，也称先验熵
  - 信息$u_i(i=1,2,\ldots r)$的发生概率$P(u_i)$组成信源数学模型，$\sum P(u_i)=1$
  - 信息量(单位是bit，对的底数取2)：$I(u_i)=\log_2\frac{1}{P(u_i)}=-\log_2 P(u_i)$
  - 信息熵：(先验不确定性) $H(U)=\sum_i P(u_i)\log_2\frac{1}{P(u_i)}=-\sum_i P(u_i)\log_2 P(u_i)$
- 信息熵$H(U)$的性质
  - $H(U)=0$时，表示只存在唯一的可能性，不存在不确定性
  - 如果信源的k个信号有相同的发出概率，即所有的u_i有$P(u_i)=1/k$，$H(U)$达到最大，不确定性最大
  - $P(u_i)$差别越小，$H(U)$就越大；$P(u_i)$差别大，$H(U)$就越小

2.3.2 C5.0（2/3）--算法原理

- 决策树中熵的应用
  - 设S是一个样本集合，目标变量C有k个分类，freq(Ci, S)表示S中属于Ci类的样本数，|S|表示样本集合S的样本数。则集合S的信息熵定义为：

$$Info(S) = -\sum_{i=1}^{k}((freq(C_i,S)/|S|)\times \log_2(freq(C_i,S)/|S|))$$

  - 如果某属性变量T，有n个分类，则属性变量T引入后的条件熵定义为：

$$Info(T) = -\sum_{i=1}^{n}((|T_i|/|T|)\times Info(T_i))$$

  - 属性变量T带来的信息增益为：

$$Gain(T) = Info(S) - Info(T)$$

2.3.3 C5.0（3/3）--算法实例

T1	T2	T3	类
A	70	1	类1
A	90	1	类2
A	85	0	类2
A	95	0	类2
A	70	0	类1
B	90	1	类1
B	78	0	类1
B	65	1	类1
B	75	0	类1
C	80	1	类2
C	70	1	类2
C	80	0	类1
C	80	0	类1
C	96	0	类1

该组样本的熵为：

$$Info(S) = -\frac{9}{14}\log_2(\frac{9}{14}) - \frac{5}{14}\log_2(\frac{5}{14}) = 0.940$$

关于T1的条件熵为：

$$Info(T1) = \frac{5}{14}(-\frac{2}{5}\log_2(\frac{2}{5}) - \frac{3}{5}\log_2(\frac{3}{5})) + \frac{4}{14}(-\frac{4}{4}\log_2(\frac{4}{4}) - \frac{0}{4}\log_2(\frac{0}{4})) + \frac{5}{14}(-\frac{3}{5}\log_2(\frac{3}{5}) - \frac{2}{5}\log_2(\frac{2}{5})) = 0.694$$

T1带来的信息增益为：

$$Gain(T1) = 0.940 - 0.694 = 0.246$$

用判定树归纳分类

判定树

- 一个类似流程图的树结构
- 每个内部节点表示在一个属性上的测试
- 每个分支代表一个测试输出
- 每个树叶节点代表类或类分布

判定树的产生包含两个方面

- 树的构造
- 树的剪枝

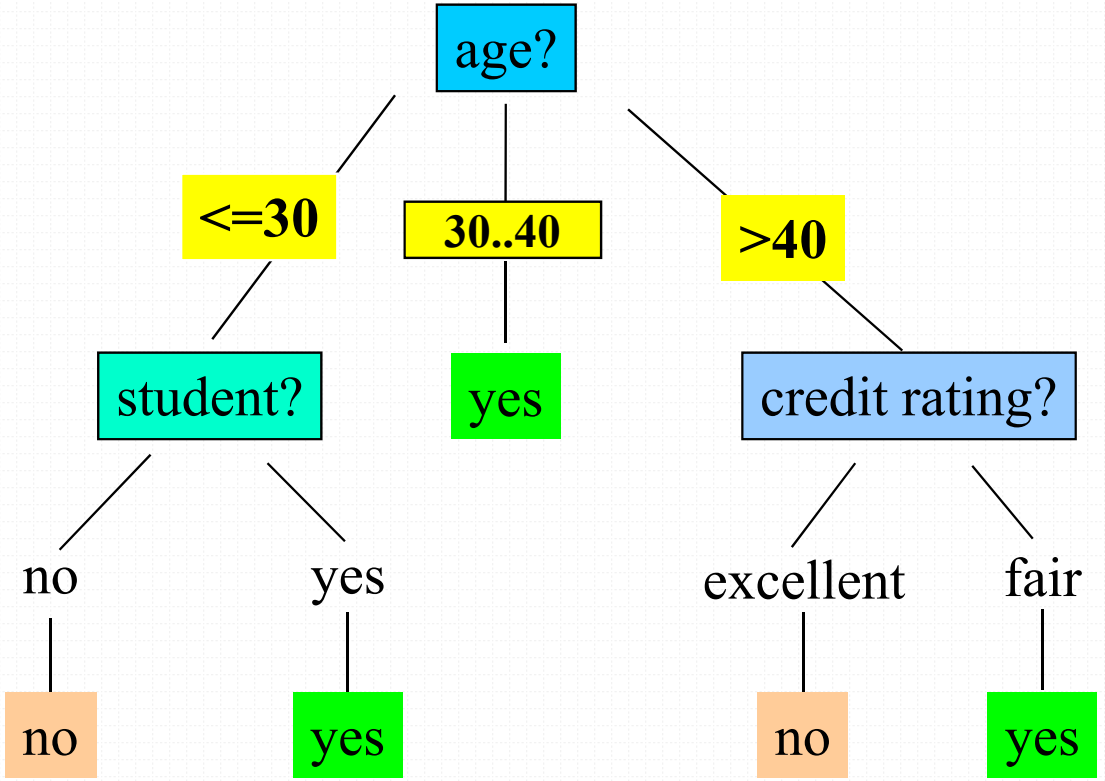
判定树的使用：对未知样本分类

- 样本的属性值在判定树上测试

分类数据集

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

输出：概念 “buys_computer”的判定树



判定树归纳算法

基本算法（贪心算法）

- 自顶向下递归的各个击破方式构造判定树
- 开始，所有的训练样本在根部
- 属性分类 (假如是连续值, 属性首先离散化)
- 基于选定的属性递归的形成每个划分
- 选择属性基于启发式或统计式策略 (比如, 信息增益)

停止划分的条件

- 给定节点的所有样本属于同一类
- 没有剩余属性可以用来进一步划分样本 –使用多数表决
- 没有样本剩余

选择具有最高信息增益的属性

假定有两类，*P* 和 *N*

样本集合S包含类P的p个元素，类N的n个元素

S中的任意样本属于类P和N,所需的期望信息,

$$I(p,n) = -\frac{p}{p+n}\log_2\frac{p}{p+n} - \frac{n}{p+n}\log_2\frac{n}{p+n}$$

判定树归纳中的信息增益

设使用属性A，S划分为子集 $\{S_1, S_2, \ldots, S_v\}$

假如S_i包含P中的 p_i 个样本和N中的n_i 个样本，熵，期望信息由下式给出

$$E(A) = \sum_{i=1}^{v} \frac{p_i + n_i}{p + n} I(p_i, n_i)$$

从 A 上分支将获得的编码信息

$$Gain(A) = I(p,n) - E(A)$$

Class P: buys_computer = “yes”

Class N: buys_computer = “no”

I(p, n) = I(9, 5) =0.940

Compute the entropy for *age*:

age	p_i	n_i	$I(p_i, n_i)$
<=30	2	3	0.971
30…40	4	0	0
>40	3	2	0.971

$$E(age) = \frac{5}{14} I(2,3) + \frac{4}{14} I(4,0) + \frac{5}{14} I(3,2) = 0.69$$

Hence

$$Gain(age) = I(p,n) - E(age)$$

Similarly

$$Gain(income) = 0.029$$

$$Gain(student) = 0.151$$

$$Gain(credit_rating) = 0.048$$

由判定树提取分类规则

以 IF-THEN 形式表示分类规则

对从根到树叶的每条路径创建一个规则

沿着给定路径上的每个属性-值对形成规则前件的一个合取项

叶结点包含类预测

规则易于理解

例子

IF *age* = "<=30" AND *student* = "*no*" THEN *buys_computer* = "*no*"

IF *age* = "<=30" AND *student* = "*yes*" THEN *buys_computer* = "*yes*"

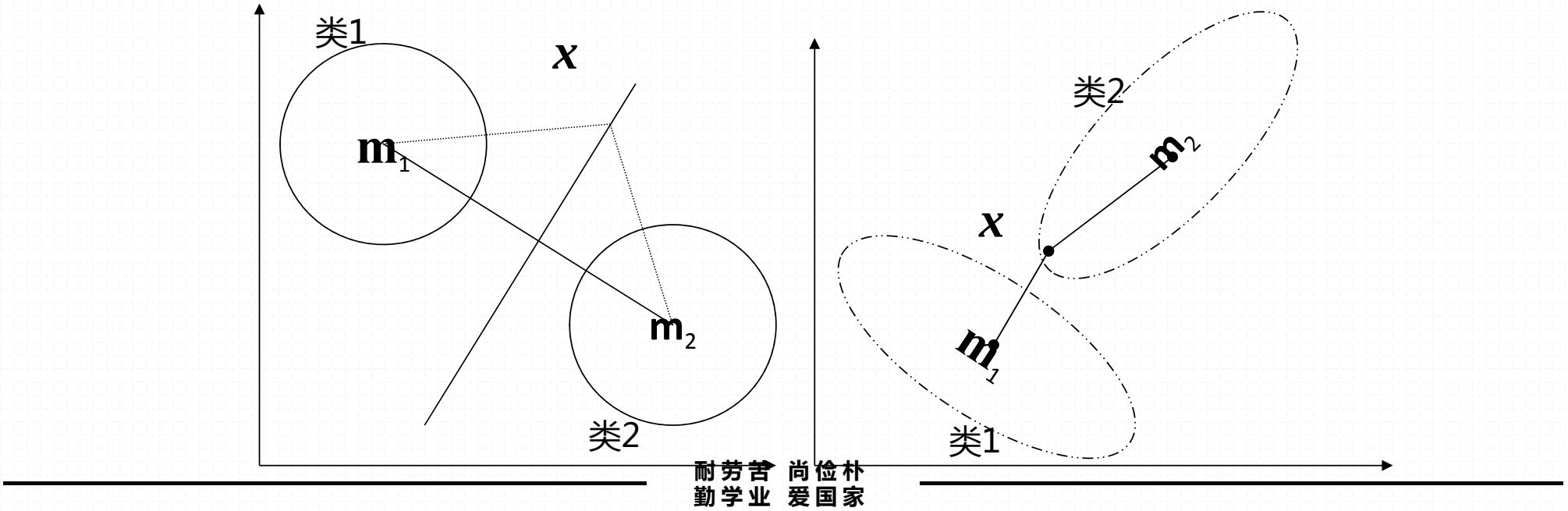
IF *age* = "31…40" THEN *buys_computer* = "*yes*"

IF *age* = ">40" AND *credit_rating* = "*excellent*" THEN *buys_computer* = "*yes*"

IF *age* = ">40" AND *credit_rating* = "*fair*" THEN *buys_computer* = "*no*"

2.4 KNN

- 最小距离分类器: 它将各类训练样本划分成若干子类，并在每个子类中确定代表点。未知样本的类别则以其与这些代表点距离最近作决策。该方法的缺点是所选择的代表点并不一定能很好地代表各类，其后果将使错误率增加。



近邻法

- 最近邻分类器(nearest neighborhood classifier, nnc): 最小距离分类器的一种极端的情况，以全部训练样本作为代表点，计算测试样本与所有样本的距离，并以最近邻者的类别作为决策
- 最初的近邻法是由Cover和Hart于1968年提出的，随后得到理论上深入的分析与研究，是非参数法中最重要的方法之一。

近邻法

最近邻

K-近邻

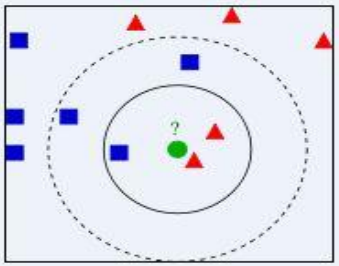
可以证明，最近邻法的错误率高于贝叶斯错误率

k-近邻法

- *k*-近邻法: 最近邻法的扩展，其基本规则是，在所有*N*个样本中找到与测试样本的*k*个最近邻者

  *k*个近邻中，各类别所占个数记为k_i, i = 1，…，c

  定义判别函数为：$g_i(x)=k_i,\ i=1, 2,\ldots,c$



- *k*-近邻一般采用*k*为奇数，避免因票数相等而难以决策
- 决策规则为：

$$j=\underset{i}{\operatorname{argmax}}\, g_i(\mathbf{x}),\quad i=1,\ldots,c$$

在$N\to\infty$的条件下，*k*-近邻法的错误率要低于最近邻法，但都高于贝叶斯错误率

Lazy vs. Eager learning

（1）积极学习法 (决策树)：先学习、再预测(分类)

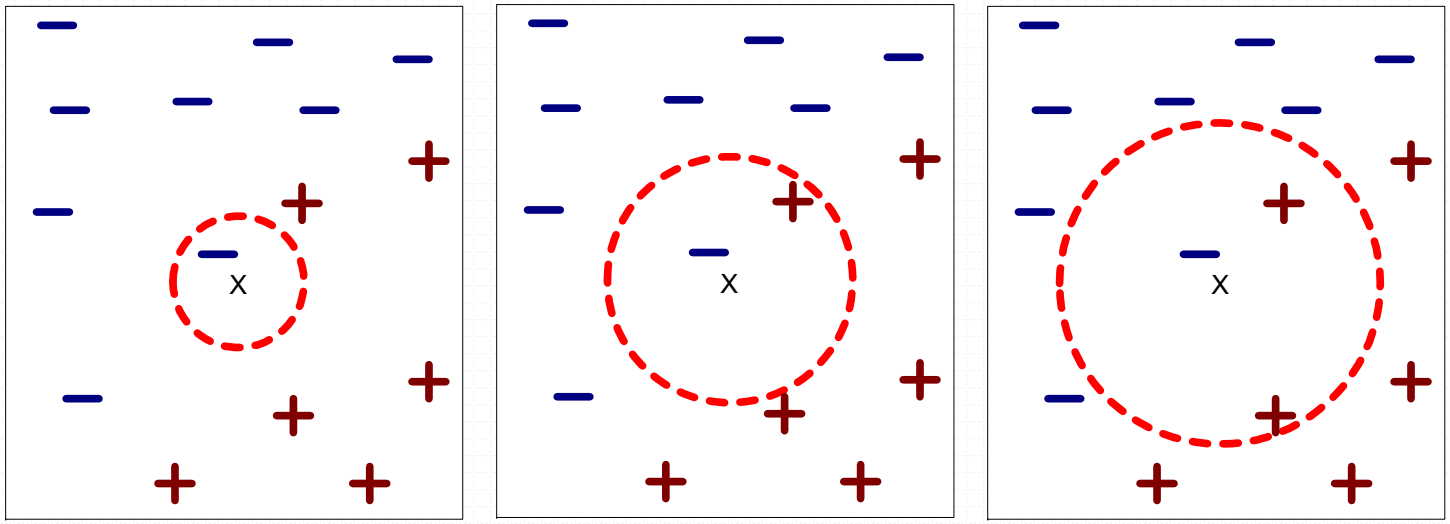
先根据训练集构造出分类模型，根据分类模型对未知样本进行分类

（2）消极学习法 (基于实例的学习法)：

推迟建模，当给定训练集时，简单地存储训练集 (或稍加处理) ，一直等到给定一个未知样本再做分类。直接预测(分类)，几乎不学习

K-NN就是一种消极学习模型

Definition of Nearest Neighbor



(a) 1-nearest neighbor (b) 2-nearest neighbor (c) 3-nearest neighbor

一个待分类x的K-最近邻就是与x的距离最近的k个数据点

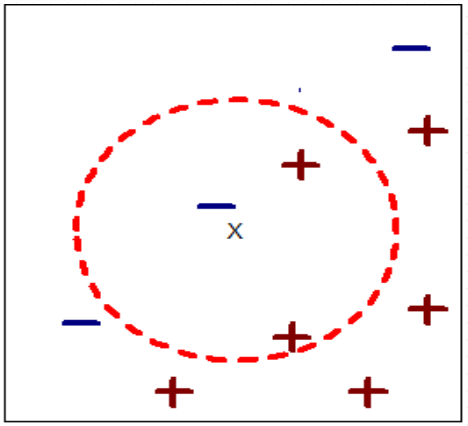
»KNN-距离度量方法（聚类算法中介绍）

- 闵可夫斯基距离度量
- 汉明距离
- 基于相似性的距离
- others

KNN方法的问题

样本不平衡问题：倾向于样本多的类

当样本不平衡时，即：一个类的样本容量很大，而其他类样本数量很小时，很有可能导致当输入一个未知样本z=(x',y')时，该样本的K个邻居中大数量类的样本占多数。 但占多数的样本并不接近目标样本，而数量小的样本很靠近目标样本，即该未知样本本应属于数量小的样本所属类的可能性更大，KNN未能考虑这种情况，即**不把距离远近考虑**在内。



解决办法：距离加权表决，远的惩罚、近的奖励

$$w_i = 1/d(\mathbf{x}', \mathbf{x}_i)^2$$

$$j = \underset{i}{\operatorname{argmax}}\, w_i \times g_i(\mathbf{x}), \quad i = 1,\ldots,c$$

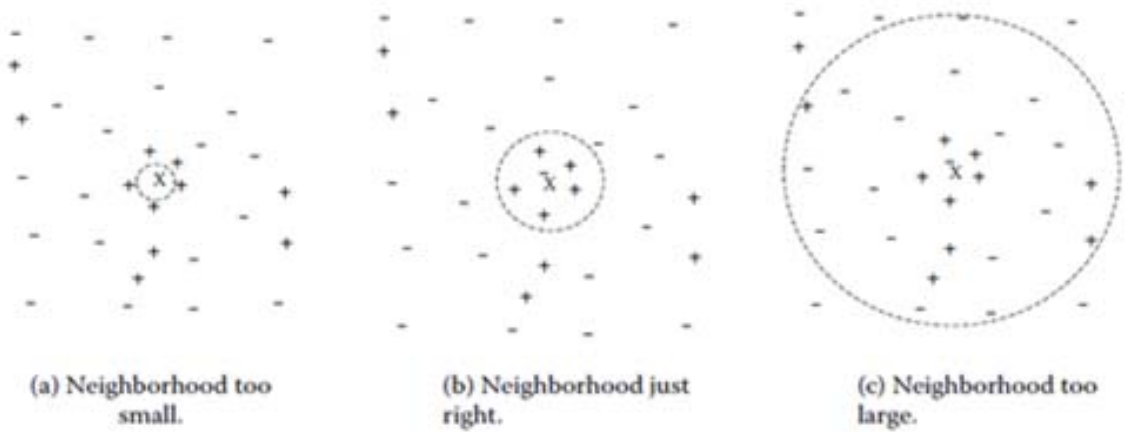
»KNN算法的几点问题

（1）k值设定为多大？

k太小，分类结果易受噪声点影响；k太大，近邻中又可能包含太多的其它类别的点。

k值通常是采用交叉检验来确定。

经验规则：k一般低于训练样本数的平方根



(a) Neighborhood too small.　(b) Neighborhood just right.　(c) Neighborhood too large.

»KNN算法几点问题

（2）类别如何判定最合适？

投票法没有考虑样本距离问题，距离更近的近邻也许更应该决定最终的分类，所以加权投票法更恰当一些（样本不平衡时更严重）

（3）如何选择合适的距离衡量？

高维度对距离衡量的影响：维度越高，欧式距离的区分能力就越差

变量值域对距离的影响：值越大的变量会在距离计算中占据主导作用，考虑归一化

（4）训练样本是否要一视同仁？

在训练集中，有些样本可能是更值得依赖。可为不同的样本施加不同的权重，加强依赖样本的权重，降低不可信赖样本的影响。

（5）性能问题，如何减少样本？

»KNN算法的性能问题

影响KNN算法性能的两个重要因素

（1）第一是需要存储全部的训练样本

（2）第二是需要进行大量的距离计算量

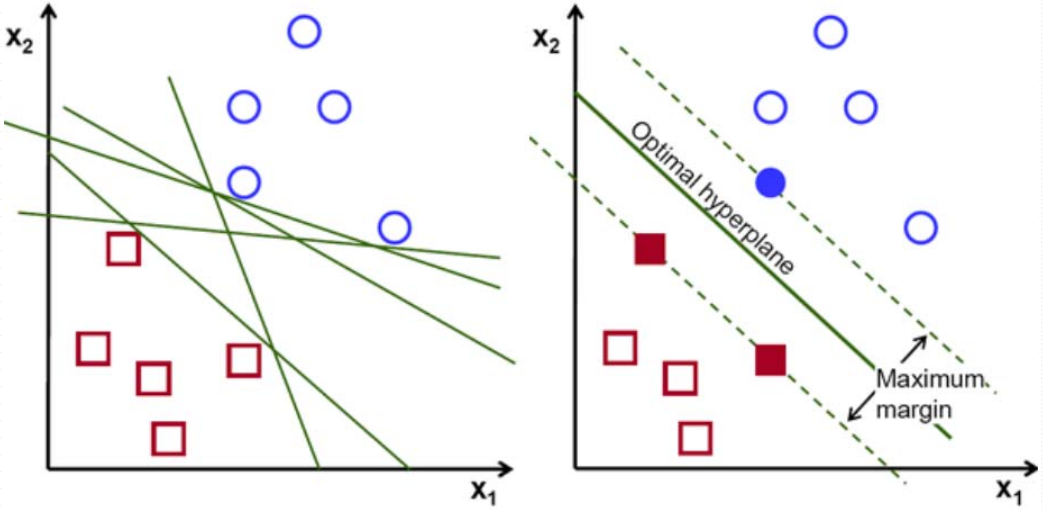
改进方法：

（1）一种是对样本集进行组织与整理，分群分层，尽可能将计算压缩到在接近测试样本邻域的小范围内，避免盲目地与训练样本集中每个样本进行距离计算。

（2）另一种则是在原有样本集中挑选出对分类计算有效的样本，使样本总数合理地减少，以同时达到既减少计算量，又减少存储量的双重效果。

2.5 SVM（1/3）--算法思想

- 支持向量机（Support Vector Machine）建立在统计学习理论VC 维理论和结构风险最小原理基础上，根据有限样本信息在模型复杂性（对特定训练样本的学习精度，Accuracy）和学习能力（无错误地识别任意样本的能力）之间寻求最佳折中，以期获得最好泛化能力。



- **SVM基本任务**。找到一个能够让两类数据都离超平面很远的超平面，在分开数据的超平面的两边建有两个互相平行的超平面。分隔超平面使两个平行超平面的距离最大化，平行超平面间的距离或差距越大，分类器的总误差越小

2.5 SVM（1/3）--算法思想

- **超平面**：超平面是指n维线性空间中维度为n-1的子空间，它可以把线性空间分割成不相交的两部分。如二维空间中的一条直线，三维空间中的一个二维平面。

超平面的方程：$\mathbf{w}^T\mathbf{x}+b=0$

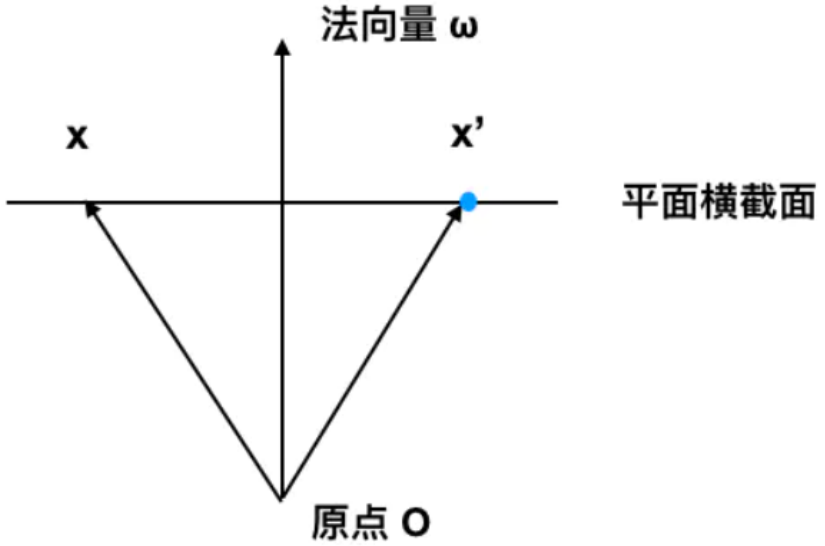
考虑：若超平面过原点，则对于超平面上的任意点**x**，满足：

$$\mathbf{w}^T\mathbf{x}=0$$

若超平面不过原点，假设超平面经过点$\mathbf{x}'=(x_1',x_2',x_3')$，则对于超平面上的任意点**x**，满足：

$$\mathbf{w}^T(\mathbf{x}-\mathbf{x}')=0$$

令$\mathbf{w}^T\mathbf{x}'=b$，因此得到超平面方程为：$\mathbf{w}^T\mathbf{x}+b=0$



2.5 SVM（1/3）--算法思想

- **点到超平面的距离**：假设**x**是超平面外一个点，**x'** 是超平面上的点，**x**到超平面的距离记为d（图中红色线段的长度），则有：

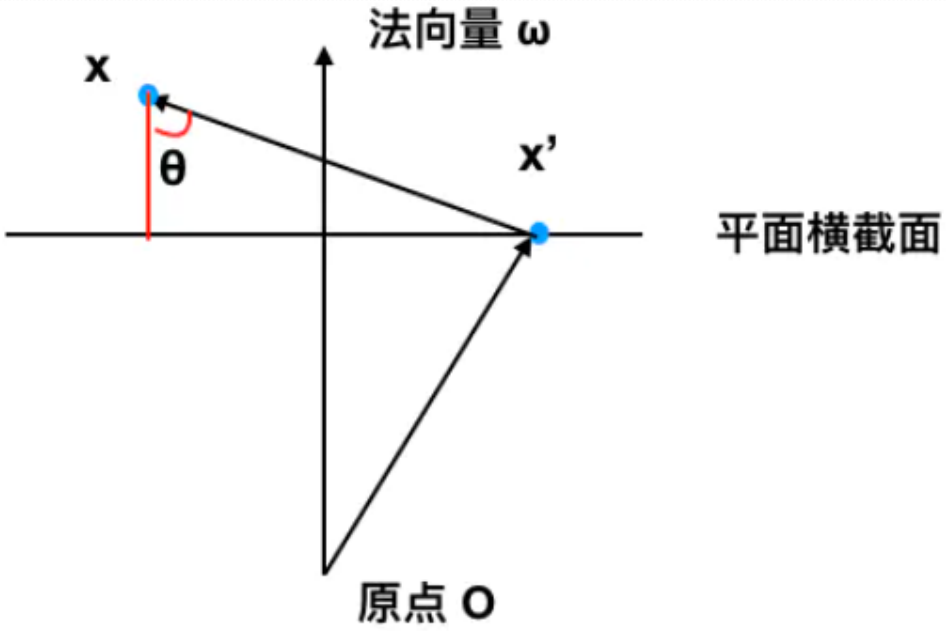
$$d = \left\| \mathbf{x} - \mathbf{x}' \right\| \cos\theta$$

进一步根据**x–x'**与法向量**w**的关系：

$$\left| \mathbf{w}^T (\mathbf{x} - \mathbf{x}') \right| = \left\| \mathbf{w} \right\| \cdot \left\| \mathbf{x} - \mathbf{x}' \right\| \cos\theta$$

可得：

$$d = \frac{\left| \mathbf{w}^T \mathbf{x} + b \right|}{\left\| \mathbf{w} \right\|}$$



2.5 SVM（2/3）--算法流程

输入：线性可分训练数据集 $T=\{(x_1,y_1),(x_2,y_2),\cdots,(x_N,y_N)\}$

$x_i \in \mathcal{X}=\mathbf{R}^n \qquad y_i \in \mathcal{Y}=\{-1,+1\}, \quad i=1,2,\cdots,N$

输出：最大间隔分离超平面和分类决策函数

1、构造并求解约束最优化问题

$$\min_{w,b} \quad \frac{1}{2}\|w\|^2$$

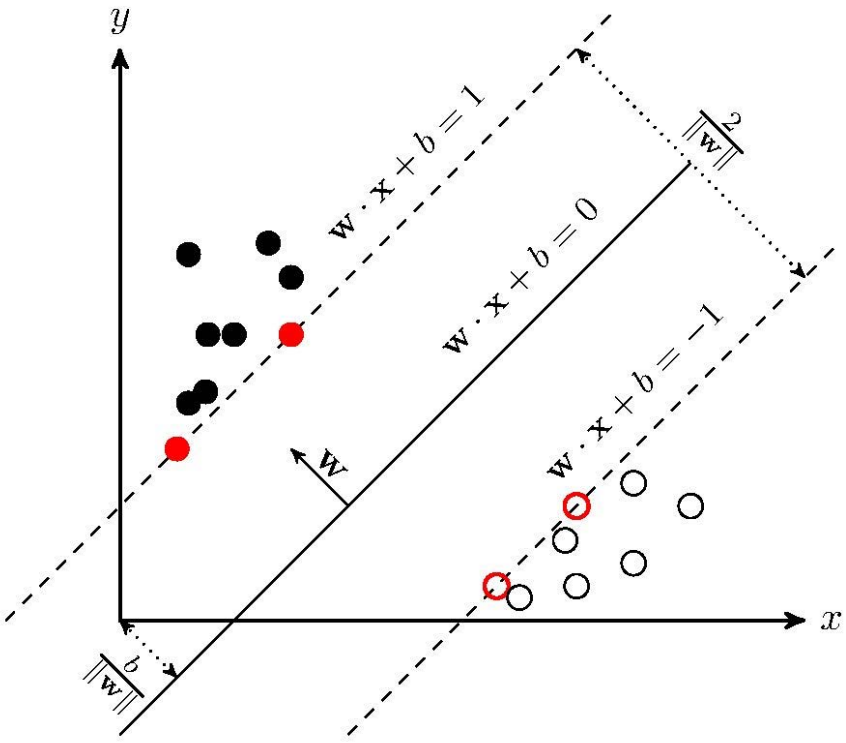
$$\text{s.t.} \quad y_i(w\cdot x_i+b)-1\geqslant 0, \quad i=1,2,\cdots,N$$

2、得到分离超平面

$$w^*\cdot x+b^*=0$$

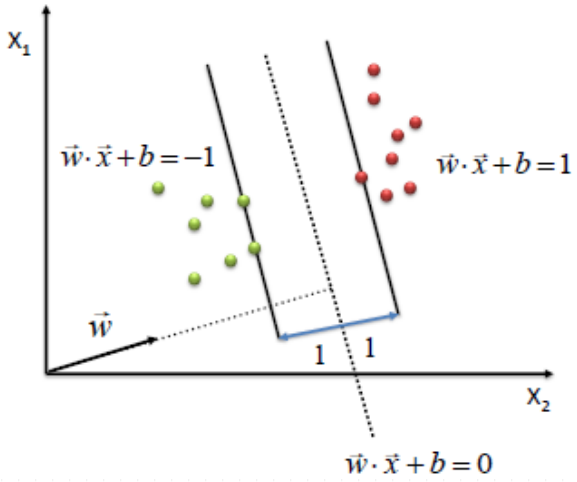
分类决策函数

$$f(x)=\text{sign}(w^*\cdot x+b^*)$$



2.5.1 SVM（3/3）-- 算法原理

■ 为了定义最佳超平面，我们需要最大化边距的宽度*w*

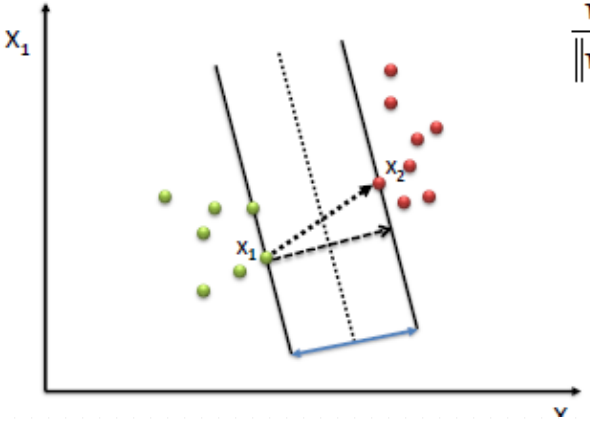


$$\max \frac{2}{\|w\|}$$

s.t.

$(w\cdot x+b)\geq 1, \forall x \text{ of class } 1$

$(w\cdot x+b)\leq -1, \forall x \text{ of class } 2$



$$\frac{w}{\|w\|}\cdot(x_2-x_1)=\text{width}=\frac{2}{\|w\|}$$

$$w\cdot x_2+b=1$$

$$w\cdot x_1+b=-1$$

$$w\cdot x_2+b-w\cdot x_1-b=1-(-1)$$

$$w\cdot x_2-w\cdot x_1=2$$

$$\frac{w}{\|w\|}(x_2-x_1)=\frac{2}{\|w\|}$$

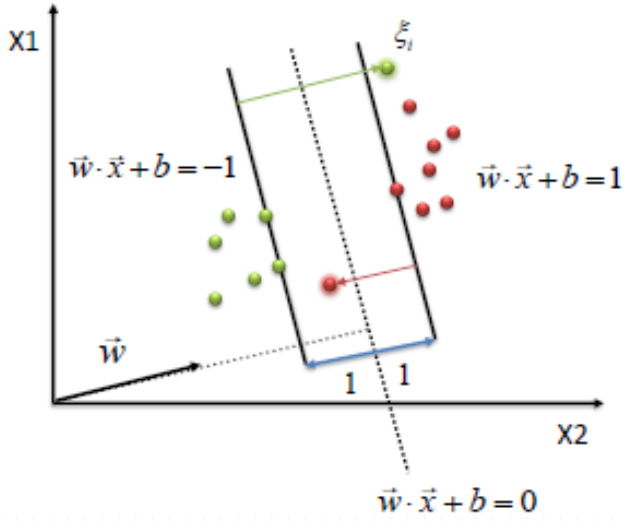
2.5.1 SVM（3/3）-- 算法原理

- 我们发现*w*和*b*通过使用二次规划求解以下目标函数

$$\min \frac{1}{2}\|w\|^2$$

$$s.t.\ y_i(w \cdot x_i + b) \geq 1,\ \forall x_i$$

- SVM的美妙之处在于，如果数据是线性可分的，那么就有一个唯一的全局最小值。然而，完全分离可能是不可能的，在这种情况下，SVM找到了最大化边际和最小化误分类的超平面



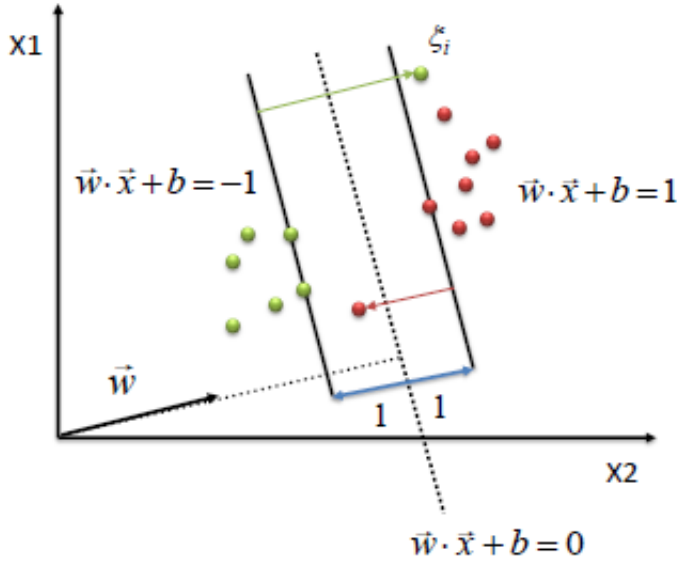
slack variable:

ξ_i

Allow some instances to fall off the margin, but penalize them

2.5.1 SVM（3 / 3）-- 算法原理

- SVM试图在最大化边距的同时将松弛变量保持为零。然而，它并没有最小化误分类的数量，而是最小化了距边缘超平面的距离之和



Constraint becomes :

$$y_i(w\cdot x_i+b)\geq 1-\xi_i,\ \forall x_i$$
$$\xi_i\geq 0$$

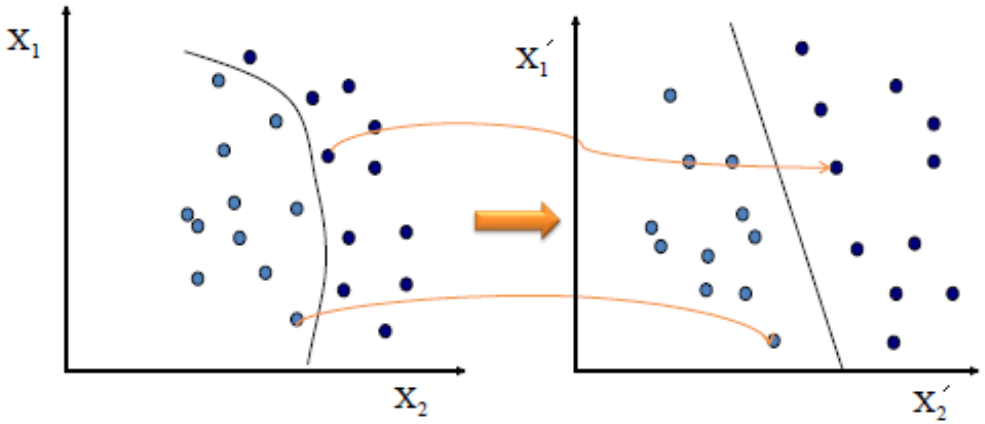
Objective function penalizes for misclassified instances and those within the margin

$$\min\frac{1}{2}\|w\|^2+C\sum_i\xi_i$$

C trades-off margin width and misclassifications

2.5.1 SVM（3/3）-- 算法原理

- 分离两组数据的最简单方法是使用直线(一维)、平面(二维)或N维超平面。然而，在某些情况下，非线性区域可以更有效地分离这些组。SVM通过使用核函数(非线性)将数据映射到一个不同的空间来处理这个问题



Linear SVM $x_i \cdot x_j$

Non-linear SVM $\phi(x_i) \cdot \phi(x_j)$

Kernel function $k(x_i \cdot x_j)$



大 数 据 算 法

CONTENTS

01 大数据挖掘算法概述
Introduction

02 分类算法
Sorting algorithms

03 聚类算法
Clustering algorithms

04 关联规则算法
Association rule algorithms

05 预测算法
Prediction algorithms

监督学习 （supervised learning）

无监督学习（unsupervised learning）：聚类

- 分析学习的数据没有标记，也没有确定结果
- 需要从数据中获得数据分布的相关知识
- 聚类是一种无监督学习模式

半监督学习（Semi-Supervised Learning）

3.1 聚类算法简介

聚类是一种典型的无监督学习方法

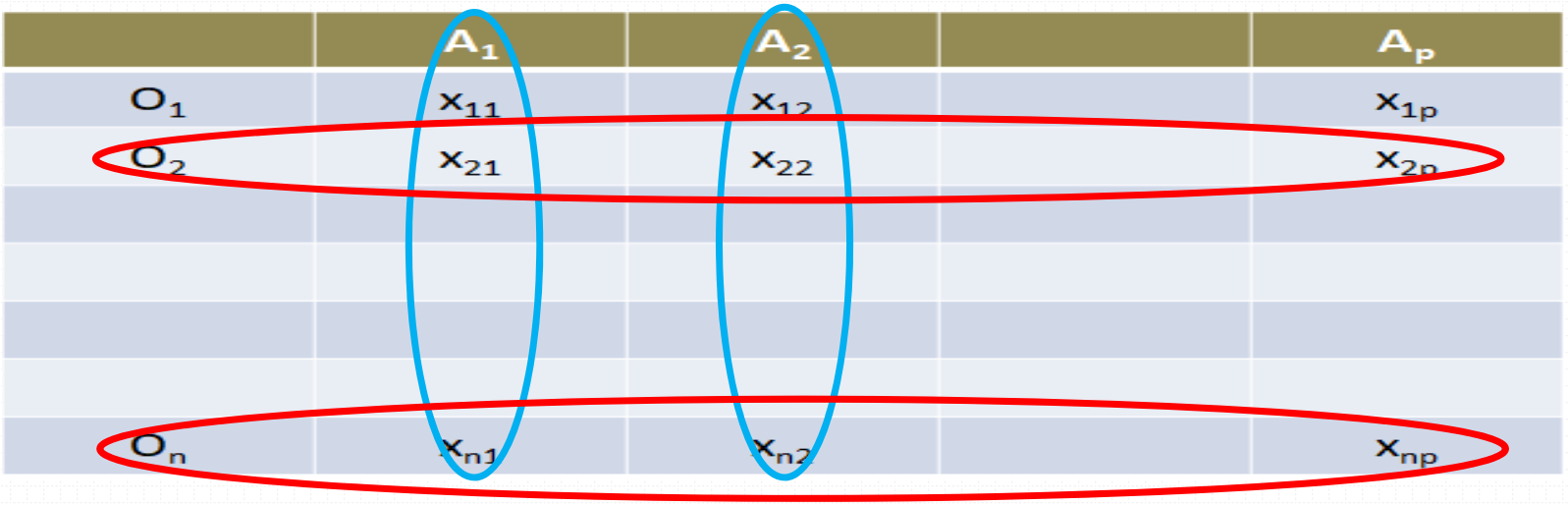
- 聚类（clustering）
  - 将具体或抽象对象的集合分组成由相似对象组成的为多个类或簇的过程
  - 由聚类生成的簇是一组数据对象集合，簇必须同时满足以下两个条件：每个簇至少包含一个数据对象；每个数据对象必须属于且唯一地属于一个簇

- 聚类分析
  - 指用数学的方法来研究与处理给定对象的分类，主要是从数据集中寻找数据间的相似性，并以此对数据进行分类，使得同一个簇中的数据对象尽可能相似，不同簇中的数据对象尽可能相异，从而发现数据中隐含的、有用的信息



聚类过程

聚类分析类别



	A_1	A_2		A_p
O_1	x_{11}	x_{12}		x_{1p}
O_2	x_{21}	x_{22}		x_{2p}
O_n	x_{n1}	x_{n2}		x_{np}

R型聚类(变量相似性)

含义：对属性进行分析，聚类

目的：找出独立又有代表性的变量

Q型聚类(距离度量)

含义：对实体进行进行分析，聚类

目的：找出实体的一种最优划分

聚类分析类别

- 聚类分析是根据给定一组对象的描述信息，发现具有共同特性的对象构成簇（cluster）
- 基本原则是
  - 属于同一个簇的对象的相似度很高，而属于不同簇的对象的相似度很低。
- 聚类问题：
  - 数据集D=$\{o_i=(x_{i1}, x_{i2},\ldots, x_{im})$，i=1,2, …n $\}$，属性集合$A=\{A_1, A_2,\ldots, A_m\}$，$x_{ij}$是第i个对象第j个属性的取值。
  - 给定D以及要聚类的簇的个数k，输出k个簇的组成。设簇的集合为C=$\{C_1, C_2,\ldots, C_k\}$，其中$C_i=\{o_{i_1}, o_{i_2}, \ldots, o_{i_l}\}\subseteq$D

聚类分析类别

- **根据簇之间的关系**
  - **划分型聚类**：属于各个簇的对象之间没有交集，即$C_i \cap C_j = \Phi$
  - **层次型聚类**：簇之间只具有包含关系，如$C_i \subset C_j$
  - **重叠聚类:** 簇之间只具有重叠关系，即$C_i \cap C_j \neq \Phi$
- 根据数据集D与簇之间的关系
  - **完全聚类:** $D = C_1 \cup C_2 \cup \ldots \cup C_k$，所有对象都被分配都簇中
  - **部分聚类**: $D \supset C_1 \cup C_2 \cup \ldots \cup C_k$
- **孤立点(**outlier)：那些未被分到任一个簇中的对象

聚类方法分类

- **划分法**（Partitioning approach）：
  - k均值（**k-means**）、k中心点（k-medoids）、CLARA、CLARANS等方法
- **顺序聚类**(Sequential Clustering Algorithms)
  - BSAS基本的顺序聚类算法、**MBSAS**改进的BSAS
- **层次法（**Hierarchical approach）：
  - **凝聚层次聚类**和**分裂层次聚类**
  - Diana、 Agnes、BIRCH、 CURE、ROCK、CHEMALONE等。
- **基于密度的方法**（Density-based approach）
  - **DBSCAN**、OPTICS和 DenClue等。
- **基于网格的方法**
  - STING、CLIQUE、WaveCluster、OPTIGRID
- **基于模型的方法**（Model-based）
  - EM、SOM、COBWEB、CLASSIT、AutoClass等

距离度量方法

（1）距离度量方法

闵科夫斯基度量方法

曼哈顿距离、欧氏距离、切比雪夫距离等

汉明距离、编辑距离、相合距离

马氏距离

（2）相似性度量方法

余弦相似性

相关系数

（3）others

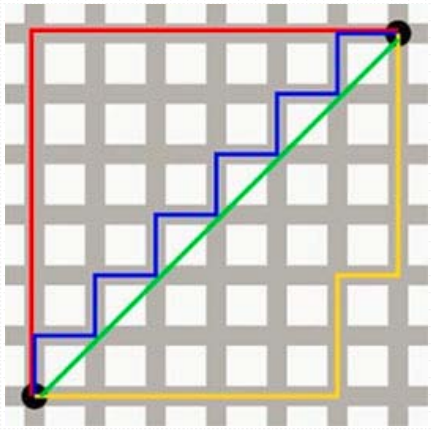
距离度量：曼哈顿距离（Manhattan Distance）

想象你在曼哈顿要从一个十字路口开车到另外一个十字路口，驾驶距离是两点间的直线距离吗？显然不是，除非你能穿越大楼。

实际驾驶距离就是这个“曼哈顿距离”。而这也是曼哈顿距离名称的来源，曼哈顿距离也称为**城市街区距离(City Block distance)**。

两个n维向量$X_1(x_{11},x_{12},\ldots,x_{1n})$与 $X_2(x_{21},x_{22},\ldots,x_{2n})$间的曼哈顿距离

$$d_{12}=\sum_{k=1}^{n}|x_{1k}-x_{2k}|$$



距离度量：欧氏距离(Euclidean Distance)

欧氏距离是最易于理解的一种距离计算方法，源自欧氏空间中两点间的距离公式。

两个n维向量$X_1(x_{11},x_{12},\ldots,x_{1n})$与 $X_2(x_{21},x_{22},\ldots,x_{2n})$间的欧氏距离：

$$d_{12}=\sqrt{\sum_{k=1}^{n}(x_{1k}-x_{2k})^2}$$

国际象棋的玩法。国王走一步能够移动到相邻的8个方格中的任意一个。那么国王从格子(x_1,y_1)走到格子(x_2,y_2)最少需要多少步？你会发现最少步数总是$\max(|x_2-x_1|,|y_2-y_1|)$步。有一种类似的距离度量方法叫切比雪夫距离。

两个n维向量$X_1(x_{11},x_{12},\ldots,x_{1n})$与$X_2(x_{21},x_{22},\ldots,x_{2n})$间的切比雪夫距离

$$d_{12} = \max_i(|x_{1i} - x_{2i}|)$$

这个公式的另一种等价形式是

$$d_{12} = \lim_{k\to\infty}\left(\sum_{i=1}^{n}|x_{1i} - x_{2i}|^k\right)^{1/k}$$

距离度量：闵可夫斯基距离(Minkowski Distance)

闵氏距离不是一种距离，而是一组距离的定义。

闵氏距离的定义

两个n维变量$X_1(x_{11},x_{12},\ldots,x_{1n})$与 $X_2(x_{21},x_{22},\ldots,x_{2n})$间的闵可夫斯基距离定义为：

$$d_{12}=\sqrt[p]{\sum_{k=1}^{n}|x_{1k}-x_{2k}|^p}$$

其中p是一个变参数。

当p=1时，就是曼哈顿距离

当p=2时，就是欧氏距离

当p→∞时，就是切比雪夫距离

根据变参数的不同，闵氏距离可以表示一类的距离

存在问题：

受量纲影响大

通常要做规范化处理

马氏距离：与量纲无关，消除了量纲影响

由印度度著名统计学家马哈拉诺比斯(Mahalanobis)所定义的一种距离

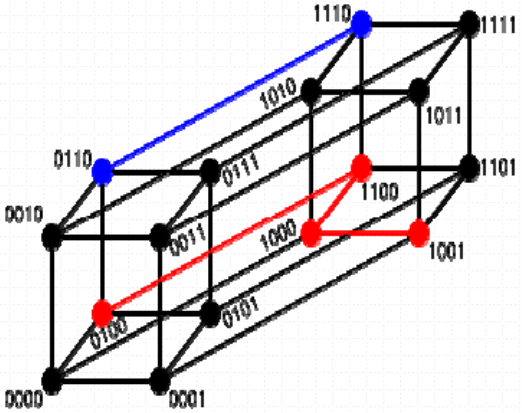
$$d_{ij} = \left[\left(x_{i1} - x_{j1}, x_{i2} - x_{j2}, \cdots, x_{ip} - x_{jp} \right) S^{-1} \begin{pmatrix} x_{i1} - x_{j1} \\ x_{i2} - x_{j2} \\ \vdots \\ x_{ip} - x_{jp} \end{pmatrix} \right]^{\frac{1}{2}}$$

$$= \left[\left(x_i - x_j \right)' S^{-1} \left(x_i - x_j \right) \right]^{\frac{1}{2}}$$

马氏距离不受指标**量纲**及**指标间相关性**的影响

马氏距离要求协方差存在，但协方差可能为 0

距离度量：汉明距离Hamming Distance



两个等长字符串s_1与s_2的汉明距离为：

（1）将一个串变为另一个串，所需要的最小字符替换次数

（2）不同位的数目

相合距离

$$X_1 = (V, Q, S, T, K)$$

$$X_2 = (V, M, S, F, K)$$

$$d_{12} = \frac{\text{不配合数}}{\text{不配合数} + \text{配合数}} = \frac{2}{2+3} = \frac{2}{5}$$

»相似性度量：夹角余弦

$$r_{ij}=\cos\theta_{ij}=\frac{\sum_{k=1}^{n}(X_{ik}X_{jk})}{\sqrt{\sum_{k=1}^{n}X_{ik}^{2}}\sqrt{\sum_{k=1}^{n}X_{jk}^{2}}}\quad(i,\ j=1,\ 2,\ \cdots,\ m)$$

在上式中，显然有：$-1\leqslant\cos\theta_{ij}\leqslant 1$。

如果 X_i 与 X_j 比较相似，则它们的夹角接近于 0，从而 $\cos\alpha_{ij}$ 接近于 1

把变量X_i，X_j看成向量空间的点

»相似性度量： Pearson相关系数

相关系数： 这是一种统计量，是将数据标准化后的夹角余弦

相关系数常用r_{ij}表示。

变量间的协方差

$$r_{ij}=\frac{\sum_{k=1}^{n}(x_{ik}-\bar{x}_i)(x_{jk}-\bar{x}_j)}{\sqrt{\sum_{k=1}^{n}(x_{ik}-\bar{x}_i)^2}\sqrt{\sum_{k=1}^{n}(x_{jk}-\bar{x}_j)^2}} \quad (i,\ j=1,2,\cdots,m)$$

相关系数，一般指变量间的相关系数，用于刻画变量间的相似关系，特别是随机变量

$$r_{ij}=\frac{Cov(X_i,X_j)}{\sqrt{D(x_i)D(X_j)}}$$

»k-means算法

- 输入：数据集D由n个对象，$D=\{o_i=(x_{i1}, x_{i2},\ldots, x_{im})$，$i=1,2,\ldots,n\}$，簇数k
- 算法：

（1）**步骤1**：从n个对象中随机选取k个分别作为k个簇的初始质心c_j，质心是每个簇的代表，通常是靠近簇中心位置的点

（2）**步骤2**：对于D中每个对象o_i通过计算与每个质心c_j的欧式距离，选择距离最近的质心，将其分配到此质心代表的簇中

（3）**步骤3**：重新计算每个簇的质心，用每个簇的样本的均值作为簇的质心

（4）步骤4：若新得到的质心与上一次迭代得到的质心完全相同，则迭代停止；否则，转至**步骤2**

»K-means算法示例

数据对象集合S，每个样本是二维的，要求簇数k=2。

O	x	y
O_1	0	2
O_2	0	0
O_3	1.5	0
O_4	5	0
O_5	5	2

(1)选择 $O_1(0,2)$、$O_2(0,0)$ 分别为C_1、C_2簇的初始质心，即

，$M_1 = O_1 = (0,2)$ $M_2 = O_2 = (0,0)$

(2)对剩余的每个对象，根据其与各个簇中心的距离，将它赋给最近的簇。

对 O_3：

$$d(M_1, O_3) = \sqrt{(0-1.5)^2 + (2-0)^2} = 2.5$$

$$d(M_2, O_3) = \sqrt{(0-1.5)^2 + (0-0)^2} = 1.5$$

$$d(M_2, O_3) \le d(M_1, O_3)$$

故将O_3被分配给C_2

»K-means算法示例

第一次迭代

O	x	y
O_1	0	2
O_2	0	0
O_3	1.5	0
O_4	5	0
O_5	5	2

质心

$M_1 = O_1 = (0,2)$

$M_2 = O_2 = (0,0)$

同理，O_4被分配给M_2

O_5被分配给M_1

$C_1=(O_1,O_5), C_2=(O_2,O_3,O_4)$

第一次迭代的平方差：$D(M_1)+D(M_2)=25+27.5=52.25$

$D(M_1)=(5-0)^2+(2-2)^2=25$

$D(M_2)=(1.5-0)^2+(0-0)^2+(5-0)^2+(0-0)_2=27.25$

计算C_1、C_2的新质心

C_1的新质心：$M_1=(O_1,O_5)/2 = (2.50,2)$

C_2的新质心：$M_2=(O_2,O_3,O_4)/3 =(2.17,0)$

第二次迭代

O	x	y
O_1	0	2
O_2	0	0
O_3	1.5	0
O_4	5	0
O_5	5	2

重复步骤2、步骤3，得到

O_1、O_5分配给C_1　　$C_1=(O_1、O_5)$

O_2、O_3、O_4分配给C_2　$C_2=(O_2、O_3、O_4)$

质心　$M_1=(2.50,2)$

$M_2=(2.17,0)$

第二次迭代的平方差：$D(M_1)+D(M_2)=12.50+13.15=25.65$

$D(C_1)=(0-2.5)^2+(2-2)^2+(5-2.5)^2+(2-2)^2=12.50$

$D(C_2)=(0-2.17)^2+(0-0)^2+(1.5-2.17)^2+(0-0)^2+(5-2.17)^2+(0-0)^2$

$=13.15$

第二次迭代后的总体平方差显著减少，从52.25➔25.65

此外，两次迭代的簇质心不变，算法停止

»K-Means算法优点

- 是解决聚类问题的一种经典算法，简单、快速
- 对处理大数据集，该算法是相对可伸缩和高效率的
- K-means的复杂度是O(nkt)，其中n是对象数目，k 是簇数，t 是迭代次数。通常k<<n 且t<<n
- 当结果簇是密集的，而簇与簇之间区别明显时，它的效果较好

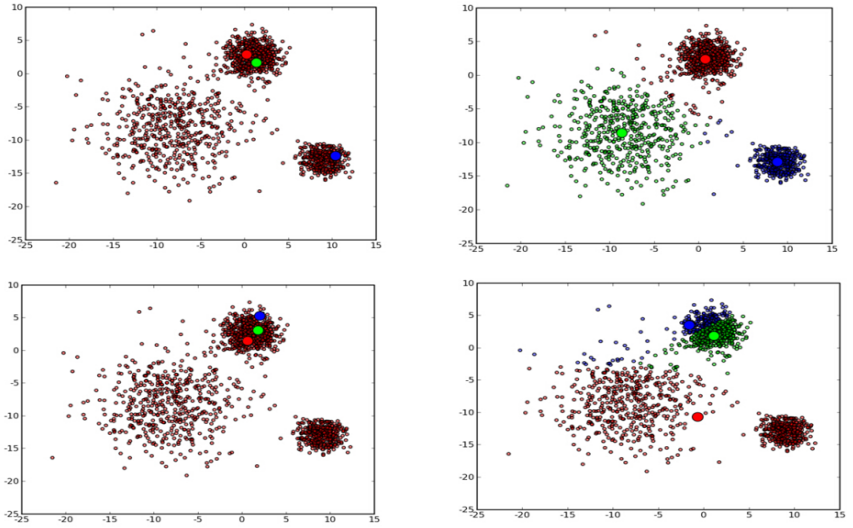
»K-Means算法的不足

（1）必须先给出要聚类的簇数目k，而k值是事先很难估计的

（2）对初值敏感，不同的初始值可能会导致不同的聚类结果

（3）对于"噪声"和孤立点数据敏感，少量的该些数据能够对平均值产生极大影响

不同初始点会产生不同聚类结果



»K-Means改进算法

（1）k-medoids算法（K-中心点算法）

不采用聚类中对象的平均值作为参照点，而是选用聚类中位置最中心的对象，即中心点（medoid）作为参照点

（2）k-means++

随机选择初始点可能会造成聚类结果与数据实际分布相差很大

选择初始聚类中心的基本思想是：初始聚类中心之间的相互距离要尽可能的远

（3） Fuzzy C-Means（模糊C均值算法FCM）

用隶属度确定每个数据点属于某个类的程度的一种聚类算法

»DBSCAN算法

（1）研究原因

划分聚类算法、层次聚类等往往只能发现**凸型的聚类簇。而数据的实际分布形状是多样的，甚至是很复杂的**

原因是距离度量，得到的是类圆形聚类结果

（2）DBSCAN核心思想

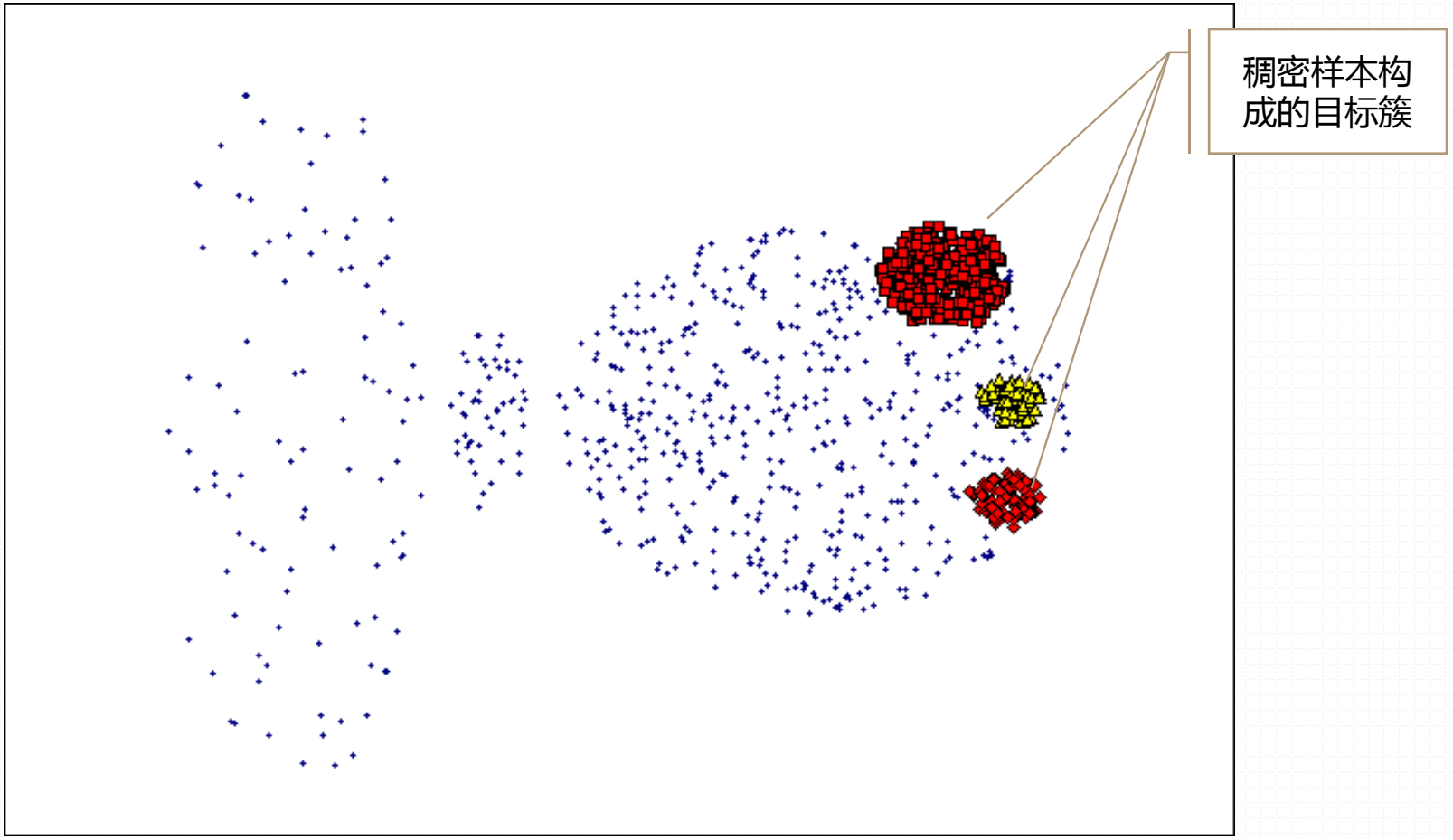
Density-**B**ased Spatial Clustering of Applications with Noise

DBSCAN认为：在整个样本空间点中，**目标簇是由一群稠密样本点组成的**

而这些稠密样本点被低密度区域分割，这类低密度区域被认为是噪声

算法的目的就是要过滤低密度区域，发现稠密样本点

»DBSCAN算法

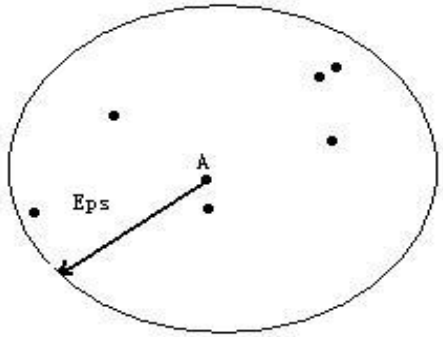


密度的定义

- 基于中心的密度定义为：

数据集中**特定点的密度**通过该点Eps半径之内的点计数(包括本身)来估计。

显然，密度依赖于半径。



基于密度定义，点被分为三类：

（1）稠密区域内部的点 （**核心点**）

（2）稠密区域边缘上的点(**边界点**)

（3）稀疏区域中的点 （**噪声或背景点**）

DBSCAN点分类

（1）核心点(core point)

在半径Eps内含有超过MinPts数目的点，则该点为核心点。其是在簇内的

（2）边界点(border point)

在半径Eps内点的数量小于MinPts，但是与核心点是邻居

（3）噪音点(noise point):

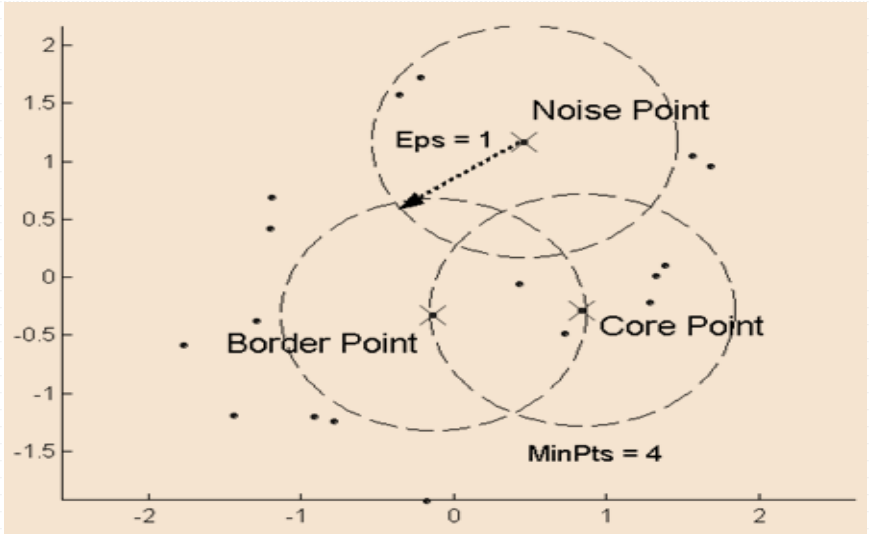
任何不是核心点或边界点的点

Minpts简单地说就是这个范围内最少要有多少个点

超参数：

Eps

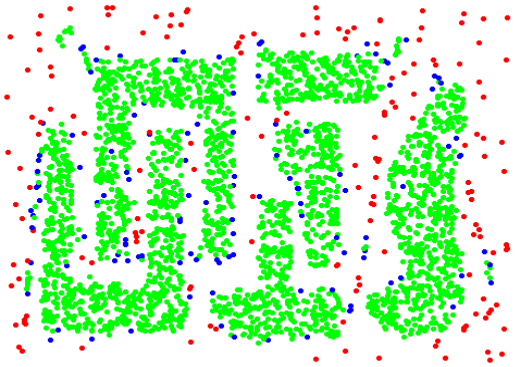
MinPts



DBSCAN: 核心点、边界点和噪音点



Original Points



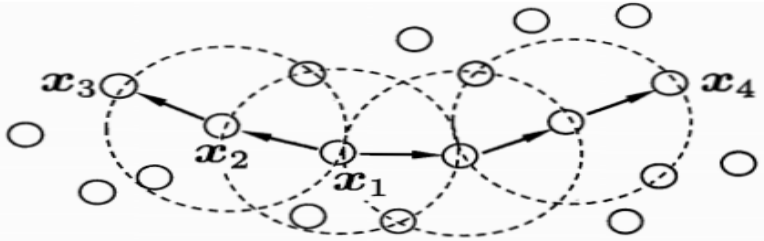
Point types: core, border and noise

Eps = 10, MinPts = 4

»DBSCAN算法概念

DBSCAN 是一种著名的密度聚类算法，它基于一组“邻域”(neighborhood) 参数 $(\epsilon, MinPts)$ 来刻画样本分布的紧密程度. 给定数据集 $D = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_m\}$，定义下面这几个概念：

- ϵ-邻域：对 $\boldsymbol{x}_j \in D$，其 ϵ-邻域包含样本集 D 中与 $\boldsymbol{x}_j$ 的距离不大于 ϵ 的样本，即 $N_\epsilon(\boldsymbol{x}_j) = \{\boldsymbol{x}_i \in D \mid \text{dist}(\boldsymbol{x}_i, \boldsymbol{x}_j) \leqslant \epsilon\}$；
- 核心对象(core object)：若 $\boldsymbol{x}_j$ 的 ϵ-邻域至少包含 $MinPts$ 个样本，即 $|N_\epsilon(\boldsymbol{x}_j)| \geqslant MinPts$，则 $\boldsymbol{x}_j$ 是一个核心对象；
- 密度直达(directly density-reachable)：若 $\boldsymbol{x}_j$ 位于 $\boldsymbol{x}_i$ 的 ϵ-邻域中，且 $\boldsymbol{x}_i$ 是核心对象，则称 $\boldsymbol{x}_j$ 由 $\boldsymbol{x}_i$ 密度直达；
- 密度可达(density-reachable)：对 $\boldsymbol{x}_i$ 与 $\boldsymbol{x}_j$，若存在样本序列 $\boldsymbol{p}_1, \boldsymbol{p}_2, \ldots, \boldsymbol{p}_n$，其中 $\boldsymbol{p}_1 = \boldsymbol{x}_i$，$\boldsymbol{p}_n = \boldsymbol{x}_j$ 且 $\boldsymbol{p}_{i+1}$ 由 $\boldsymbol{p}_i$ 密度直达，则称 $\boldsymbol{x}_j$ 由 $\boldsymbol{x}_i$ 密度可达；
- 密度相连(density-connected)：对 $\boldsymbol{x}_i$ 与 $\boldsymbol{x}_j$，若存在 $\boldsymbol{x}_k$ 使得 $\boldsymbol{x}_i$ 与 $\boldsymbol{x}_j$ 均由 $\boldsymbol{x}_k$ 密度可达，则称 $\boldsymbol{x}_i$ 与 $\boldsymbol{x}_j$ 密度相连.



DBSCAN 定义的基本概念($MinPts = 3$)：虚线显示出 ϵ-邻域，$\boldsymbol{x}_1$ 是核心对象，$\boldsymbol{x}_2$ 由 $\boldsymbol{x}_1$ 密度直达，$\boldsymbol{x}_3$ 由 $\boldsymbol{x}_1$ 密度可达，$\boldsymbol{x}_3$ 与 $\boldsymbol{x}_4$ 密度相连

»DBSCAN算法原理

基于这些概念, DBSCAN 将“簇”定义为: 由密度可达关系导出的最大的密度相连样本集合. 形式化地说, 给定邻域参数 $(\epsilon, MinPts)$, 簇 $C \subseteq D$ 是满足以下性质的非空样本子集:

连接性(connectivity): $\boldsymbol{x}_i \in C, \boldsymbol{x}_j \in C \Rightarrow \boldsymbol{x}_i$ 与 $\boldsymbol{x}_j$ 密度相连

最大性(maximality): $\boldsymbol{x}_i \in C, \boldsymbol{x}_j$ 由 $\boldsymbol{x}_i$ 密度可达 $\Rightarrow \boldsymbol{x}_j \in C$

那么, 如何从数据集 D 中找出满足以上性质的聚类簇呢? 实际上, 若 $\boldsymbol{x}$ 为核心对象, 由 $\boldsymbol{x}$ 密度可达的所有样本组成的集合记为 $X = \{\boldsymbol{x}' \in D \mid \boldsymbol{x}'$ 由 $\boldsymbol{x}$ 密度可达$\}$, 则不难证明 X 即为满足连接性与最大性的簇.

- DBSCAN通过检查数据集中每点的邻域来搜索簇，如果点x的邻域包含的点多于MinPts个，则创建一个以x为核心点的簇
- 然后，**DBSCAN迭代地聚集从这些核心点直接密度可达的对象**，这个过程可能涉及一些密度可达簇的合并
- 当没有新的点添加到任何簇时，该过程结束

»DBSCAN算法伪代码

输入： 样本集 $D = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_m\}$;
邻域参数 $(\epsilon, MinPts)$.

过程：

初始化核心对象集合: $\Omega = \varnothing$
for $j = 1, 2, \ldots, m$ **do**
  确定样本 $\boldsymbol{x}_j$ 的 ϵ-邻域 $N_\epsilon(\boldsymbol{x}_j)$;
  if $|N_\epsilon(\boldsymbol{x}_j)| \geqslant MinPts$ **then**
    将样本 $\boldsymbol{x}_j$ 加入核心对象集合: $\Omega = \Omega \bigcup \{\boldsymbol{x}_j\}$
  end if
end for
初始化聚类簇数: $k = 0$
初始化未访问样本集合: $\Gamma = D$
while $\Omega \neq \varnothing$ **do**
  记录当前未访问样本集合: $\Gamma_{\text{old}} = \Gamma$;
  随机选取一个核心对象 $\boldsymbol{o} \in \Omega$, 初始化队列 $Q =< \boldsymbol{o} >$;
  $\Gamma = \Gamma \setminus \{\boldsymbol{o}\}$;
  while $Q \neq \varnothing$ **do**
    取出队列 Q 中的首个样本 $\boldsymbol{q}$;
    if $|N_\epsilon(\boldsymbol{q})| \geqslant MinPts$ **then**
      令 $\Delta = N_\epsilon(\boldsymbol{q}) \bigcap \Gamma$;
      将 Δ 中的样本加入队列 Q;
      $\Gamma = \Gamma \setminus \Delta$;
    end if
  end while
  $k = k + 1$, 生成聚类簇 $C_k = \Gamma_{\text{old}} \setminus \Gamma$;
  $\Omega = \Omega \setminus C_k$
end while

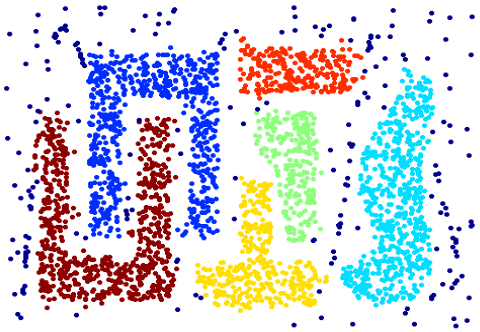
输出： 簇划分 $\mathcal{C} = \{C_1, C_2, \ldots, C_k\}$

密度可达点、可达簇的合并
产生任意簇形状的原因所在

DBSCAN运行效果好的时候



Original Points

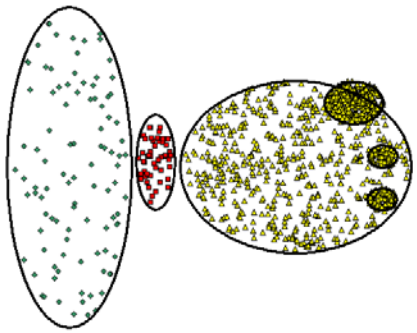


Clusters

- 对噪音不敏感
- 可以处理不同形状和大小的数据

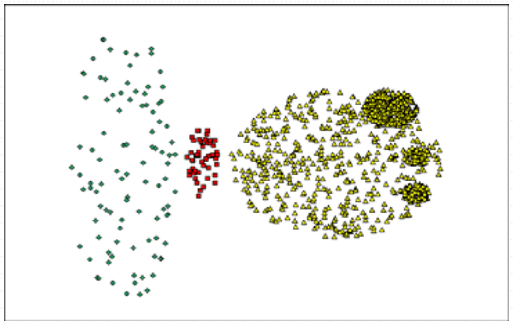
»DBSCAN运行不好的效果

高维数据的难点在于对密度的定义，密度基于eps，eps要有欧几里得距离，欧氏距离在高维中难以定义计算。

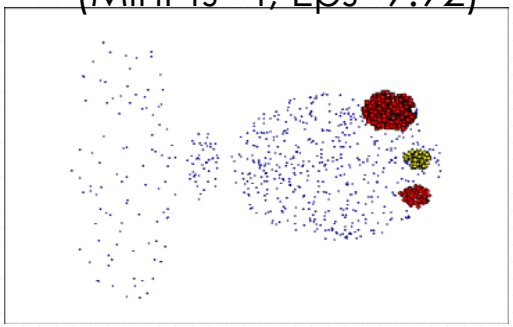


Original Points

- **密度变化的数据**
- **高维数据**



(MinPts=4, Eps=9.92)



(MinPts=4, Eps=9.75)

»DBSCAN算法示例

西瓜数据集 4.0

编号	密度	含糖率	编号	密度	含糖率	编号	密度	含糖率
1	0.697	0.460	11	0.245	0.057	21	0.748	0.232
2	0.774	0.376	12	0.343	0.099	22	0.714	0.346
3	0.634	0.264	13	0.639	0.161	23	0.483	0.312
4	0.608	0.318	14	0.657	0.198	24	0.478	0.437
5	0.556	0.215	15	0.360	0.370	25	0.525	0.369
6	0.403	0.237	16	0.593	0.042	26	0.751	0.489
7	0.481	0.149	17	0.719	0.103	27	0.532	0.472
8	0.437	0.211	18	0.359	0.188	28	0.473	0.376
9	0.666	0.091	19	0.339	0.241	29	0.725	0.445
10	0.243	0.267	20	0.282	0.257	30	0.446	0.459

以西瓜数据集 4.0 为例, 假定邻域参数 $(\epsilon, MinPts)$ 设置为 **(0.11,5)**

DBSCAN 算法先找出各样本的 ϵ-邻域并确定核心对象集合: Ω

$$\Omega = \{\boldsymbol{x}_3, \boldsymbol{x}_5, \boldsymbol{x}_6, \boldsymbol{x}_8, \boldsymbol{x}_9, \boldsymbol{x}_{13}, \boldsymbol{x}_{14}, \boldsymbol{x}_{18}, \boldsymbol{x}_{19}, \boldsymbol{x}_{24}, \boldsymbol{x}_{25}, \boldsymbol{x}_{28}, \boldsymbol{x}_{29}\}$$

然后, 从 Ω 中随机选取一个核心对象作为种子, 找出由它密度可达的所有样本, 这就构成了第一个聚类簇

»DBSCAN算法示例

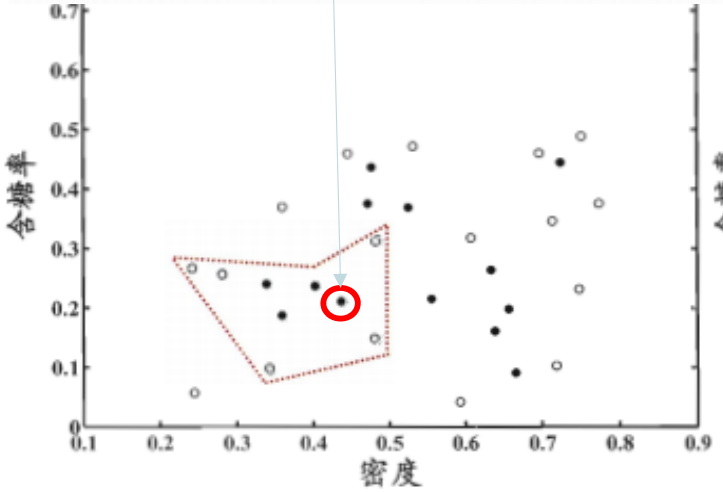
假定核心对象 $\boldsymbol{x}_8$ 被选中作为种子，则 DBSCAN 生成的第一个聚类簇为

$$C_1 = \{\boldsymbol{x}_6, \boldsymbol{x}_7, \boldsymbol{x}_8, \boldsymbol{x}_{10}, \boldsymbol{x}_{12}, \boldsymbol{x}_{18}, \boldsymbol{x}_{19}, \boldsymbol{x}_{20}, \boldsymbol{x}_{23}\}$$

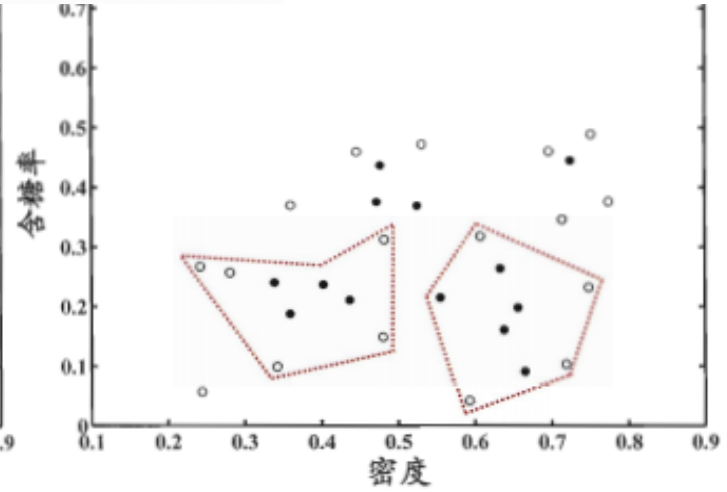
然后，DBSCAN 将 C_1 中包含的核心对象从 Ω 中去除: $\Omega = \Omega \setminus C_1 = \{\boldsymbol{x}_3, \boldsymbol{x}_5, \boldsymbol{x}_9, \boldsymbol{x}_{13}, \boldsymbol{x}_{14}, \boldsymbol{x}_{24}, \boldsymbol{x}_{25}, \boldsymbol{x}_{28}, \boldsymbol{x}_{29}\}$. 再从更新后的集合 Ω 中随机选取一个核心对象作为种子来生成下一个聚类簇. 上述过程不断重复，直至 Ω 为空.

假定核心对象 $\boldsymbol{x}_8$ 被选中作为种子，则 DBSCAN 生成的第一个聚类簇为

$$C_1 = \{\boldsymbol{x}_6, \boldsymbol{x}_7, \boldsymbol{x}_8, \boldsymbol{x}_{10}, \boldsymbol{x}_{12}, \boldsymbol{x}_{18}, \boldsymbol{x}_{19}, \boldsymbol{x}_{20}, \boldsymbol{x}_{23}\}$$



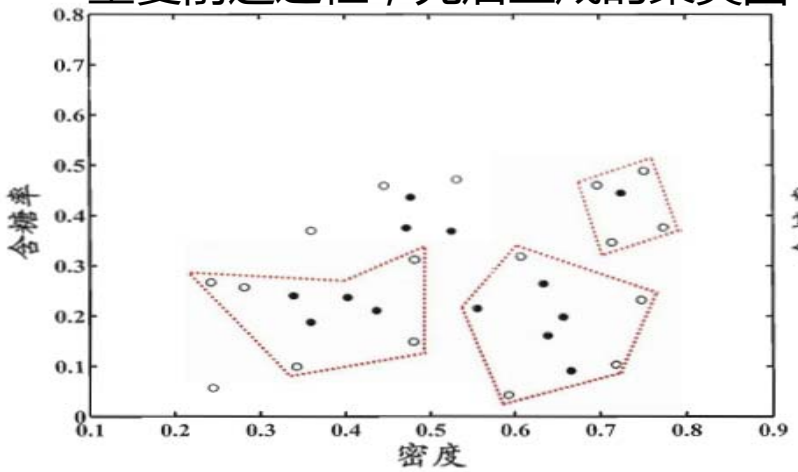
(a) 生成聚类簇 C_1



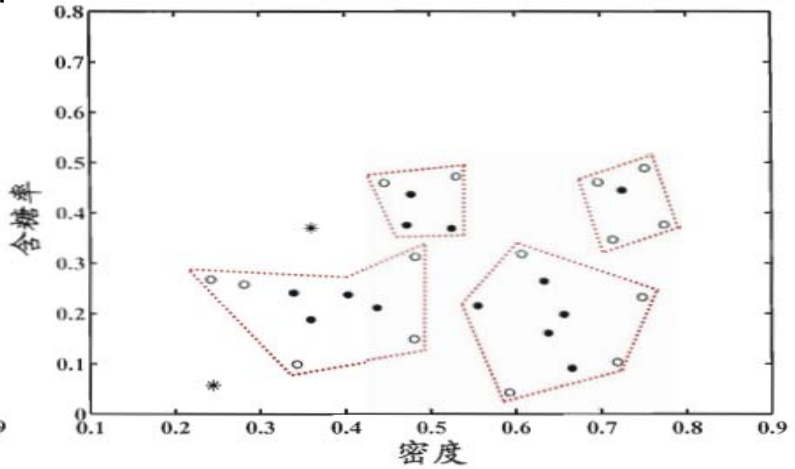
(b) 生成聚类簇 C_2

»DBSCAN算法示例

重复前述过程，先后生成的聚类图



(c) 生成聚类簇 C_3



(d) 生成聚类簇 C_4

DBSCAN 算法($\epsilon = 0.11$, $MinPts = 5$)生成聚类簇的先后情况. 核心对象、非核心对象、噪声样本分别用"•" "○" "*"表示. 红色虚线显示出簇划分.

$$C_2 = \{\boldsymbol{x}_3, \boldsymbol{x}_4, \boldsymbol{x}_5, \boldsymbol{x}_9, \boldsymbol{x}_{13}, \boldsymbol{x}_{14}, \boldsymbol{x}_{16}, \boldsymbol{x}_{17}, \boldsymbol{x}_{21}\};$$

$$C_3 = \{\boldsymbol{x}_1, \boldsymbol{x}_2, \boldsymbol{x}_{22}, \boldsymbol{x}_{26}, \boldsymbol{x}_{29}\};$$

$$C_4 = \{\boldsymbol{x}_{24}, \boldsymbol{x}_{25}, \boldsymbol{x}_{27}, \boldsymbol{x}_{28}, \boldsymbol{x}_{30}\}.$$

DBSCAN算法的优缺点

- 优点
  - 基于密度定义，相对抗噪音，能处理任意形状和大小的簇，无需定义簇数目
- 缺点
  - 当簇的密度变化太大时，会有麻烦
  - 对于高维问题，密度定义是个比较麻烦的问题

Eps取什么？

MinPts取什么？



大 数 据 算 法

CONTENTS

01 大数据挖掘算法概述
Introduction

02 分类算法
Sorting algorithms

03 聚类算法
Clustering algorithms

04 关联规则算法
Association rule algorithms

05 预测算法
Prediction algorithms

4.1 关键规则算法简介

■ 关联规则

- 数据挖掘中最活跃的研究方法之一，是指搜索业务系统中的所有细节或事务，找出所有能把一组事件或数据项与另一组事件或数据项联系起来的规则，以获得存在于数据库中的不为人知的或不能确定的信息，它侧重于确定数据中不同领域之间的联系，也是在无指导学习系统中挖掘本地模式的最普通形式



4.1 关键规则算法简介

■ 关联规则概念

- 一般来说，关联规则挖掘是指从一个大型的数据集（Dataset）发现有趣的关联（Association）或相关关系（Correlation），即从数据集中识别出频繁出现的属性值集（Sets of Attribute Values），也称为频繁项集（Frequent Itemsets，频繁集），然后利用这些频繁项集创建描述关联关系的规则的过程

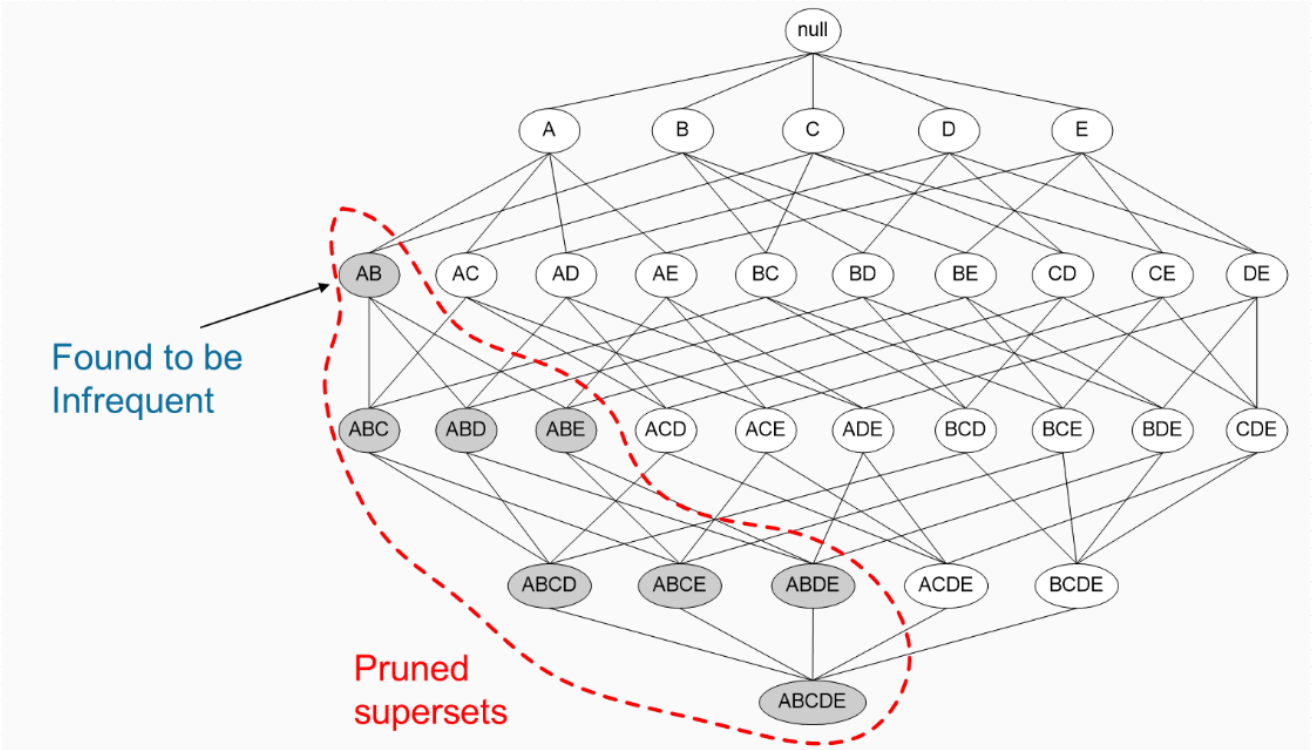
■ 关联规则挖掘问题

- 发现频繁项集
  - 发现所有的频繁项集是形成关联规则的基础。通过用户给定的最小支持度，寻找所有支持度大于或等于Minsupport的频繁项集
- 生成关联规则
  - 通过用户给定的最小可信度，在每个最大频繁项集中，寻找可信度不小于Minconfidence的关联规则

核心问题：如何迅速高效地发现所有频繁项集？

4.2 频繁项集的产生及经典算法

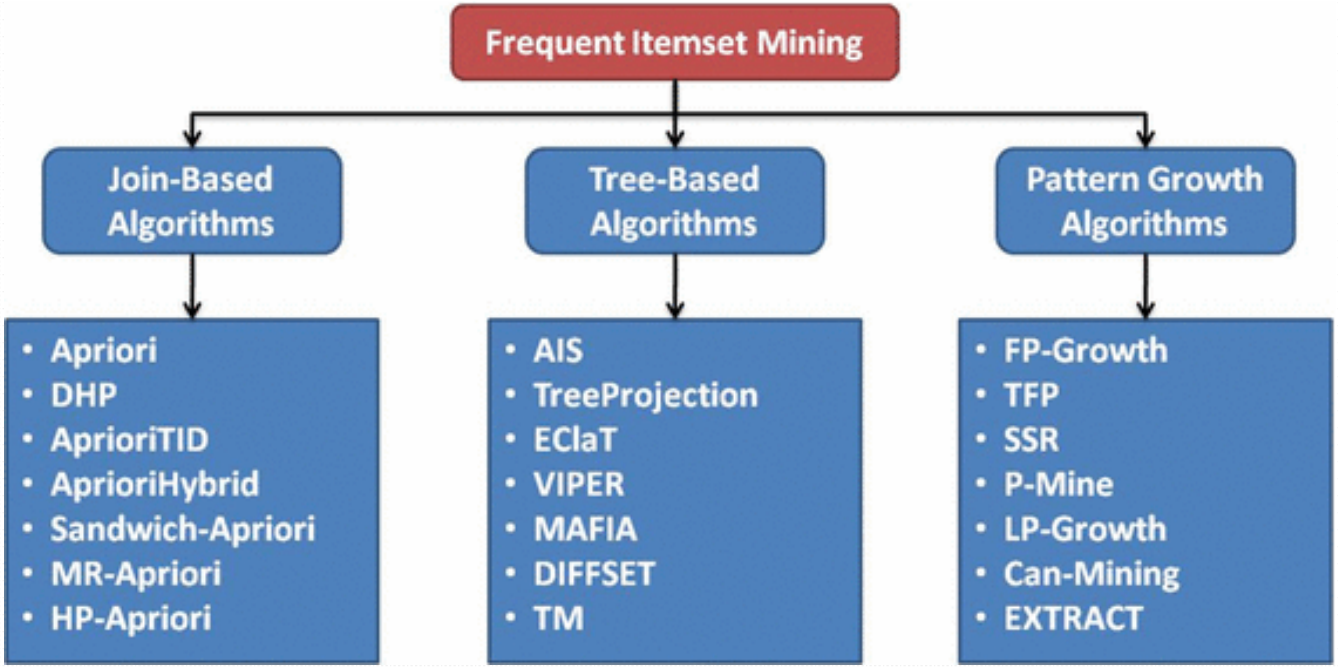
- 格结构（Lattice Structure）常常被用来枚举所有可能的项集



重庆大学 CHONGQING UNIVERSITY

4.2 频繁项集的产生及经典算法

■ 查找频繁项集经典算法



关联规则基本模型

设$I=\{i_1, i_2, \ldots, i_m\}$为所有项目的集合，$D$为事务数据库，事务$T$是一个项目子集（$T \subseteq I$）。每一个事务具有唯一的事务标识$TID$。设$A$是一个由项目构成的集合，称为项集。事务$T$包含项集$A$，当且仅当$A \subseteq T$。如果项集$A$中包含$k$个项目，则称其为$k$项集。项集$A$在事务数据库$D$中出现的次数占$D$中总事务的百分比叫做项集的支持度。如果项集的支持度超过用户给定的最小支持度阈值，就称该项集是频繁项集（或大项集）。

关联规则基本模型（续）

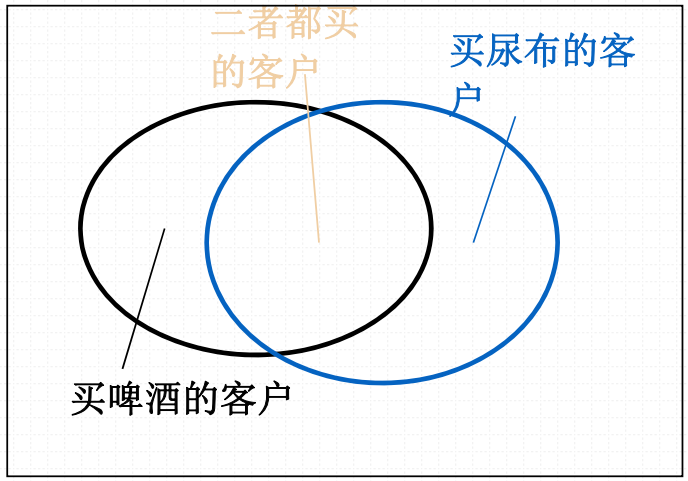
关联规则是形如$X \Rightarrow Y$的逻辑蕴含式，其中$X \subset I$，$Y \subset I$，且$X \cap Y = \varnothing$。如果事务数据库D中有$s\%$的事务包含$X \cup Y$，则称关联规则$X \Rightarrow Y$的支持度为$s\%$，实际上，支持度是一个概率值。若项集X的支持度记为$support(X)$，规则的信任度为$support(X \cup Y) / support(X)$。这是一个条件概率$P(Y \mid X)$。

也就是：

$$support(X \Rightarrow Y) = P(X \cup Y)$$

$$confidence(X \Rightarrow Y) = P(Y \mid X)$$

规则度量：支持度与可信度



查找所有的规则 $X \& Y \Rightarrow Z$ 具有最小支持度和可信度

支持度, *s*, 一次交易中包含{X、 Y、 Z}的可能性

可信度, *c*, 包含{X、 Y}的交易中也包含Z的条件概率

交易ID	购买的商品
2000	A,B,C
1000	A,C
4000	A,D
5000	B,E,F

设最小支持度为50%, 最小可信度为50%, 则可得到

- $A \Rightarrow C$ (50%, 66.6%)
- $C \Rightarrow A$ (50%, 100%)

关联规则基本模型（续）

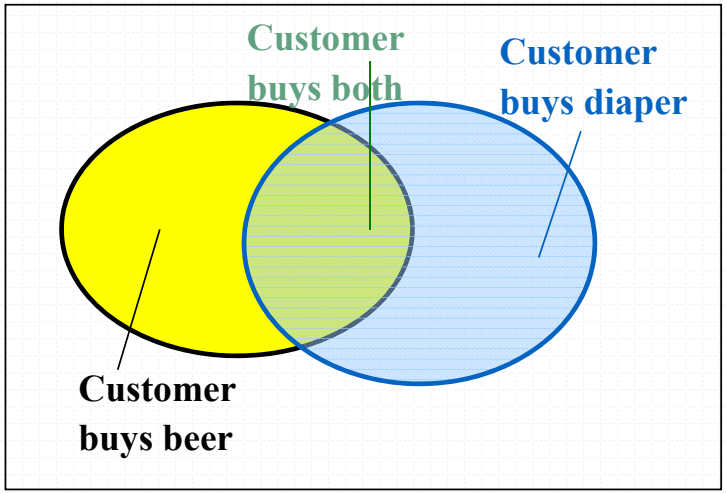
关联规则就是支持度和信任度分别满足用户给定阈值的规则。

发现关联规则需要经历如下两个步骤：

找出所有频繁项集。

由频繁项集生成满足最小信任度阈值的规则。

Transaction-id	Items bought
10	A, B, C
20	A, C
30	A, D
40	B, E, F



Let min_support = 50%, min_conf = 50%:

$A \rightarrow C$ (50%, 66.7%)

$C \rightarrow A$ (50%, 100%)

Transaction-id	Items bought
10	A, B, C
20	A, C
30	A, D
40	B, E, F

Min. support 50%
Min. confidence 50%

Frequent pattern	Support
{A}	75%
{B}	50%
{C}	50%
{A, C}	50%

For rule $A \Rightarrow C$:

support = support($\{A\} \cup \{C\}$) = 50%

confidence = support($\{A\} \cup \{C\}$)/support($\{A\}$) = 66.6%

4.2.1 Apriori（1/2）--- 算法思想与流程

■ 算法思想

□ 基于频繁项集性质的先验知识，使用由下至上逐层搜索的迭代方法，即从频繁1项集开始，采用频繁*k*项集搜索频繁*k*+1项集，直到不能找到包含更多项的频繁项集为止

■ 算法流程

$\text{Apriori}(T, \epsilon)$
$L_1 \leftarrow \{\text{large } 1-\text{itemsets}\}$
$k \leftarrow 2$
while $L_{k-1} \neq \emptyset$
$C_k \leftarrow \{c = a \cup \{b\} \mid a \in L_{k-1} \wedge b \notin a, \{s \subseteq c \mid |s| = k-1\} \subseteq L_{k-1}\}$
for transactions $t \in T$
$D_t \leftarrow \{c \in C_k \mid c \subseteq t\}$
for candidates $c \in D_t$
$count[c] \leftarrow count[c] + 1$
$L_k \leftarrow \{c \in C_k \mid count[c] \geq \epsilon\}$
$k \leftarrow k + 1$
return $\bigcup L_k$

Apriori算法的步骤

Apriori算法命名源于算法使用了频繁项集性质的先验（Prior）知识。

Apriori算法将发现关联规则的过程分为两个步骤：

通过迭代，检索出事务数据库中的所有频繁项集，即支持度不低于用户设定的阈值的项集；

利用频繁项集构造出满足用户最小信任度的规则。

挖掘或识别出所有频繁项集是该算法的核心，占整个计算量的大部分。

频繁项集

为了避免计算所有项集的支持度（实际上频繁项集只占很少一部分），**Apriori**算法引入潜在频繁项集的概念。若潜在频繁k项集的集合记为C_k，频繁k项集的集合记为L_k，m个项目构成的k项集的集合为C_m^k，则三者之间满足关系$L_k \subseteq C_k \subseteq C_m^k$。构成潜在频繁项集所遵循的原则是“频繁项集的子集必为频繁项集”。(m 为项目总的项目数)

关联规则的性质

性质1：频繁项集的子集必为频繁项集。

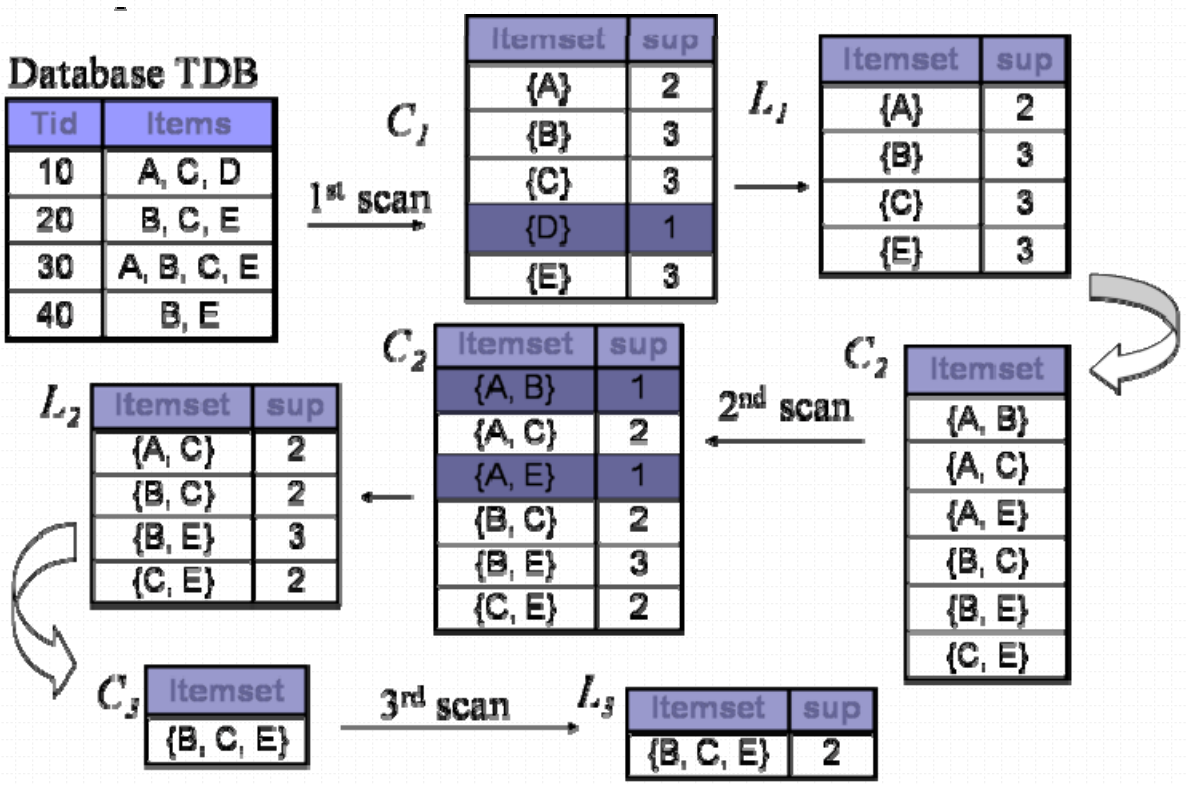
性质2：非频繁项集的超集一定是非频繁的。

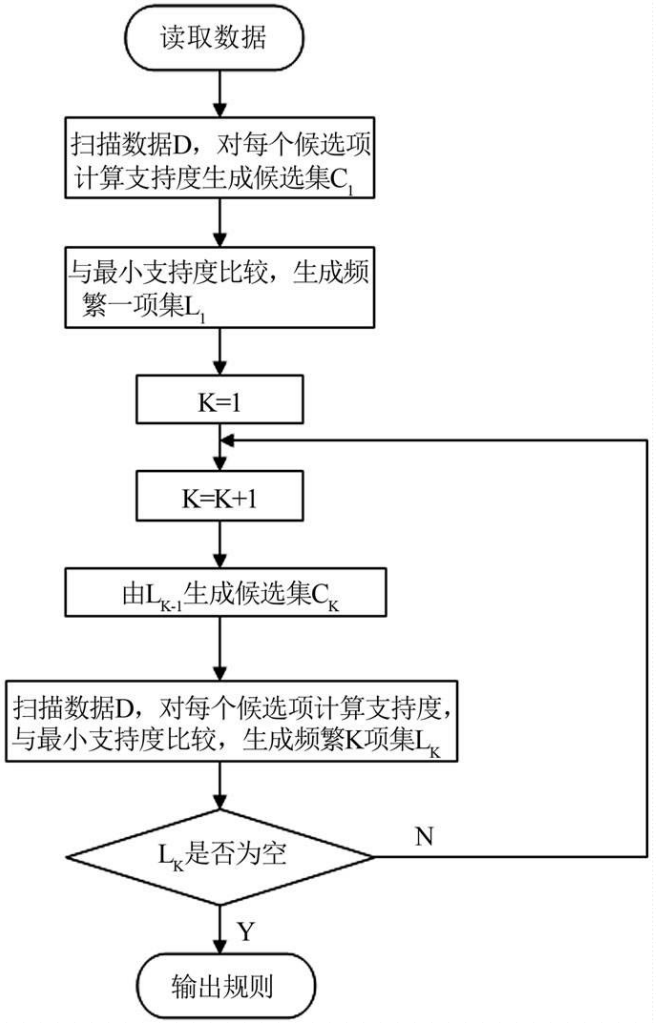
Apriori算法运用性质1，通过已知的频繁项集构成长度更大的项集，并将其称为潜在频繁项集。潜在频繁k项集的集合C_k是指由有可能成为频繁k项集的项集组成的集合。以后只需计算潜在频繁项集的支持度，而不必计算所有不同项集的支持度，因此在一定程度上减少了计算量。

Apriori算法

(1) L_1={频繁1项集};
(2) for(k=2;$L_{k-1}\neq\varnothing$;k++) do begin
(3) C_k=apriori_gen(L_{k-1}); //新的潜在频繁项集
(4) for all *transactions* $t\in D$ do begin
(5) C_t=subset(C_k,t); //t中包含的潜在频繁项集
(6) for all *candidates* $c\in C_t$ do
(7) *c.count*++;
(8) end;
(9) $L_k=\{c\in C_k|c.count\geq minsup\}$
(10) end;
(11) Answer= $\bigcup_k L_k$

4.2.1 Apriori（2/2）--- 算法实例





4.2.2 FP-Growth——算法思想与流程

- 算法思想
  - 频繁模式树增长算法（Frequent Pattern Tree Growth）采用分而治之的基本思想，将数据库中的频繁项集压缩到一棵频繁模式树中，同时保持项集之间的关联关系。然后将这棵压缩后的频繁模式树分成一些条件子树，每个条件子树对应一个频繁项，从而获得频繁项集，最后进行关联规则挖掘。与Apriori算法相比，FP-Growth算法只需要扫描数据库两次，效率更高。
- 算法流程[1]
  - **建立项头表**：里面记录了所有的1项频繁集出现的次数，按照次数降序排列。
  - **建立FP-Tree**：将原始数据集映射为内存中的一颗FP树，这棵树反映了频繁模式的关系。
  - **从FP-Tree挖掘模式**：从构建好的FP树里挖掘出频繁项集。

[1]参考自cnblogs：https://www.cnblogs.com/pinard/p/6307064.html

4.2.2 FP-Growth——项头表的建立

- 第一次扫描数据，得到所有频繁一项集的的计数。然后删除支持度低于阈值的项，将1项频繁集放入项头表，并按照支持度降序排列。
- 如右图所示，我们有10条数据，首先第一次扫描数据并对1项集计数，我们发现O，I，L，J，P，M，N都只出现一次，支持度低于20%的阈值，因此他们不会出现在下面的项头表中。剩下的A,C,E,G,B,D,F按照支持度的大小降序排列，组成了我们的项头表。
- 第二次扫描数据，对于每条数据剔除非频繁1项集，并按照支持度降序排列。
- 例如：第一条记录：A B C E F O提出非频繁1项集并排序之后，变为：A C E B F

数据
A B C E F O
A C G
E I
A C D E G
A C E G L
E J
A B C E F P
A C D
A C E G M
A C E G N

项头表.
支持度大于20%

A:8	
C:8	
E:8	
G:5	
B:2	
D:2	
F:2	

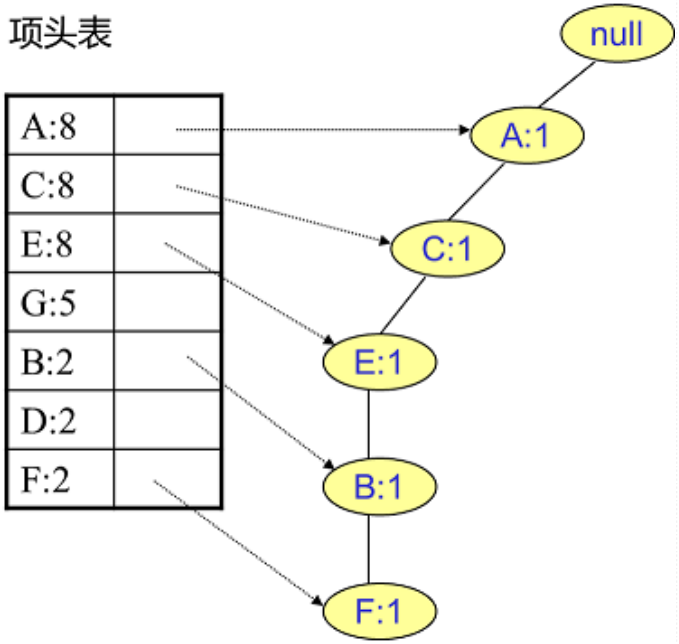
排序后的数据集
A C E B F
A C G
E
A C E G D
A C E G
E
A C E B F
A C D
A C E G
A C E G

4.2.2 FP-Growth——FP-Tree的建立

- 基本流程：逐条读入排序后的数据集，按顺序插入FP树，并对FP树中的节点进行计数。若插入后有新节点出现，则项头表对应的节点会通过节点链表链接上新节点。直到所有的数据都插入到FP树后，FP树的建立完成。

- 首先，我们插入第一条数据ACEBF，如下图所示。此时FP树没有节点，因此ACEBF是一个独立的路径，所有节点计数为1, 项头表通过节点链表链接上对应的新增节点。

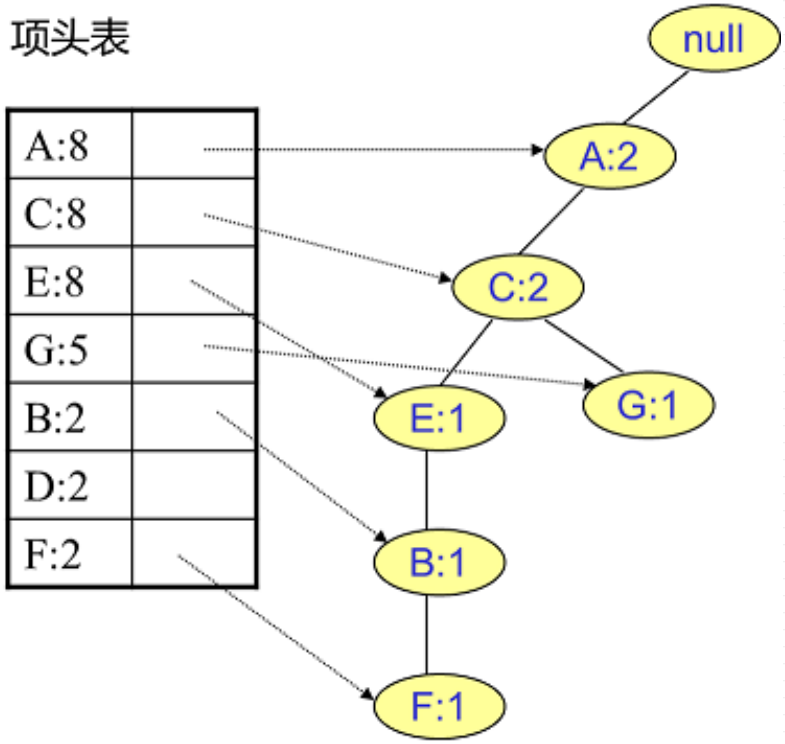
A C E B F
A C G
E
A C E G D
A C E G
E
A C E B F
A C D
A C E G
A C E G



4.2.2 FP-Growth——FP-Tree的建立

- 接着插入数据ACG，如图所示。由于ACG和现有的FP树可以有共有的祖先节点序列AC，因此只需要增加一个新节点G，将新节点G的计数记为1。同时A和C的计数加1成为2。当然，对应的G节点的节点链表要更新

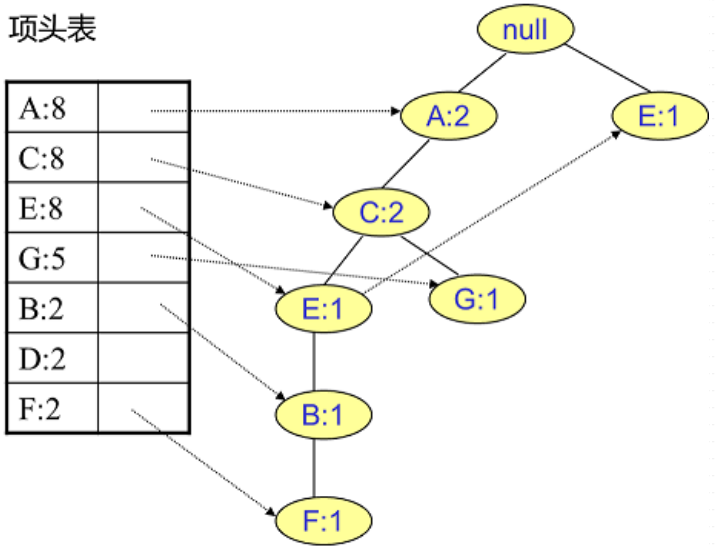
A C E B F
A C G
E
A C E G D
A C E G
E
A C E B F
A C D
A C E G
A C E G



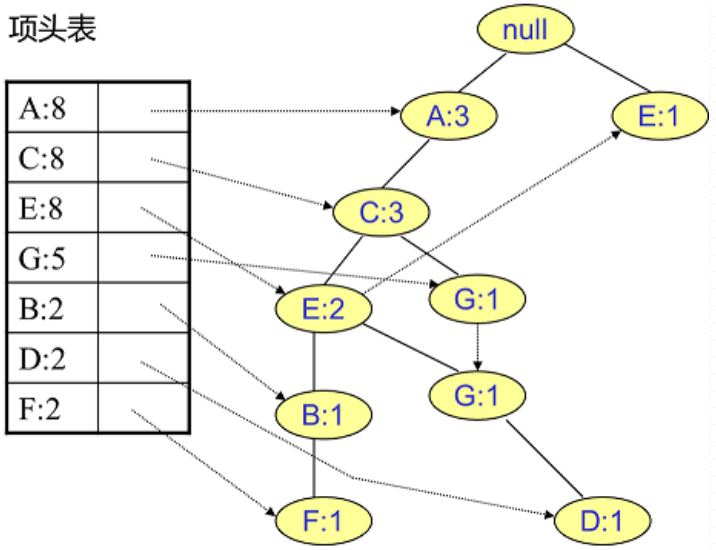
4.2.2 FP-Growth——FP-Tree的建立

- 同样的办法可以更新后面8条数据

A C E B F
A C G
E
A C E G D
A C E G
E
A C E B F
A C D
A C E G
A C E G



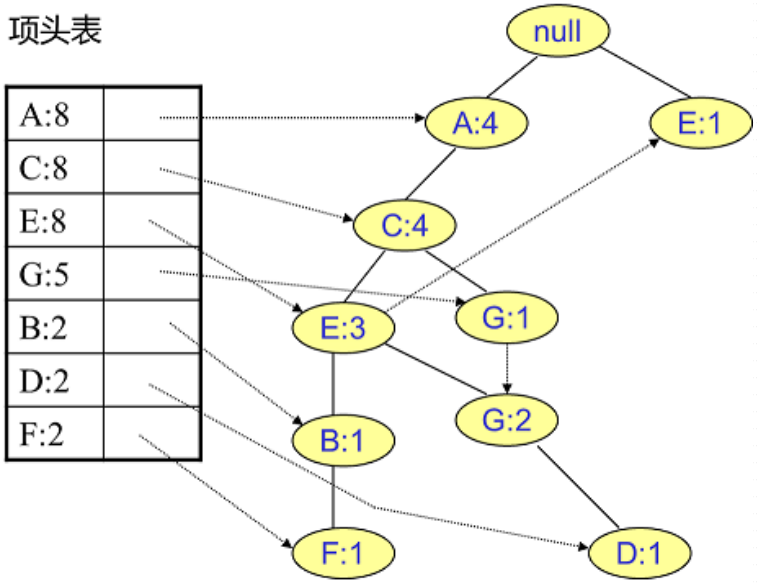
A C E B F
A C G
E
A C E G D
A C E G
E
A C E B F
A C D
A C E G
A C E G



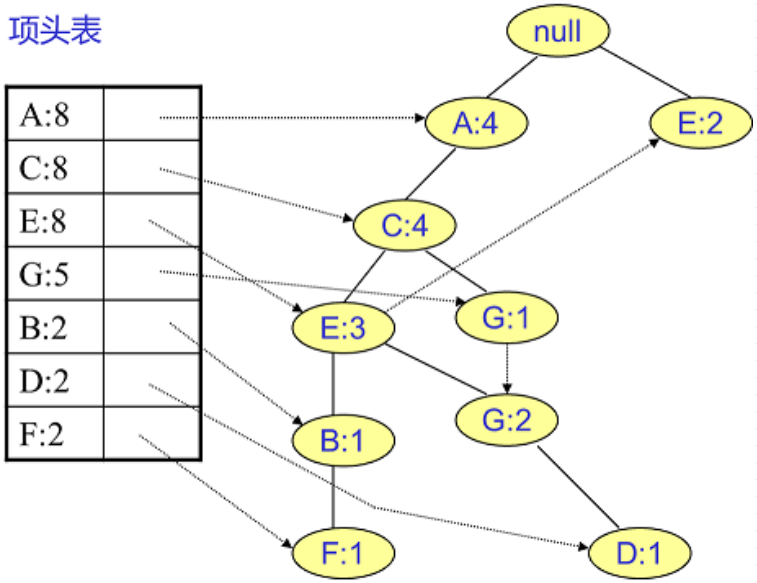
4.2.2 FP-Growth——FP-Tree的建立

- 同样的办法可以更新后面8条数据

A C E B F
A C G
E
A C E G D
A C E G
E
A C E B F
A C D
A C E G
A C E G



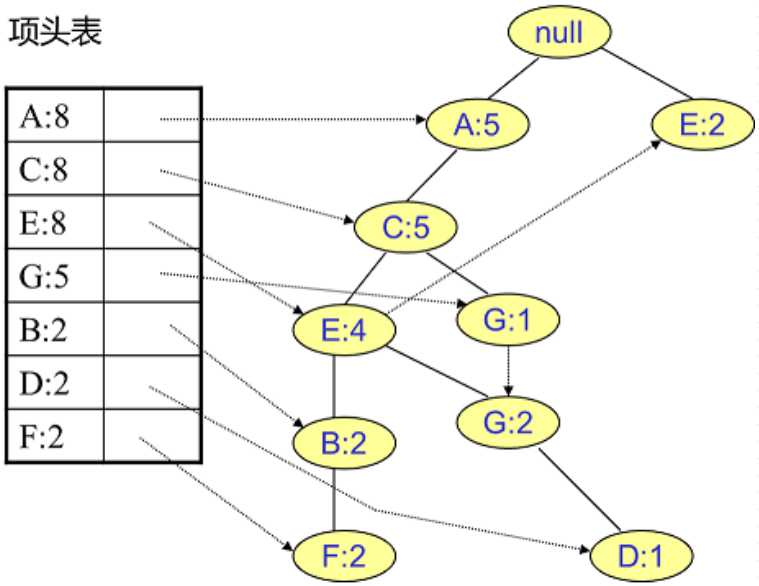
A C E B F
A C G
E
A C E G D
A C E G
E
A C E B F
A C D
A C E G
A C E G



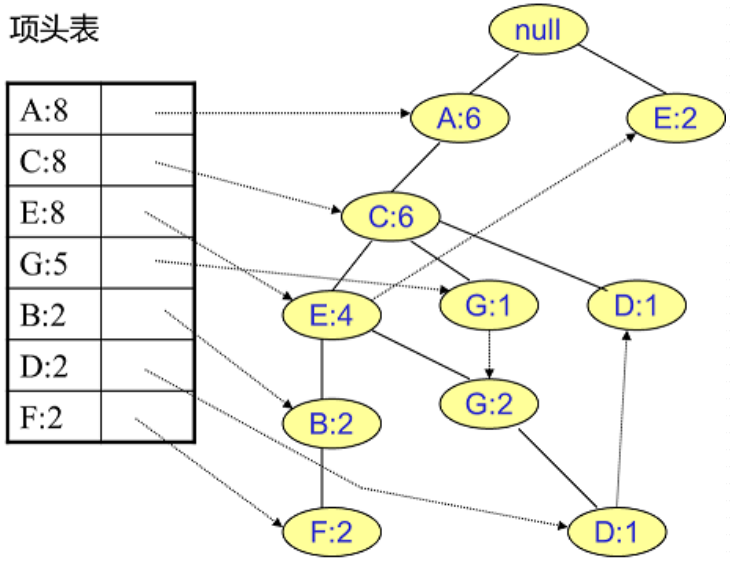
4.2.2 FP-Growth——FP-Tree的建立

- 同样的办法可以更新后面8条数据

A C E B F
A C G
E
A C E G D
A C E G
E
A C E B F
A C D
A C E G
A C E G



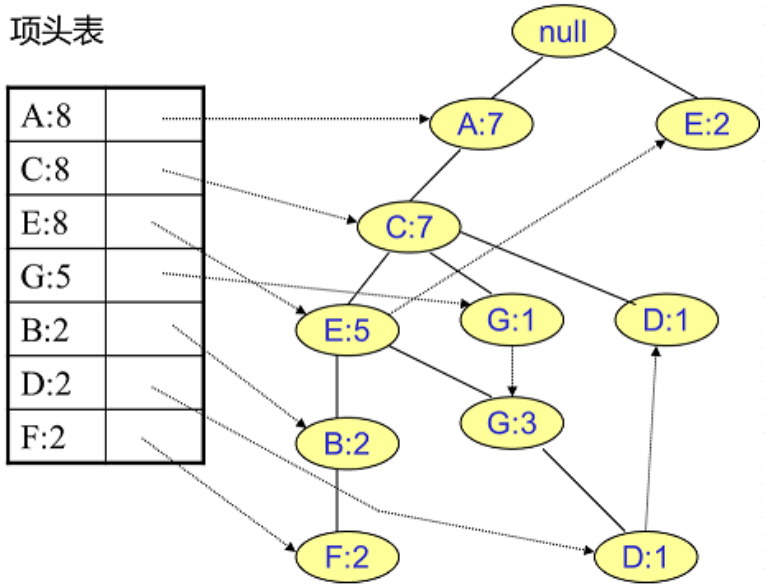
A C E B F
A C G
E
A C E G D
A C E G
E
A C E B F
A C D
A C E G
A C E G



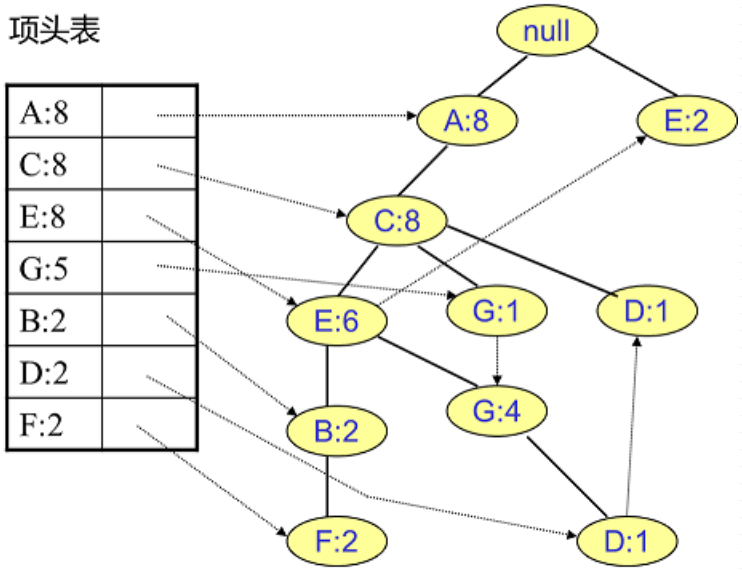
4.2.2 FP-Growth——FP-Tree的建立

- 同样的办法可以更新后面8条数据

A C E B F
A C G
E
A C E G D
A C E G
E
A C E B F
A C D
A C E G
A C E G

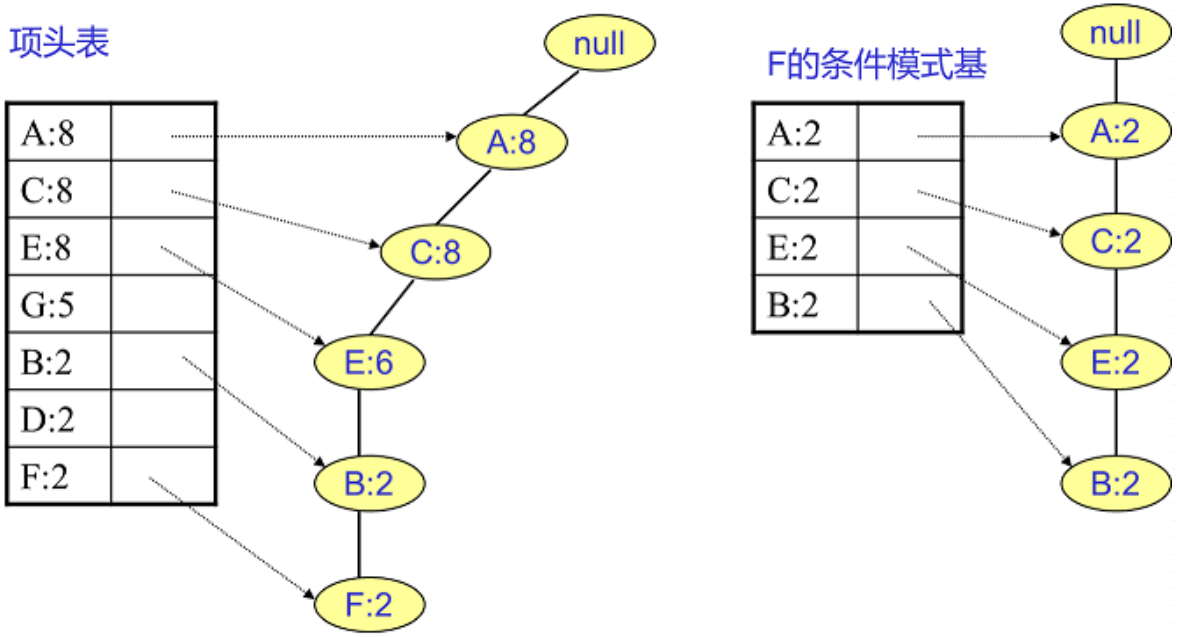


A C E B F
A C G
E
A C E G D
A C E G
E
A C E B F
A C D
A C E G
A C E G



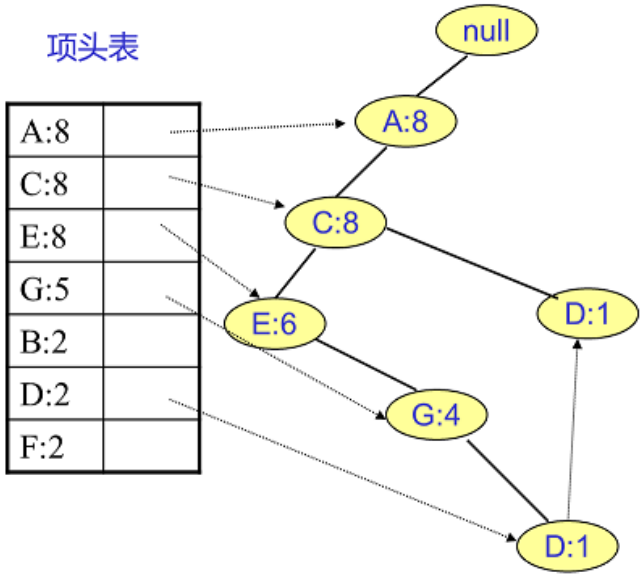
4.2.2 FP-Growth——FP-Tree的挖掘

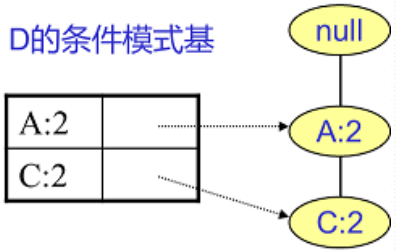
- 从最底下的F节点开始，先寻找F节点的条件模式基，由于F在FP树中只有一个节点，因此候选就只有下图左所示的一条路径，对应{A:8,C:8,E:6,B:2, F:2}。
- 接着将所有的祖先节点计数设置为叶子节点的计数，即FP子树变成{A:2,C:2,E:2,B:2, F:2}。
- 一般条件模式基可以不写叶子节点，因此最终的F的条件模式基如图右所示。

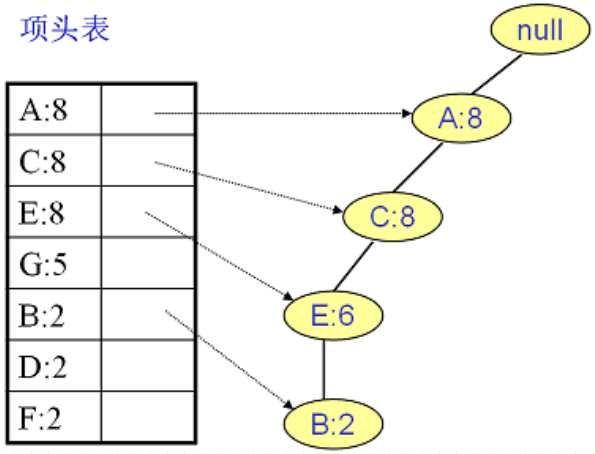


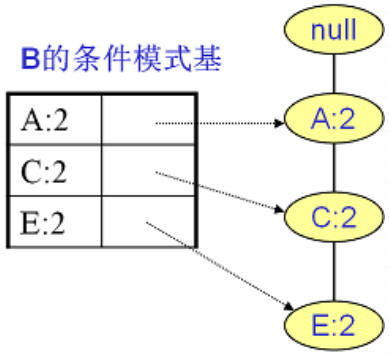
4.2.2 FP-Growth——FP-Tree的挖掘

- 采用类似的方法，挖掘其他模式



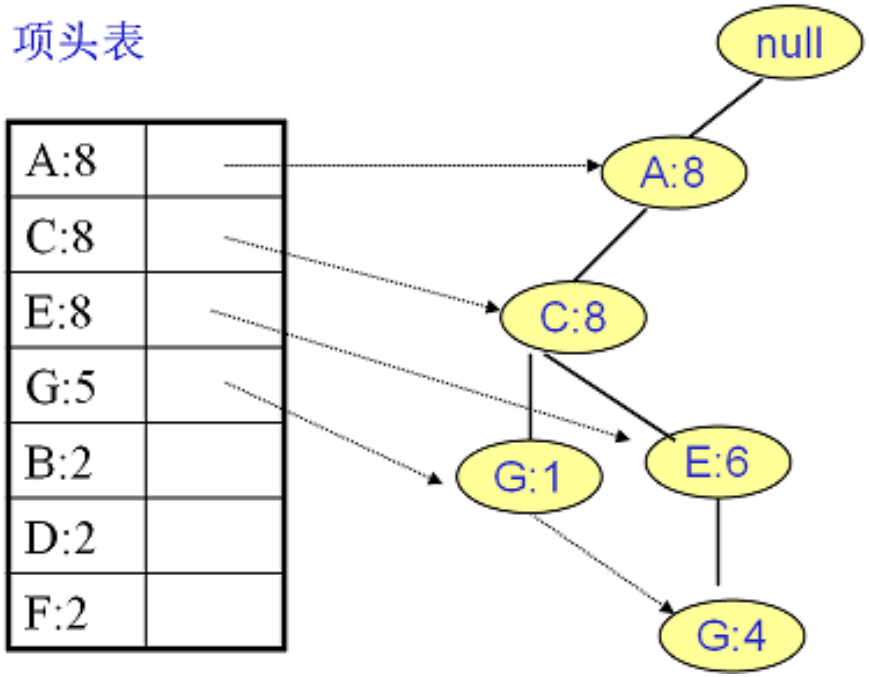


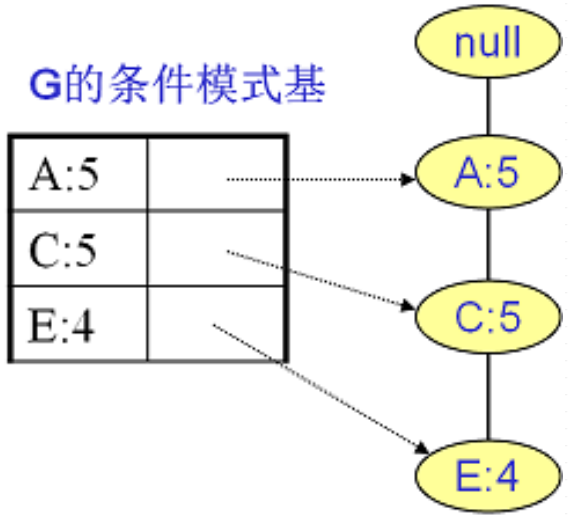




4.2.2 FP-Growth——FP-Tree的挖掘

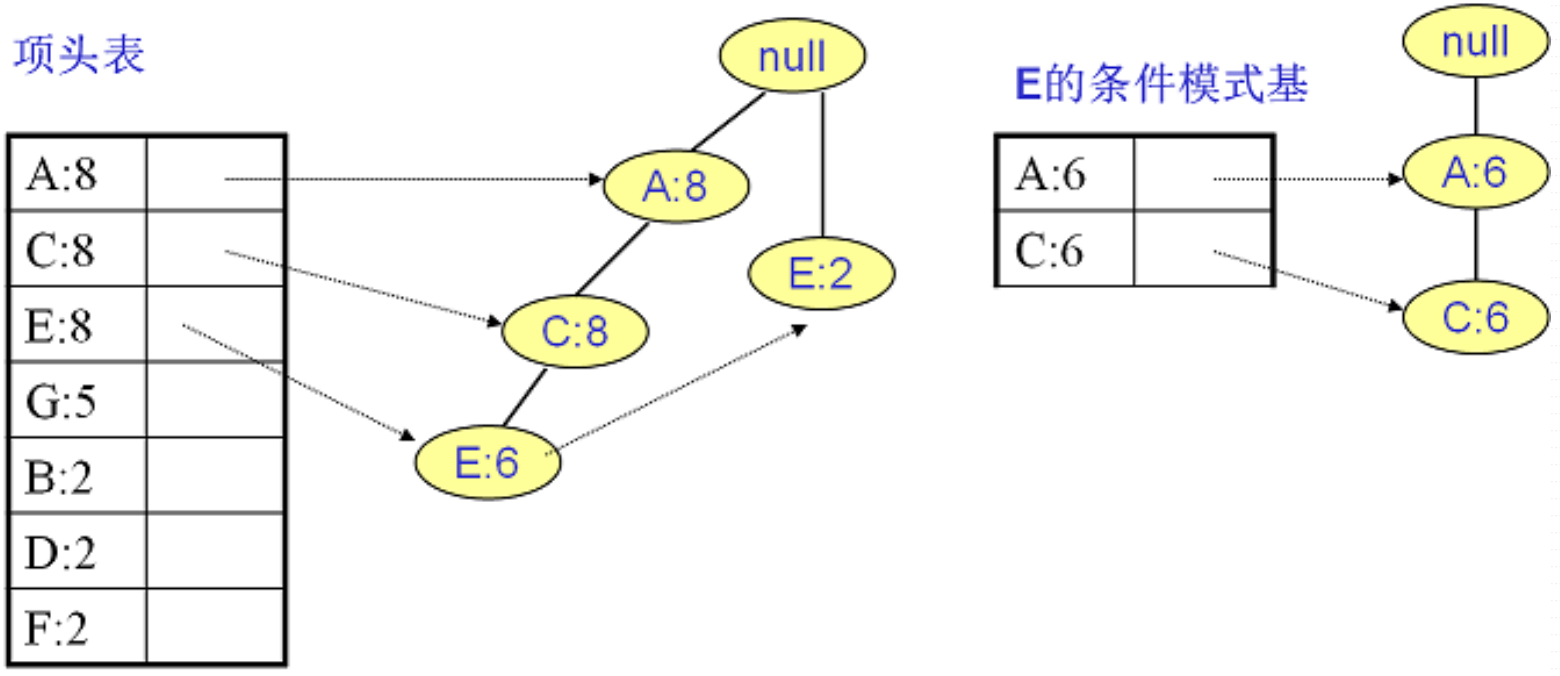
- 采用类似的方法，挖掘其他模式





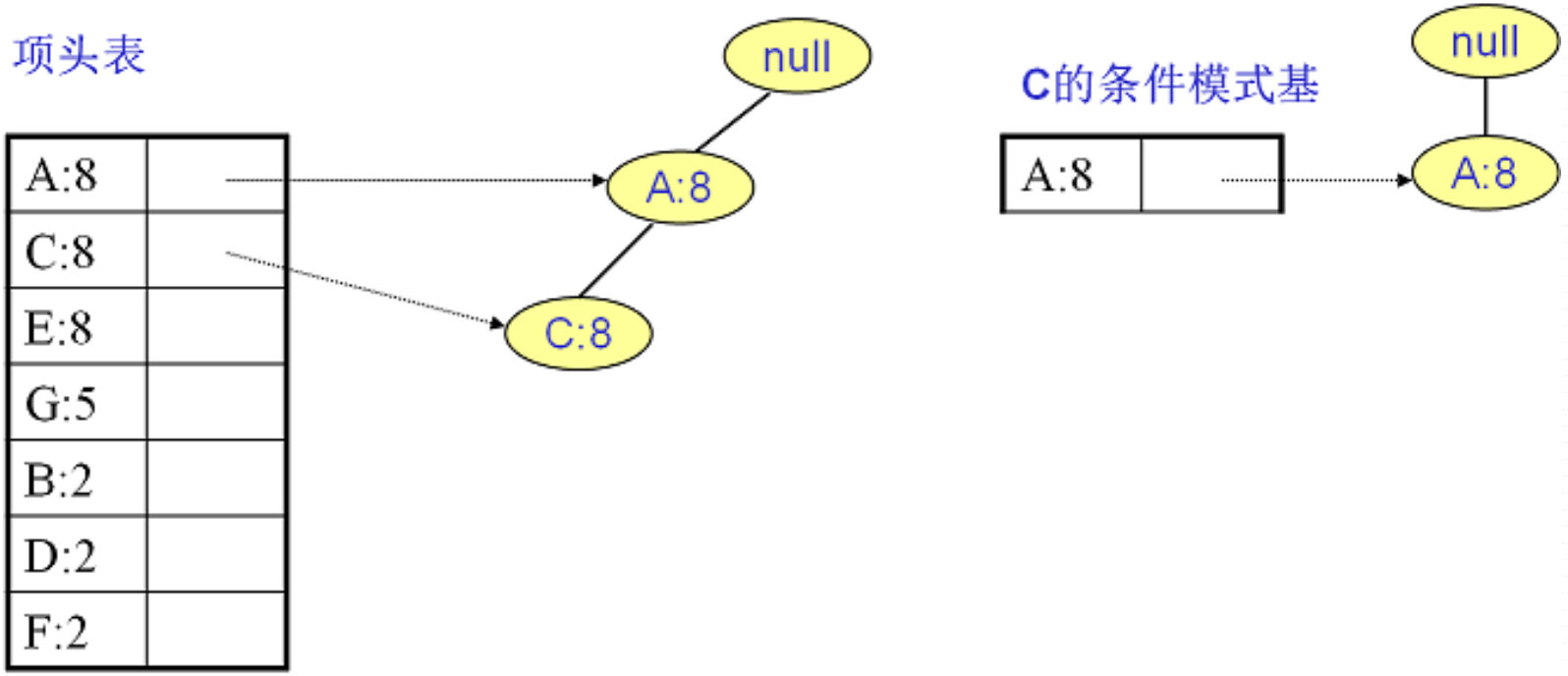
4.2.2 FP-Growth——FP-Tree的挖掘

- 采用类似的方法，挖掘其他模式

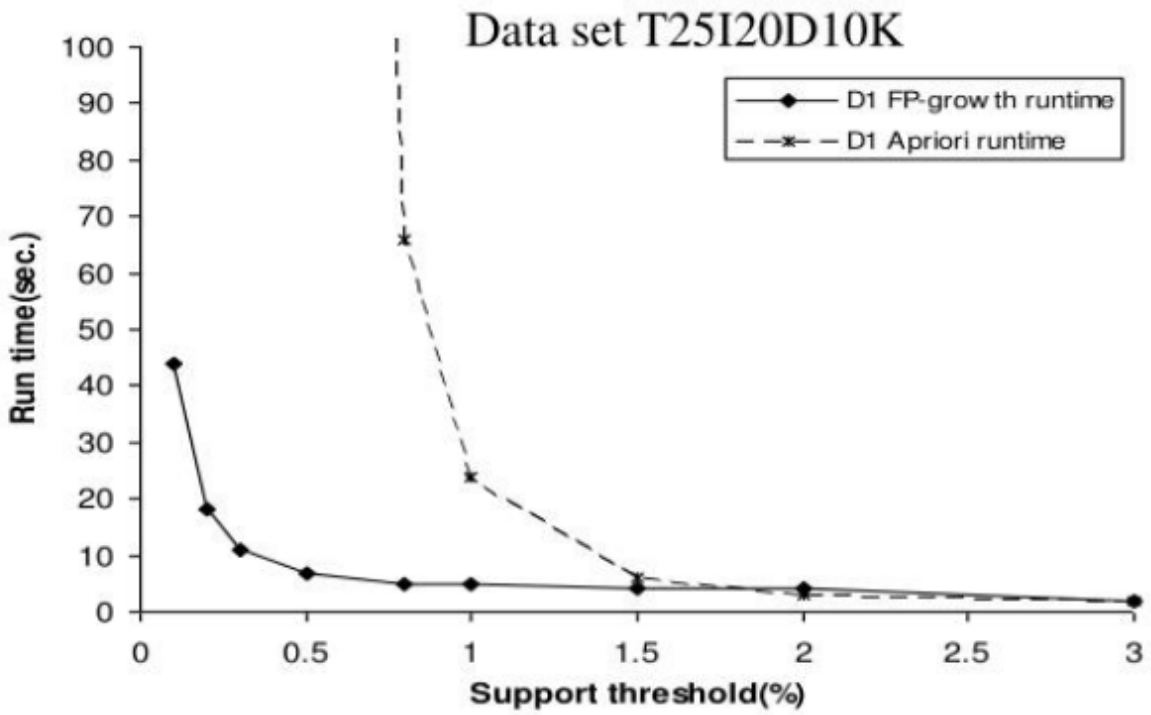


4.2.2 FP-Growth——FP-Tree的挖掘

- 采用类似的方法，挖掘其他模式



4.2.2 Apriori 与 FP-Growth 算法对比





大 数 据 算 法

CONTENTS

01 大数据挖掘算法概述
Introduction

02 分类算法
Sorting algorithms

03 聚类算法
Clustering algorithms

04 关联规则算法
Association rule algorithms

05 预测算法
Prediction algorithms

5.1 预测算法简介

- 预测分析
  - 一种统计或数据挖掘解决方案，包含可在结构化与非结构化数据中使用以确定未来结果的算法和技术，可为预测、优化、预报和模拟等许多其他相关用途而使用
- 时间序列预测
  - 一种历史资料延伸预测，以时间序列所能反映的社会经济现象的发展过程和规律性，进行引申外推预测发展趋势的方法
- 神经网络预测
  - 神经网络是强大的建模工具，可复制极其复杂的关系。神经网络本质上为非线性。很多年以来，线性建模一直是大多数领域的主要建模方法，因为它的优化程序非常完善。在线性近似不充分的任务中，线性模型运行状况不佳。除此之外，神经网络还解决了“维数灾难”问题，即当存在大量变量时不允许对线性关系进行建模的问题

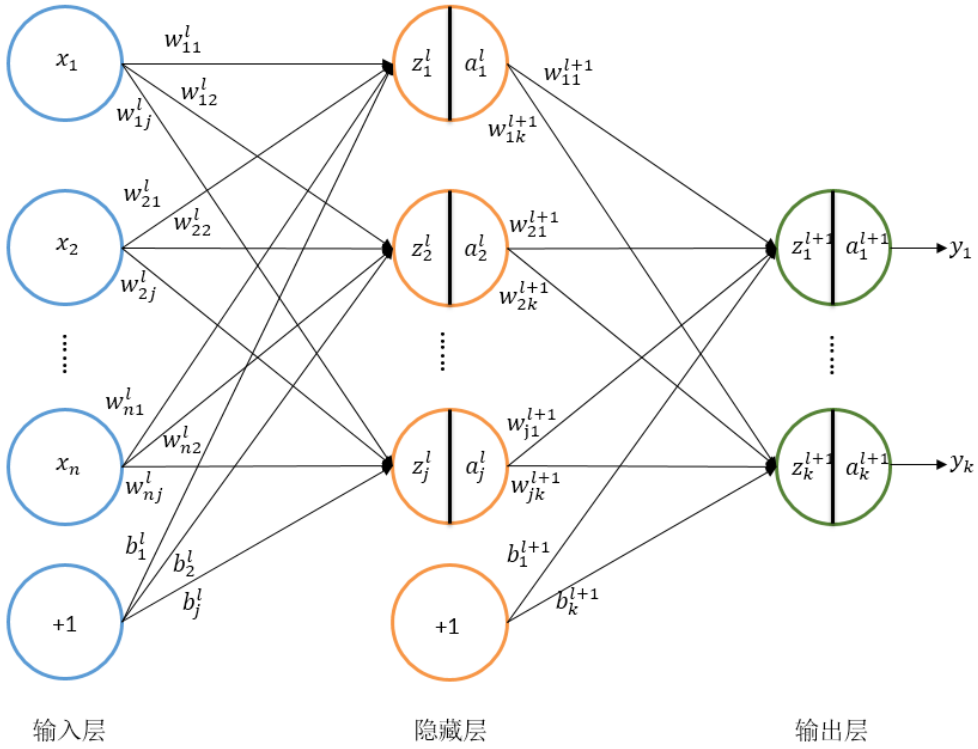
人工神经网络的特点

1、人工神经网络是一个非线性的有向图，图中含有可以通过改变权值大小来存放模式的加权边，并且可以从不完整的或未知的输入找到模式。
2、人工神经网络可以根据所在的环境去改变它的行为。
3、不同的人工神经网络模型，有不同的学习/训练算法。
4、由于其运算的不精确性，表现成“去噪音、容残缺”的能力，利用这种不精确性，实现模式的自动分类。
5、信息的分布存储提供容错功能
1)当其中某一个或某几个点被破坏时，信息仍可被存取
2)系统在受到局部损伤时还可以正常工作
3)对一类网络来说，当它完成学习后，如果再让它学习新的东西，这时就会破坏原来已学会的东西。

5.2 BP 神经网络---算法思想

■ BP神经网络

- BP神经网络是一种多层前馈神经网络，该网络的主要特点是：信号前向传递，误差反向传播
- 前向传递
  - 前向传递中，输入信号从输入层经隐含层处理，直至输出层，每一层的神经元状态只影响下一层神经元状态。如果输出层得不到期望输出，则转入反向传播
- 反向传播
  - 根据预测误差调整网络权值和阈值，使BP神经网络预测输出不断逼近期望输出



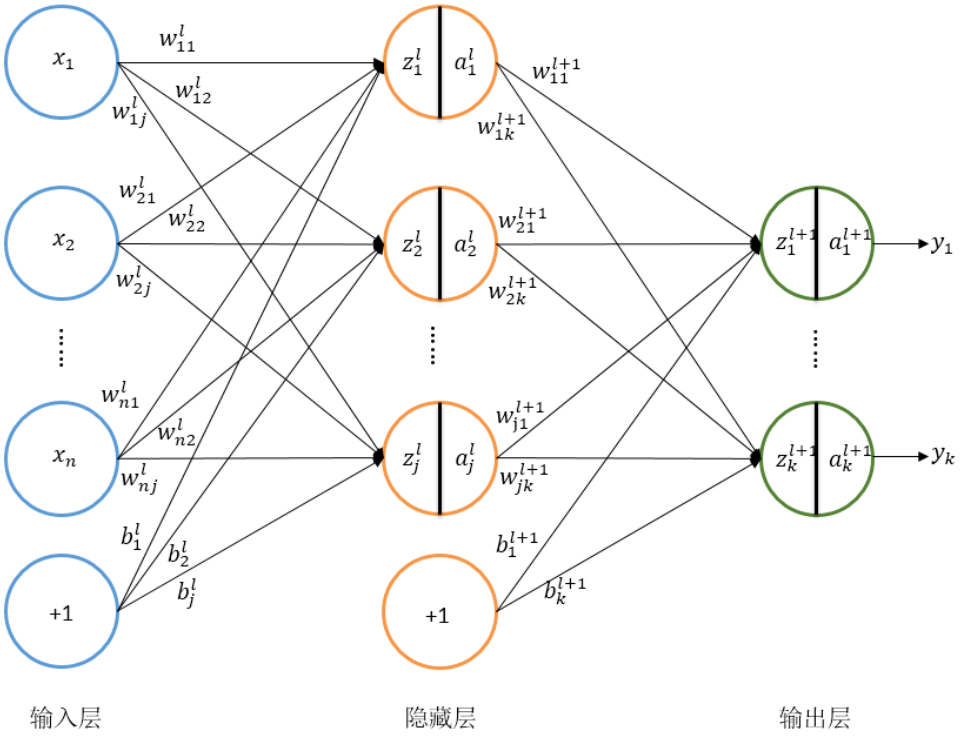
5.2.1 前向传播过程

- 神经元输出值a
  - 除了输入层的神经元，每个神经元都会有加权求和得到的输入值 z 和将 z 通过 Sigmoid 函数（也即是激活函数）非线性转化后的输出值 a

$$z_j^l = \sum_{i=1\ldots n} w_{ij} \cdot a_{ij}^{l-1} - b_j^l$$

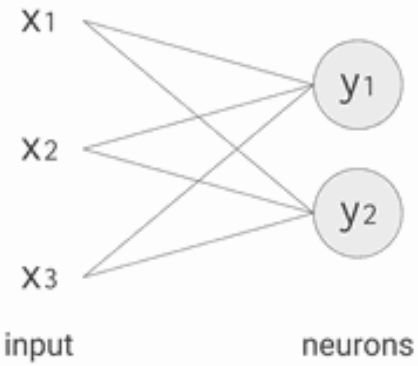
$$a_j^l = f\left(z_j^{(l)}\right) = \frac{1}{1 + e^{-z_j^{(l)}}}$$

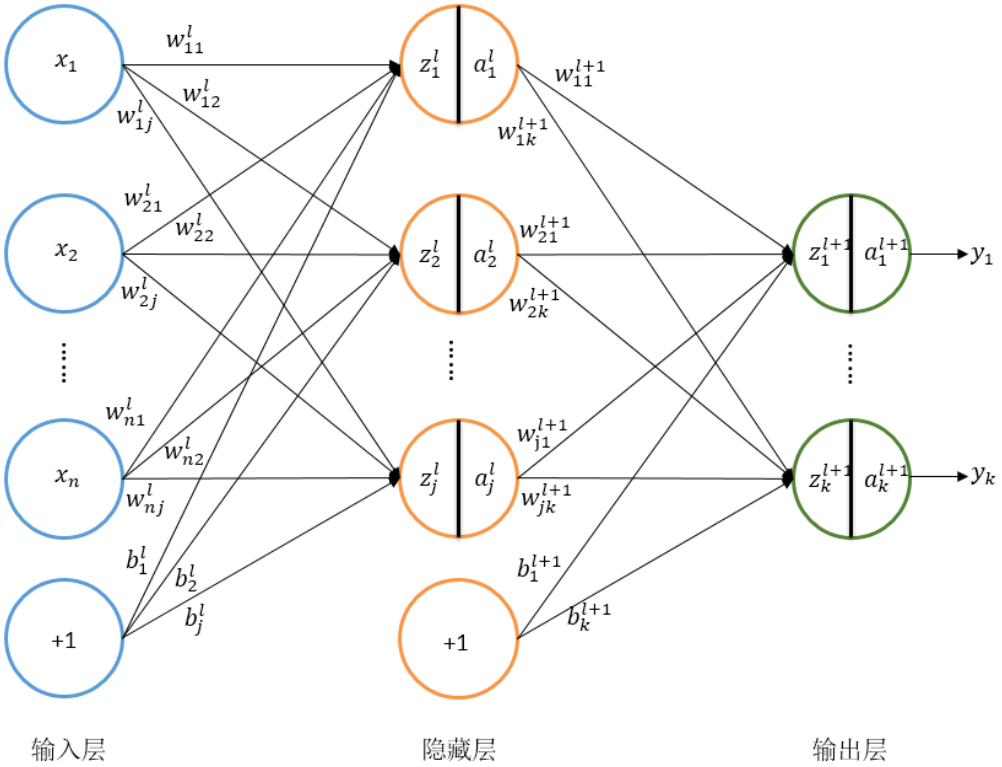
- l 和 j：表示的是第 l 层的第 j 个神经元，
- ij ：表示从第 i 个神经元到第 j 个神经元之间的连线
- w ：表示的是权重；b： 表示的是偏置



5.2.1 前向传播过程

- 神经元输出值a
  - 下图为动图展示计算过程





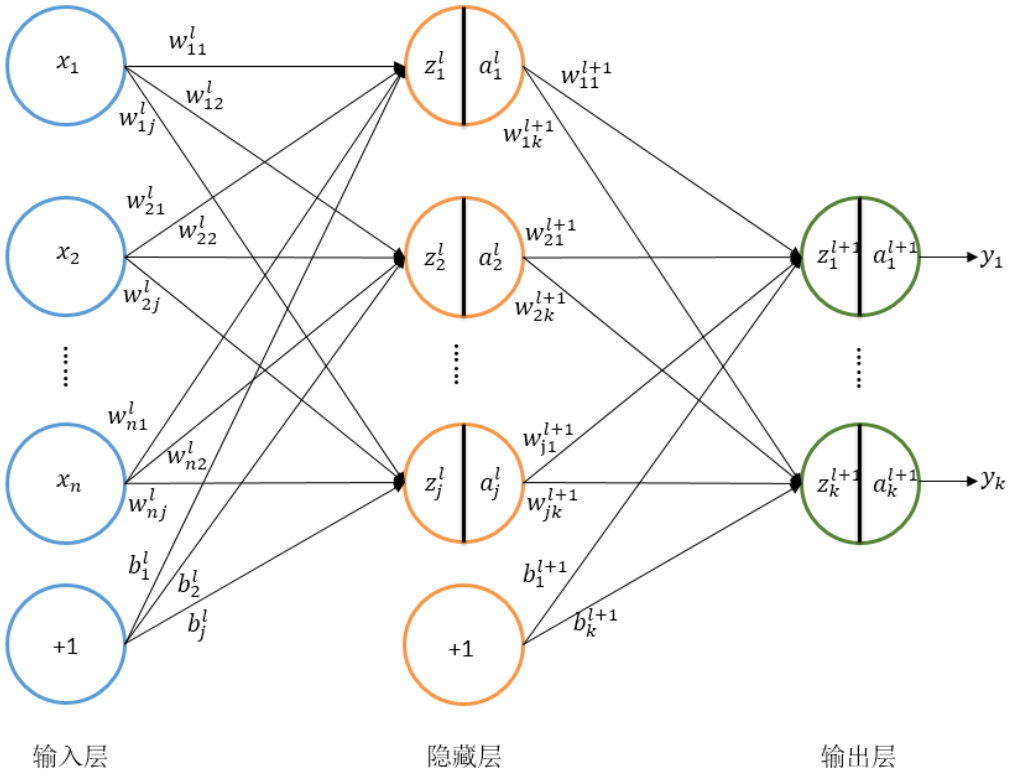
5.2.2 反向传播过程

- 损失函数
  - 因为输出层的值与真实的值会存在误差，我们可以用均方误差来衡量预测值和真实值之间的误差

$$E = \frac{1}{2}\sum_{j=1}^{l}(y_j^k - a_j^l)^2$$

- 反向传播目标
  - 让误差函数E的值尽可能的小，要让误差函数达到最小，因为误差函数实际只和w、b有关，所以就要调整w和b值， 使得误差函数的值最小

$$w = w - \nabla_w$$
$$b = b - \nabla_b$$



5.2.4 BP神经网络--算法实例

1. 确定网络结构

- 隐藏层数、激活函数、每层激活单元数..

2. 训练

- 根据已有数据集对网络进行训练，获取最佳网络权重，如一组昆虫数据集：

序号	触角	身长	攻击性
昆虫1	1.23	2.45	3.88
昆虫2	1.43	2.87	3.96
昆虫3	1.11	1.45	2.88
…	…	…	…

3. 预测

- 根据已经训练好的网络开始预测

序号	触角	身长	攻击性
昆虫n	3.23	4.45	?



大 数 据 算 法

CONTENTS

推荐算法

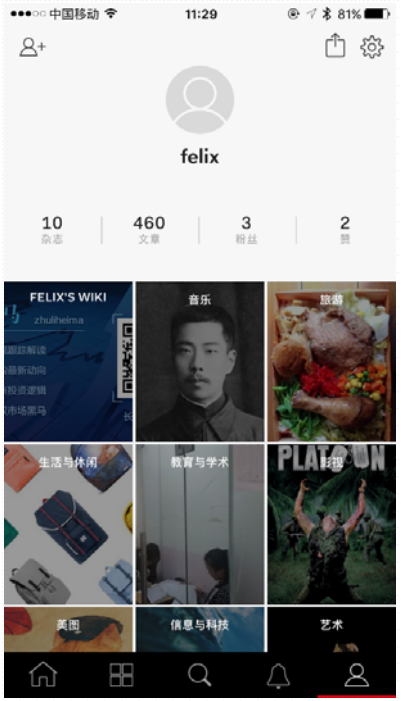
Recommendation algorithms

推荐无处不在



今日头条的新闻推荐

兴趣标签+习惯+地理位置



Flipboard的新闻推荐

兴趣标签+用户习惯+时间序列



网易的音乐推荐

收听习惯+音乐标签+口味



亚马逊商品推荐

用户行为

推荐无处不在



大众点评美食推荐

历史+地理位置+偏好



Apple的朋友推荐

地理位置+生态链



AppStore的软件推荐

地理位置+热门



百度糯米美食推荐

地理位置+搜索行为

推荐因素的定性分析

- 用户行为
  - 浏览行为
  - 收藏行为
  - 搜索行为
  - 点击次数
  - 用户评分
  - 用户评论
  - ……
- 兴趣标签：服务提供商提供的一种兴趣选项
- 地理位置：在社交行为上大多数用户侧重较近的地理位置
- 时间序列：时间通常也是重要的决策项，会影响社交和选择

推荐系统的应用分析

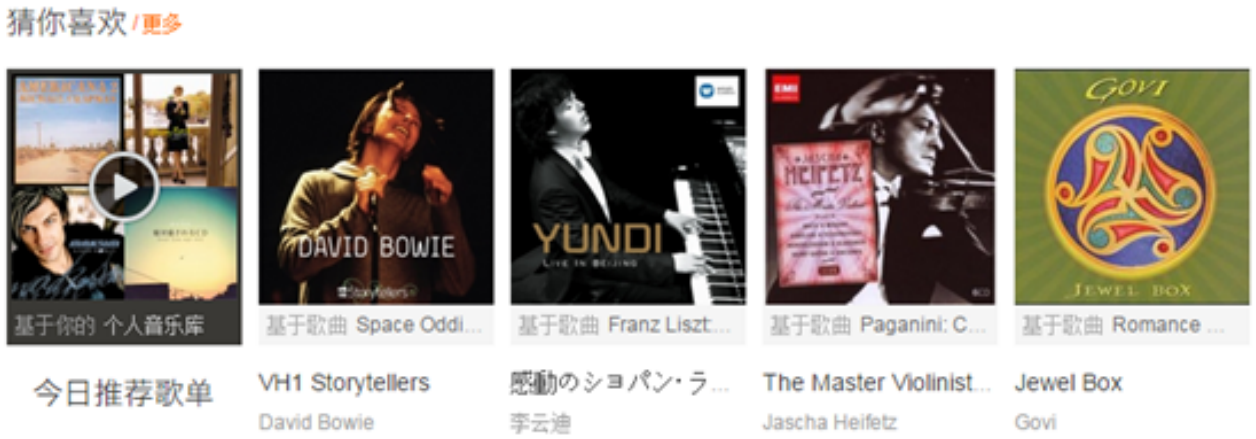
- 如亚马逊网站利用用户的浏览历史记录来为用户推荐商品，推荐的主要是用户未浏览过，但可能感兴趣、有潜在购买可能性的商品



亚马逊网站根据用户的浏览记录来推荐商品

»推荐系统的应用分析

- 音乐相比于电影数量更为庞大，个人口味偏向也更为明显，仅依靠热门推荐是远远不够的
- 虾米音乐网根据用户的音乐收藏记录来分析用户的音乐偏好，以进行推荐。例如，推荐同一风格的歌曲，或是推荐同一歌手的其他歌曲



虾米音乐网根据用户的音乐收藏来推荐歌曲

推荐系统概述

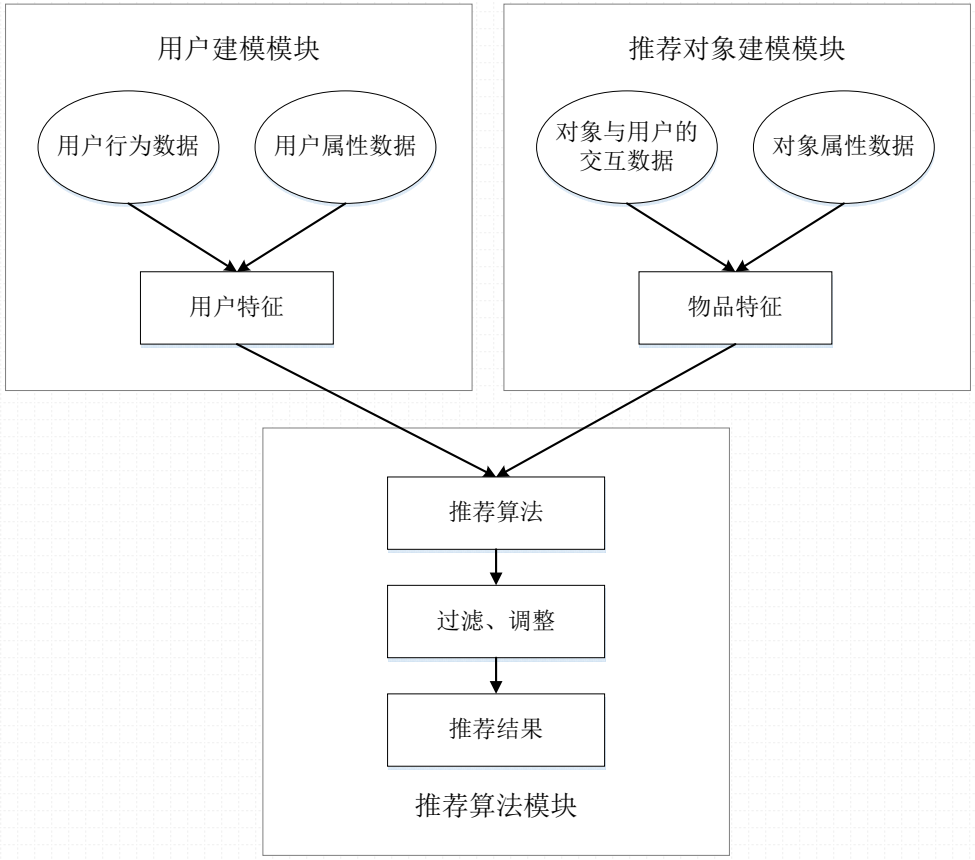
- 互联网的发展已使我们进入了信息过载的时代，搜索引擎可以帮助我们查找内容，但只能解决明确的需求
- 推荐系统的产生是为了让用户从海量信息中高效地获得自己所需的信息
- 推荐系统是大数据在互联网领域的典型应用，它可以通过分析用户的历史记录来了解用户的喜好，从而主动为用户推荐其感兴趣的信息，满足用户的个性化推荐需求

推荐系统概述——长尾理论

- “长尾”概念于2004年提出，用来描述以亚马逊为代表的电子商务网站的商业和经济模式
- 电子商务网站销售种类繁多，虽然绝大多数商品都不热门，但这些不热门的商品总数量极其庞大，所累计的总销售额将是一个可观的数字，也许会超过热门商品所带来的销售额
- 因此，可以通过发掘长尾商品并推荐给感兴趣的用户来提高销售额。这需要通过个性化推荐来实现
- 热门推荐是常用的推荐方式，广泛应用于各类网站中，如热门排行榜。但热门推荐的主要缺陷在于推荐的范围有限，所推荐的内容在一定时期内也相对固定
- 个性化推荐可通过推荐系统来实现。推荐系统通过发掘用户的行为记录，找到用户的个性化需求，发现用户潜在的消费倾向，从而将长尾商品准确地推荐给需要它的用户，进而提升销量，实现用户与商家的双赢

推荐系统概述——推荐系统的基本模型

- 一个完整的推荐系统通常包括3个组成模块：用户建模模块、推荐对象建模模块、推荐算法模块：
  - 用户建模模块：对用户进行建模，根据用户行为数据和用户属性数据来分析用户的兴趣和需求
  - 推荐对象建模模块：根据对象数据对推荐对象进行建模
  - 推荐算法模块：基于用户特征和物品特征，采用推荐算法计算得到用户可能感兴趣的对象，并根据推荐场景对推荐结果进行一定调整，将推荐结果最终展示给用户



推荐系统基本架构

»经典的推荐算法——协同过滤

- 推荐技术从被提出到现在已有十余年，在多年的发展历程中诞生了很多新的推荐算法。协同过滤作为最早、最知名的推荐算法，不仅在学术界得到了深入研究，而且至今在业界仍有广泛的应用
- 协同过滤可分为基于用户的协同过滤和基于物品的协同过滤
  - 基于用户的协同过滤（UserCF）
  - 基于物品的协同过滤（ItemCF）
  - UserCF算法和ItemCF算法的对比

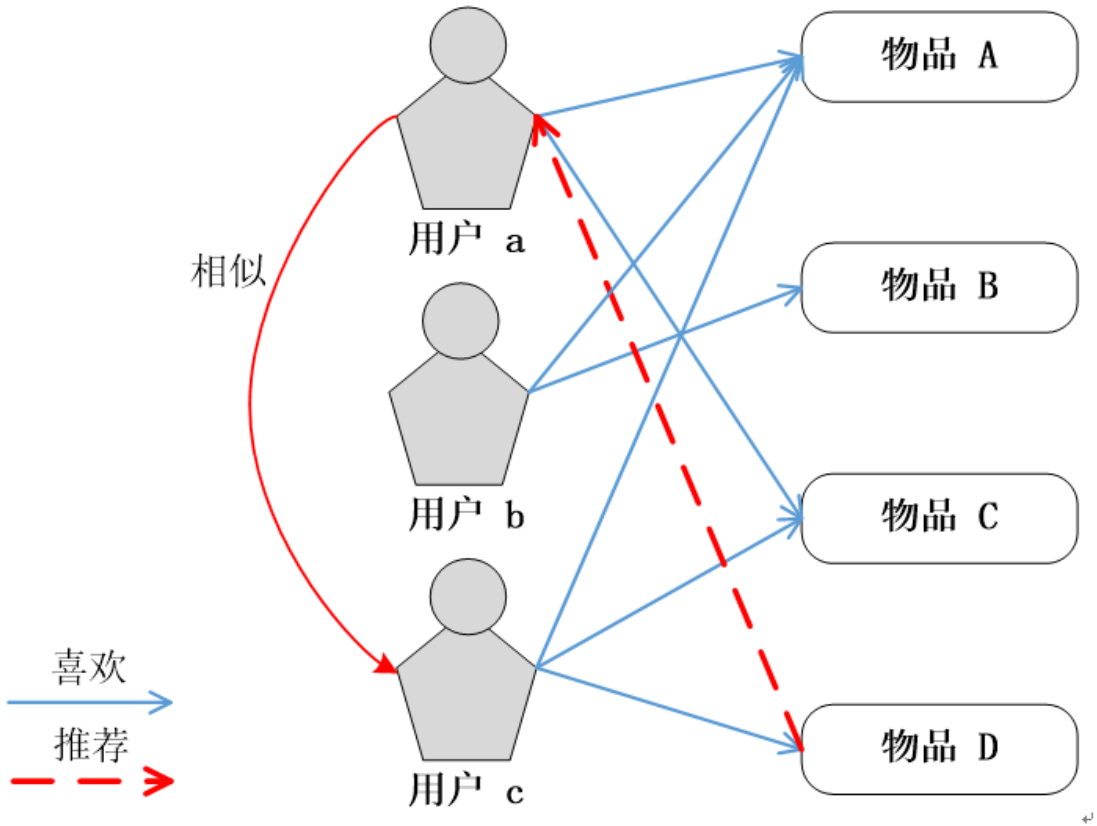
基于用户的协同过滤（UserCF）



重庆大学
CHONGQING UNIVERSITY

- 基于用户的协同过滤算法（简称UserCF算法）在1992年被提出，是推荐系统中最古老的算法
- UserCF算法符合人们对于“趣味相投”的认知，即兴趣相似的用户往往有相同的物品喜好：当目标用户需要个性化推荐时，可以先找到和目标用户有相似兴趣的用户群体，然后将这个用户群体喜欢的、而目标用户没有听说过的物品推荐给目标用户
- UserCF算法的实现主要包括两个步骤：
  - 第一步：找到和目标用户兴趣相似的用户集合
  - 第二步：找到该集合中的用户所喜欢的、且目标用户没有听说过的物品推荐给目标用户

基于用户的协同过滤（UserCF）



基于用户的协同过滤（User CF）

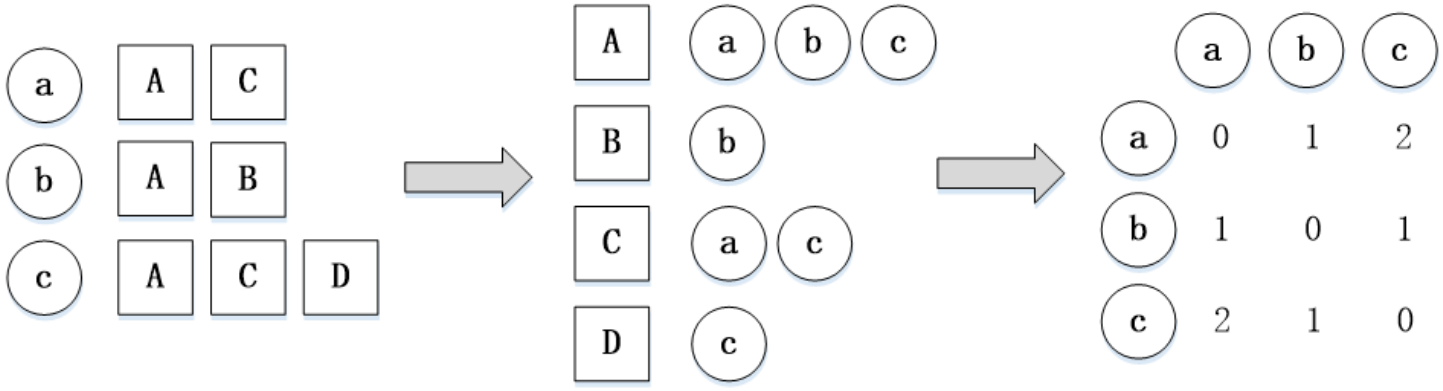
基于用户的协同过滤（UserCF）

- 实现UserCF算法的关键步骤是计算用户与用户之间的兴趣相似度。目前较多使用的相似度算法有：
  - 泊松相关系数（Person Correlation Coefficient）
  - 余弦相似度（Cosine-based Similarity）
  - 调整余弦相似度（Adjusted Cosine Similarity）
- 给定用户u和用户v，令N(u)表示用户u感兴趣的物品集合，令N(v)为用户v感兴趣的物品集合，则使用余弦相似度进行计算用户相似度的公式为：

$$w_{uv} = \frac{|N(u) \cap N(v)|}{\sqrt{|N(u)||N(v)|}}$$

基于用户的协同过滤（UserCF）

- 由于很多用户相互之间并没有对同样的物品产生过行为，因此其相似度公式的分子为0，相似度也为0
- 我们可以利用物品到用户的倒排表（每个物品所对应的、对该物品感兴趣的用户列表），仅对有对相同物品产生交互行为的用户进行计算



（a）用户喜欢的物品列表　（b）物品对应的用户列表　（b）相似度矩阵W

物品到用户倒排表及用户相似度矩阵

»基于用户的协同过滤（UserCF）

- 得到用户间的相似度后，再使用如下公式来度量用户u对物品i的兴趣程度P_{ui}：

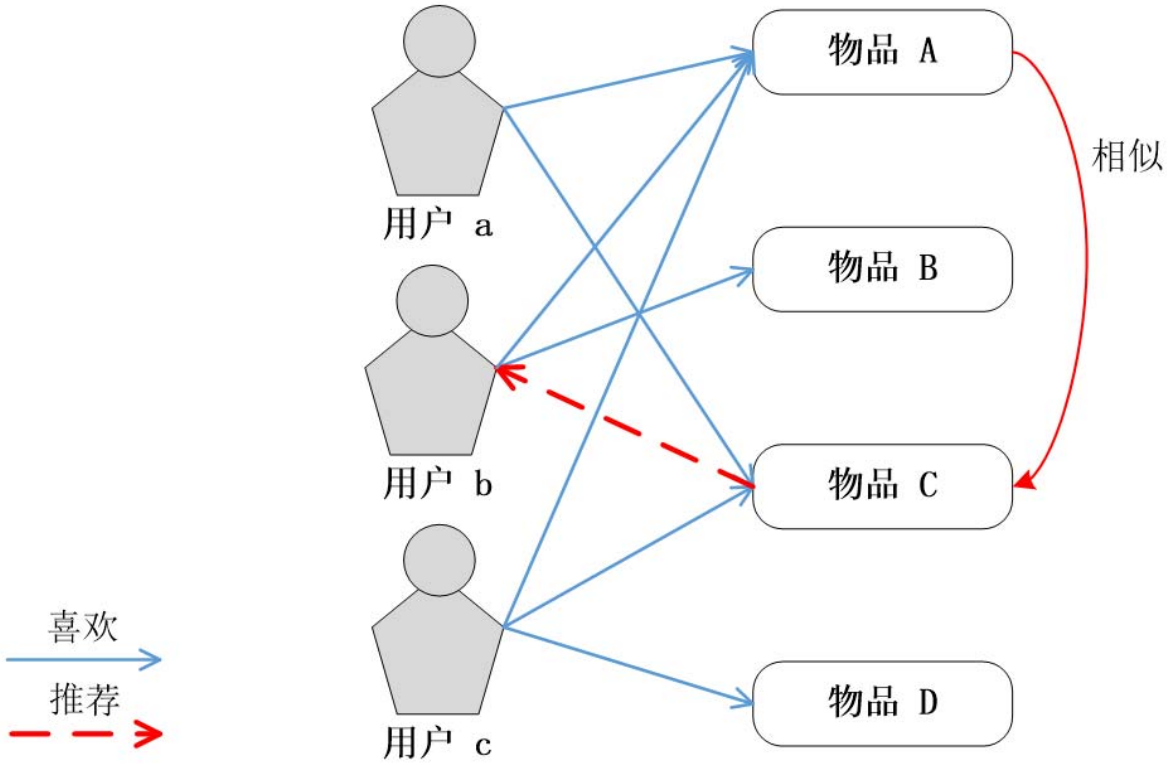
$$p(u,i)=\sum_{v\in S(u,K)\cap N(i)} w_{uv}r_{vi}$$

- 其中，S(u, K)是和用户u兴趣最接近的K个用户的集合，N(i)是喜欢物品i的用户集合，W_{uv}是用户u和用户v的相似度，r_{vi}是隐反馈信息，代表用户v对物品i的感兴趣程度，为简化计算可令r_{vi}=1
- 对所有物品计算P_{ui}后，可以对P_{ui}进行降序处理，取前N个物品作为推荐结果展示给用户u（称为Top-N推荐）

基于物品的协同过滤（ItemCF）

- 基于物品的协同过滤算法（简称ItemCF算法）是目前业界应用最多的算法。无论是亚马逊还是Netflix，其推荐系统的基础都是ItemCF算法，虽然Netflix已经用了全新的推荐算法，但是也经历了ItemCF算法阶段
- ItemCF算法是给目标用户推荐那些和他们之前喜欢的物品相似的物品。ItemCF算法主要通过分析用户的行为记录来计算物品之间的相似度
- 该算法基于的假设是：物品A和物品B具有很大的相似度是因为喜欢物品A的用户大多也喜欢物品B。例如，该算法会因为你购买过《数据挖掘》而给你推荐《人工智能》，因为买过《数据挖掘》的用户多数也购买了《人工智能》

基于物品的协同过滤（ItemCF）



基于物品的协同过滤（Item CF）

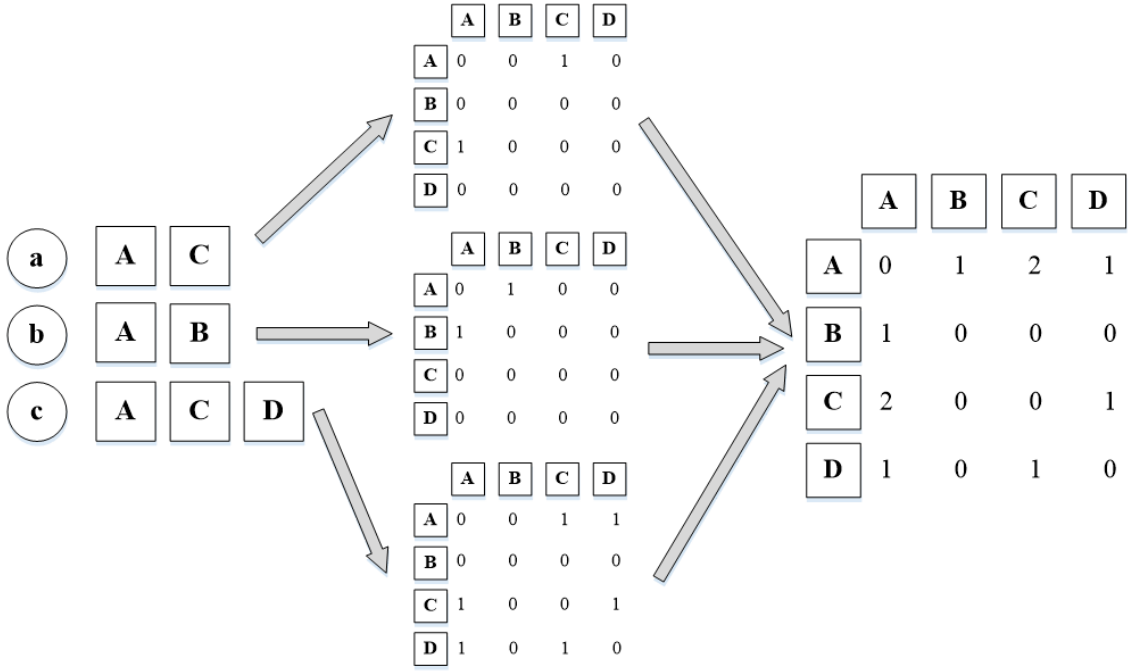
基于物品的协同过滤（ItemCF）

- ItemCF算法与UserCF算法类似，计算也分为两步：
  - 第一步：计算物品之间的相似度；
  - 第二步：根据物品的相似度和用户的历史行为，给用户生成推荐列表。
- ItemCF计算的是物品相似度，再使用如下公式来度量用户u对物品j的兴趣程度P_{uj}(与UserCF类似)：

$$p_{uj} = \sum_{i \in N(u) \cap S(j,K)} w_{ji} r_{ui}$$

基于物品的协同过滤（ItemCF）

- ItemCF算法通过建立用户到物品倒排表（每个用户喜欢的物品的列表）来计算物品相似度



（a）用户喜欢的物品列表　（b）物品相似度矩阵M　（c）物品相似度矩阵R

用户到物品倒排表及物品相似度矩阵

UserCF算法和ItemCF算法的对比

- UserCF算法和ItemCF算法的思想、计算过程都相似
- 两者最主要的区别：
  - UserCF算法推荐的是那些和目标用户有共同兴趣爱好的其他用户所喜欢的物品
  - ItemCF算法推荐的是那些和目标用户之前喜欢的物品类似的其他物品
- UserCF算法的推荐更偏向社会化：适合应用于新闻推荐、微博话题推荐等应用场景，其推荐结果在新颖性方面有一定的优势
  - UserCF缺点：随着用户数目的增大，用户相似度计算复杂度越来越高。而且UserCF推荐结果相关性较弱，难以对推荐结果作出解释，容易受大众影响而推荐热门物品
- ItemCF算法的推荐更偏向于个性化：适合应用于电子商务、电影、图书等应用场景，可以利用用户的历史行为给推荐结果作出解释，让用户更为信服推荐的效果
  - ItemCF缺点：倾向于推荐与用户已购买商品相似的商品，往往会出现多样性不足、推荐新颖度较低的问题



Thank You!